CIS 5210 Artificial Intelligence
Midterm Review Recitation
February 15, 2026

Benjamin Lam, Emily Paul, Dan Anafi, Shaun Khan

Slides created by Benjamin Lam, Elizabeth Jin, and Matthew Harrington

AGENDA

- Exam instructions and Logistics
- Study Guide and Review of topics
  - Python Skills
  - Rational Agents
  - (Un)informed Search
  - Adversarial Search
  - Constraint Satisfaction Problems
  - Logical Agents
- Q&A

LOGISTICS

EXAM INSTRUCTIONS

The midterm will cover modules 1-6 and general Python programming.

- The midterm will be a timed 90 minute exam. It consists of multiple choice questions and short-answer questions. No partial credit will be given for incorrect answers.
- The midterm is **open notes** with **4 pages** of handwritten or typed notes. That's **two sheets of normal paper (A4 or letter size)**, front and back. **10 point font minimum**.
- The midterm is a **closed book exam** and you cannot use any materials other than the notes you write. No textbook. No internet searches. No slides. No course videos. No Python compilers. No talking to friends.

EXAM INFORMATION #266

- Please be sure to review the Timed Exam 1 Instructions before you take your exam.
- There is a 15-minute buffer in case of technical issues. It will be available for you to take from **February 20th at 12:01 AM to February 22nd at 11:59 PM ET.** This exam will be proctored via Honorlock, as will the final exam. You can take the exam any time during this window, but you must start the exam with enough time to finish it before the deadline.
- You will be allowed short (5-minute) bathroom breaks during the exam. There is no need to notify Honorlock for such breaks, you can just get up and leave your desk. Please note that if you are away from your computer for more than 5 minutes, a live proctor will pop in to check on you.
- The exam will be four pages TOTAL (two pages front and back) of open notes, either typed in at least a 10 point font or handwritten. All notes must be put into one PDF and submitted to Gradescope here: https://www.gradescope.com/courses/1189673/assignments/7229636, before taking the exam - PLEASE ONLY SUBMIT TO GRADESCOPE ONCE. Failure to submit your notes prior - and still using them - will be considered a breach of academic integrity. **You must show these notes to the camera before beginning your exam.**

EXAM INFORMATION #266

- Please review Honorlock instructions, Getting Support, and FAQ sections prior to taking the exam: https://online.seas.upenn.edu/student-knowledge-base/honorlock/ (please log in with your Pennkey in order to access the Honorlock site)
- Penn Online staff members will be online for support from 9 AM ET to 9 PM ET on Friday, Saturday and Sunday. If you run into an issue whilst taking your exam outside the times previously provided, please use the Honorlock support feature ("Chat Support" tab on the bottom of the proctoring window).
- If you experience technical difficulties during the exam, please post a private message on Ed Discussion so course staff can assist you in real time. Only private posts are permitted during the exam. Please note that support will be limited to technical issues; content-related questions cannot be answered.
- PLEASE BE AWARE THAT ROOM SCANS ARE BACK AS OF THIS SEMESTER; SO, YOU WILL BE EXPECTED TO PROVIDE A 360 DEGREE VIEW OF YOUR ROOM AT THE START OF THE EXAM.

EXAM INFORMATION

- You should start your exam from Canvas, which will link you to the Gradescope exam.
- Please make sure you are being proctored by Honorlock through the Canvas Quiz prior opening DO NOT OPEN UNLESS BEING PROCTORED BY HONORLOCK - Timed Exam 1 - Spring 2026. You should have presented your ID, done a room scan, and also presented any notes in the accepted format.
- If not, please follow these steps:
  - 1. Stop working immediately.
  - 2. Create a private post on Ed Discussion for assistance.
    Support hours are 9AM-9PM ET.
  - 3. Wait for a response, then start your proctoring with Honorlock through Canvas and restart the exam.
- If you fail to do the above steps, this will be considered as a violation of academic integrity, and your submission may incur a substantial penalty / get zeroed out.

PRIVATE OFFICE HOUR CANCELLATION

- All office hours for this Friday–Tuesday (**February 20th to 24th inclusive**) the exam period for online students (with extension).
- Office hours will resume on Wednesday (**February 25th**).
- To ensure fairness in the exam, Ed Discussion will be blacked out from **February 20th to 24th** and will remain "private thread only" until all students have completed their exams
  - Jean will also be disabled during the exam period
  - For the Optional Wampa HW, we will provide some support on ED
  - We will not answer any other content-related questions

STUDY GUIDE

QUESTIONS OVERVIEW

Python Skills	5
Rational Agents	7
Uninformed Search	9
Informed Search	14
Adversarial Search	5
CSPs	10
Logical Agents	5
TOTAL	55 points

- Breakdown per topic:
  7 topics × (4–7) per topic = 37 total
  Questions are not necessarily in increasing difficulty per section, but points will reflect effort/ difficulty
- Breakdown per type (approximate):
  - 21 Multiple Choice
    - Exactly one option is correct
  - 3 Select All That Apply
    - 1 or more must be selected
    - we don't have all FALSE
  - 12 Short Answer
- Sample questions:
  - Definitions / Problem statement / Properties
  - Walking through code/ algorithms
  - Proofs

TOPICS COVERED

- Textbook readings
  - Chapter 1 "Introduction", Section 1.1 "What is AI"
  - Chapter 27 "Philosophical Foundations", Sections 27.1 "The Limits of AI" and 27.2 "Can Machines Really Think?"
  - Chapter 2 "Intelligent Agents" (whole chapter)
  - Chapter 3 "Problem Solving by Search" (3.1--3.6)
  - Chapter 5 "Adversarial Search" (5.1-5.3, 5.5)
  - Chapter 16 "Making Simple Decisions" (16.1-16.3)
  - Chapter 6 "Constraint Satisfaction Problems" (6.1-6.5)
  - Chapter 7 "Logical Agents", Sections 7.1-7.5

- Content Recitations and slides #15

- Homework implementation (your own)

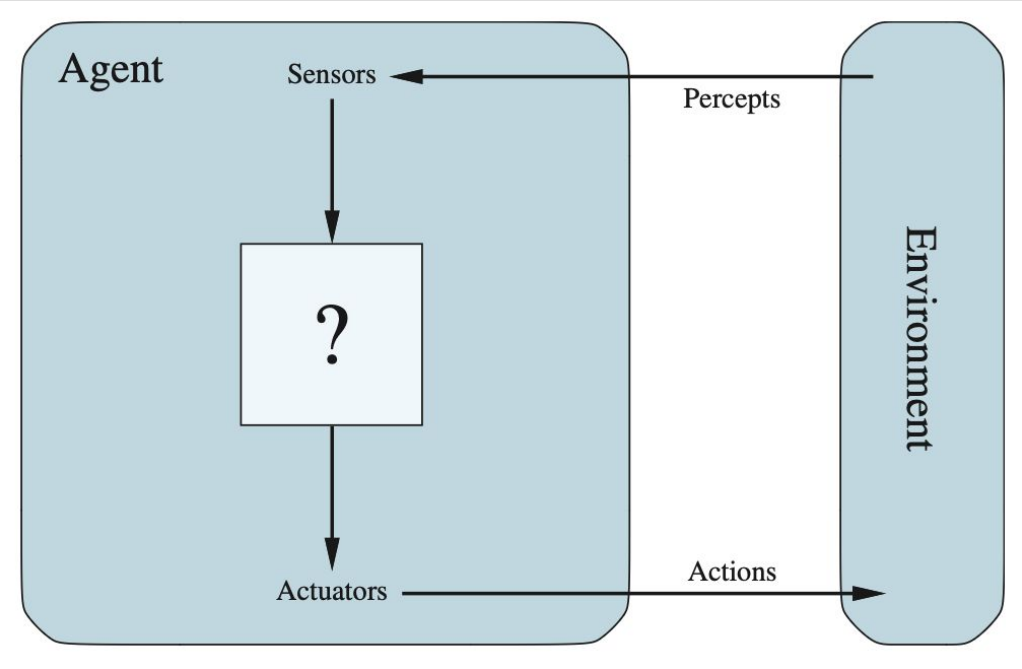
RATIONAL AGENTS

Week 1

AIMA Chapter 2 "Intelligent Agents" (2.1-2.4)

RATIONAL AGENTS

- Rationality: doing the right thing
  - Performance measure
- Agent function:
  - Simple reflex: current percept only
  - Model-based reflex: state, transition, and {percept, action} history
  - Goal-based: cf. Search
  - Utility-based: internalizing utility function/ performance measure
- PEAS: to specify a task environment
  - Performance measure
  - Environment
  - Actuators
  - Sensors



PYTHON SKILLS

QUESTION TYPES

- Walking through a Code snippet
  - Flow control: e.g. if, else, for, break, continue, pass
  - Recursion
  - Evaluation of expressions

- Types and Properties
  - Properties: (Un)ordered, (Im)mutable, (Un)hashable
  - e.g. List, Tuple, Sets, Dictionaries, Generators, PriorityQueue

USEFUL DATA STRUCTURES & ITERABLE OBJECTS

- Lists: Ordered collections of items.
- Tuples: Immutable ordered collections of items.
- Sets: Unordered collections of unique items.
- Dictionaries: Collections of key-value pairs.
- Strings: Immutable sequences of characters.
- Generators: Objects that produce a sequence of values.
- Files: Objects representing files that can be read or written sequentially.
- Built-in range() function
- PriorityQueue: Min heap from which we get elements sorted by lowest priority number.

(UN)INFORMED GRAPH SEARCH

Weeks 2–3

AIMA Chapter 3 "Problem Solving by Search" (3.1-3.4)

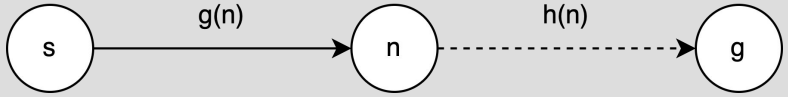
AIMA Chapter 3 "Problem Solving by Search" (3.5-3.6)

SEARCH ALGORITHMS

- BFS, DFS, IDDFS, uniform cost (Dijkstra), greedy best-first, A*.
- Completeness: A search algorithm is considered complete when it guarantees to find a solution if one exists and correctly reports failure if none exists.
- Cost-Optimality: A search algorithm is considered optimal when it guarantees to find a **cost-optimal** solution if one exists and correctly reports failure if none exists.
- Assume all edge costs are non-negative.
- Complete: BFS (finite branching factor; either finite state space or solution exists), IDDFS (finite branching factor; either finite state space or solution exists), uniform cost (finite branching factor; either finite state space or solution exists; positive edge costs), A*.
- Incomplete: DFS, greedy best-first.
- Cost-Optimal: BFS (all edge costs are equal), IDDFS (all edge costs are equal), uniform cost, A* with consistent/admissible heuristic.
- Suboptimal: DFS, greedy best-first, A* with inadmissible heuristic.

UNIFORM COST, GREEDY BEST-FIRST, A*

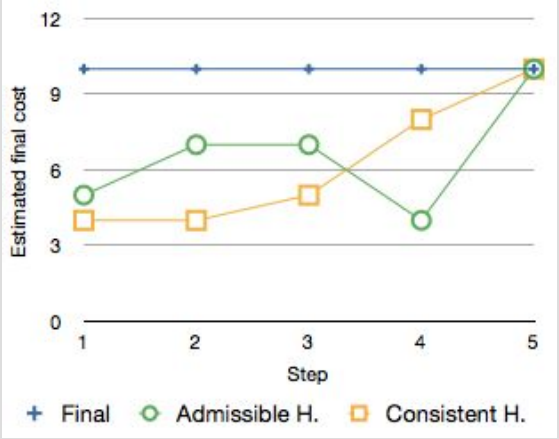
- Uniform cost: f(n) = g(n)
- Greedy best-first: f(n) = h(n)
- A*: f(n) = g(n) + h(n)



- f(n) = priority function = the estimated total cost of the path from start to goal through the node n
- g(n) = cost function = the true cost from start to n
- h(n) = heuristic function = the estimated cost from n to goal

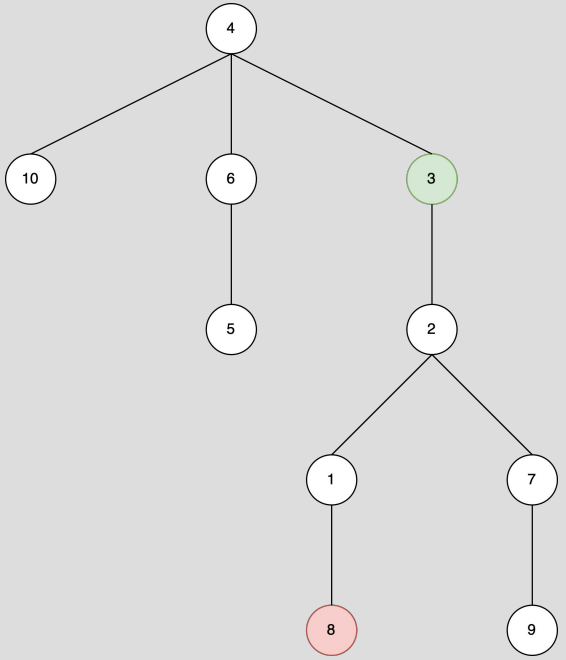
A* SEARCH

- Admissible heuristic: $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost.
- Consistent heuristic: satisfies the triangle inequality.
  - $h(goal) = 0$
  - $h(n) \leq c(n, n') + h(n')$ where n' is a successor n.
- Consistency implies admissibility
- Inadmissibility implies inconsistency



- $h(n) - h(n') \leq c(n, n')$, the heuristic difference/step cost never overestimates the true step cost.
- Three types of heuristics:
  - Consistent (monotone) and admissible (optimistic)
  - Inconsistent and admissible
  - Inconsistent and inadmissible

BFS EXAMPLE



In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3 Explored:

Frontier: 2, 4 Explored: 3

Frontier: 4, 1, 7 Explored: 3, 2

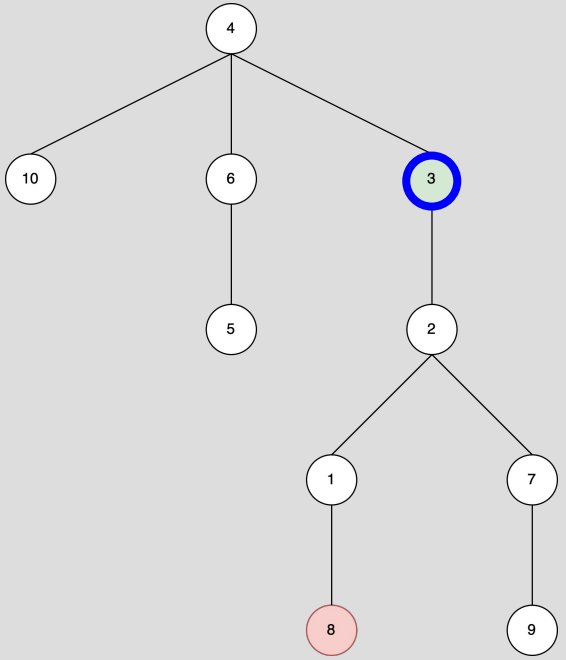
Frontier: 1, 7, 6, 10 Explored: 3, 2, 4

Frontier: 7, 6, 10, 8 Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3 Explored:

Frontier: 2, 4 Explored: 3

Frontier: 4, 1, 7 Explored: 3, 2

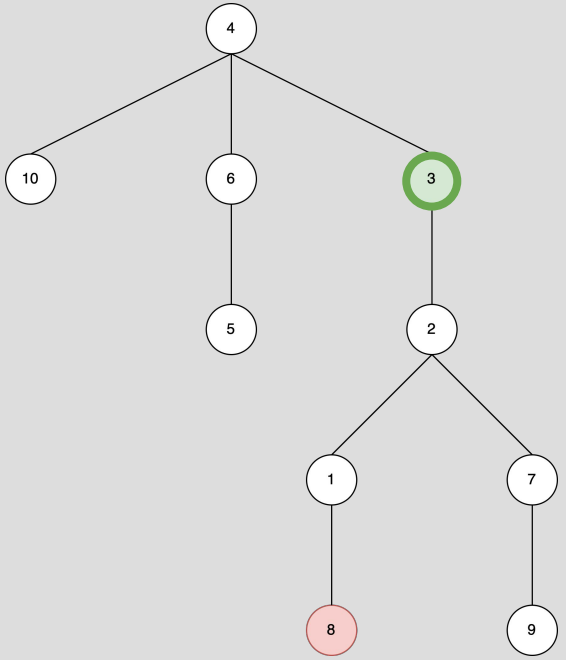
Frontier: 1, 7, 6, 10 Explored: 3, 2, 4

Frontier: 7, 6, 10, 8 Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3 Explored:

Frontier: 2, 4 Explored: 3

Frontier: 4, 1, 7 Explored: 3, 2

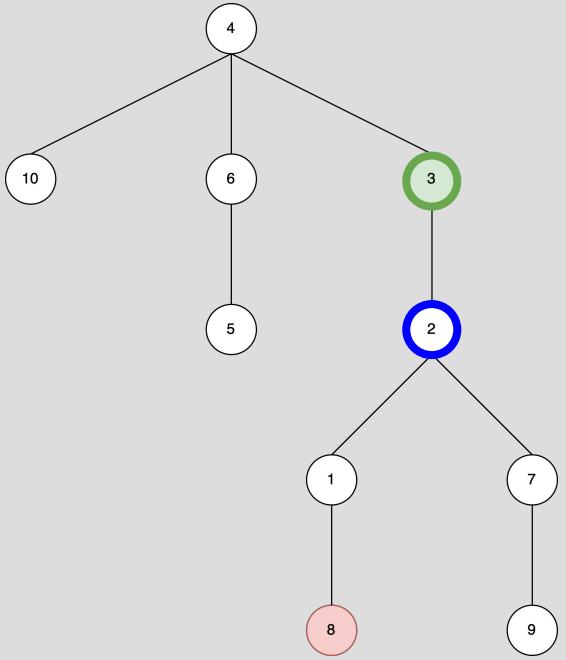
Frontier: 1, 7, 6, 10 Explored: 3, 2, 4

Frontier: 7, 6, 10, 8 Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



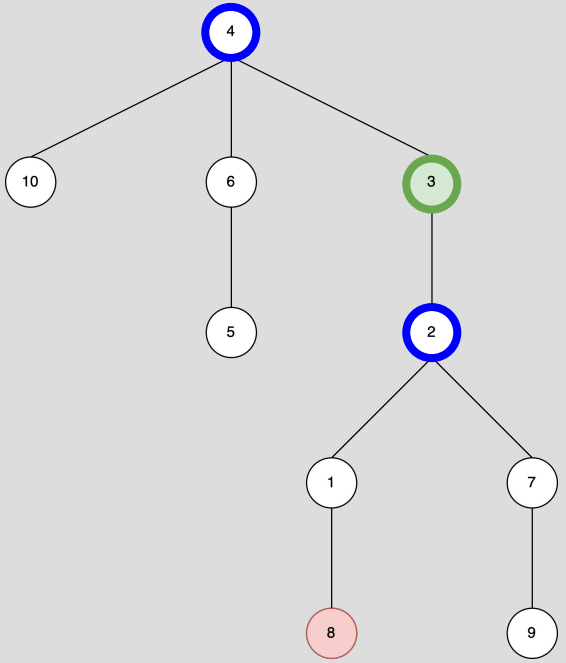
In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3	Explored:
Frontier: 2, 4	Explored: 3
Frontier: 4, 1, 7	Explored: 3, 2
Frontier: 1, 7, 6, 10	Explored: 3, 2, 4
Frontier: 7, 6, 10, 8	Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



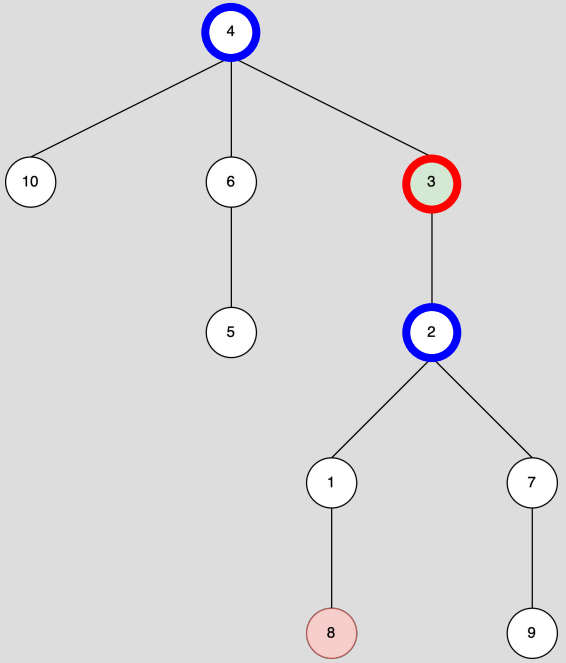
In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3	Explored:
Frontier: 2, 4	Explored: 3
Frontier: 4, 1, 7	Explored: 3, 2
Frontier: 1, 7, 6, 10	Explored: 3, 2, 4
Frontier: 7, 6, 10, 8	Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3 Explored:

Frontier: 2, 4 Explored: 3

Frontier: 4, 1, 7 Explored: 3, 2

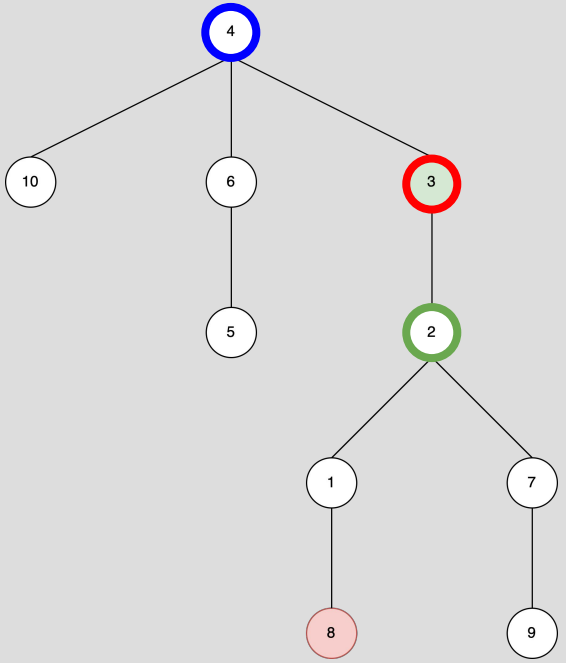
Frontier: 1, 7, 6, 10 Explored: 3, 2, 4

Frontier: 7, 6, 10, 8 Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3 Explored:

Frontier: 2, 4 Explored: 3

Frontier: 4, 1, 7 Explored: 3, 2

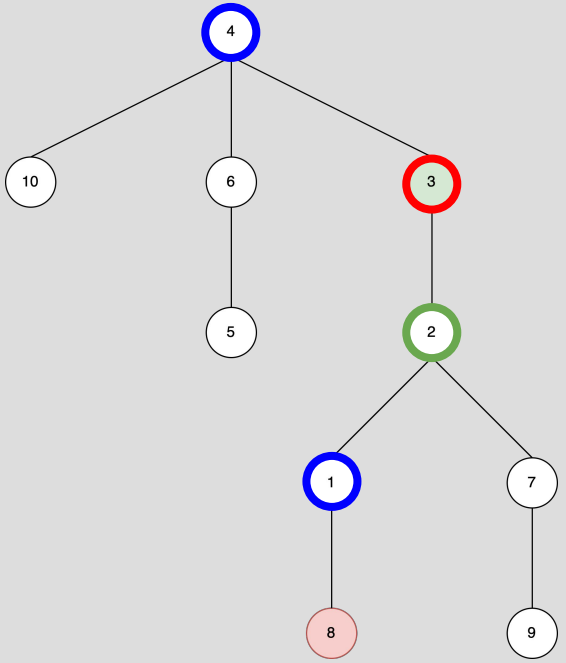
Frontier: 1, 7, 6, 10 Explored: 3, 2, 4

Frontier: 7, 6, 10, 8 Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3 Explored:

Frontier: 2, 4 Explored: 3

Frontier: 4, 1, 7 Explored: 3, 2

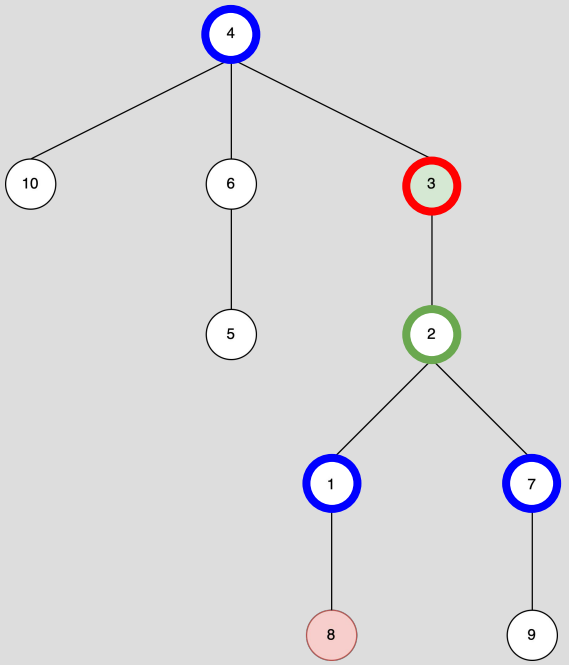
Frontier: 1, 7, 6, 10 Explored: 3, 2, 4

Frontier: 7, 6, 10, 8 Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3 Explored:

Frontier: 2, 4 Explored: 3

Frontier: 4, 1, 7 Explored: 3, 2

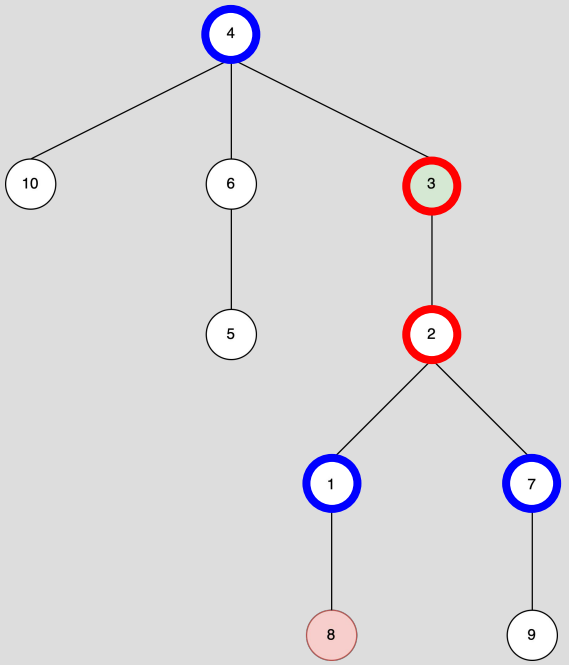
Frontier: 1, 7, 6, 10 Explored: 3, 2, 4

Frontier: 7, 6, 10, 8 Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3 Explored:

Frontier: 2, 4 Explored: 3

Frontier: 4, 1, 7 Explored: 3, 2

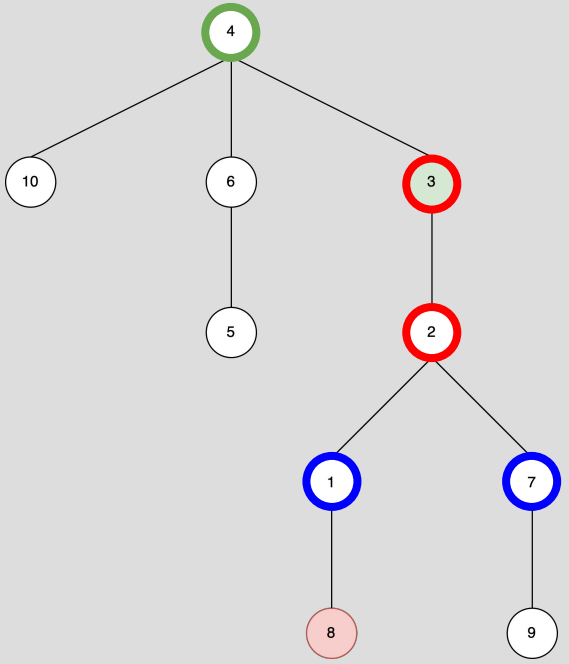
Frontier: 1, 7, 6, 10 Explored: 3, 2, 4

Frontier: 7, 6, 10, 8 Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



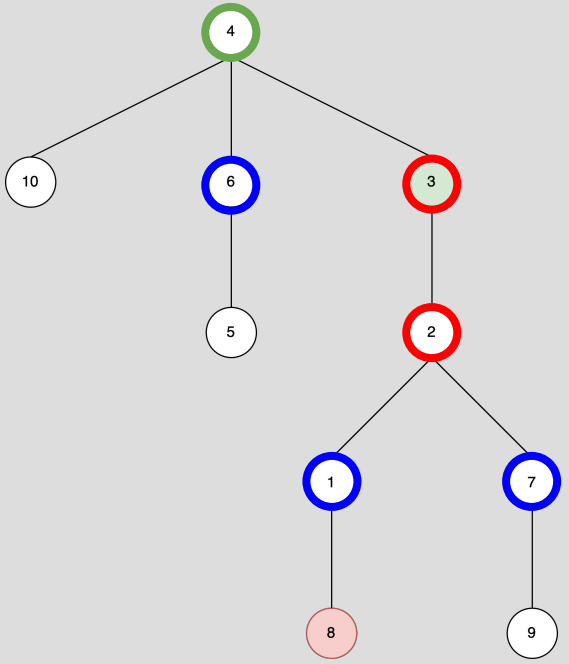
In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3	Explored:
Frontier: 2, 4	Explored: 3
Frontier: 4, 1, 7	Explored: 3, 2
Frontier: 1, 7, 6, 10	Explored: 3, 2, 4
Frontier: 7, 6, 10, 8	Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3 Explored:

Frontier: 2, 4 Explored: 3

Frontier: 4, 1, 7 Explored: 3, 2

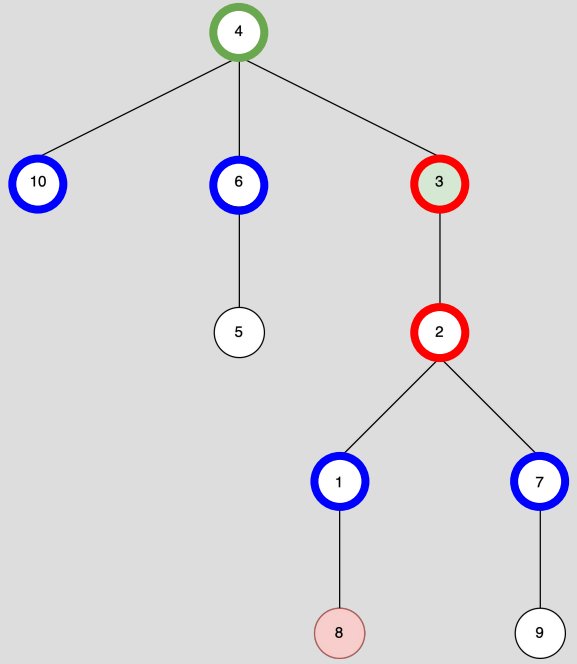
Frontier: 1, 7, 6, 10 Explored: 3, 2, 4

Frontier: 7, 6, 10, 8 Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



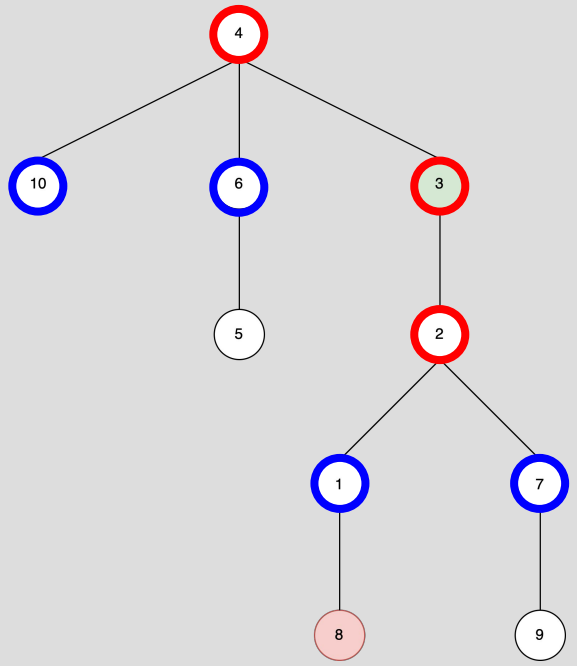
In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3	Explored:
Frontier: 2, 4	Explored: 3
Frontier: 4, 1, 7	Explored: 3, 2
Frontier: 1, 7, 6, 10	Explored: 3, 2, 4
Frontier: 7, 6, 10, 8	Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



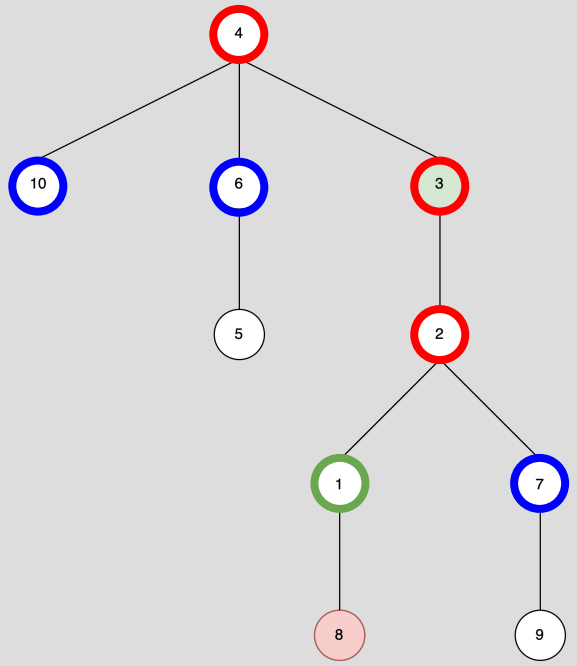
In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3	Explored:
Frontier: 2, 4	Explored: 3
Frontier: 4, 1, 7	Explored: 3, 2
Frontier: 1, 7, 6, 10	Explored: 3, 2, 4
Frontier: 7, 6, 10, 8	Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



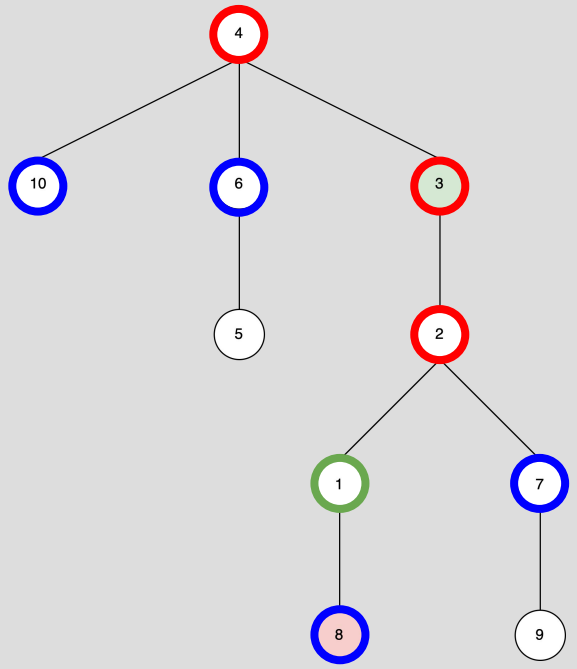
In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3	Explored:
Frontier: 2, 4	Explored: 3
Frontier: 4, 1, 7	Explored: 3, 2
Frontier: 1, 7, 6, 10	Explored: 3, 2, 4
Frontier: 7, 6, 10, 8	Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



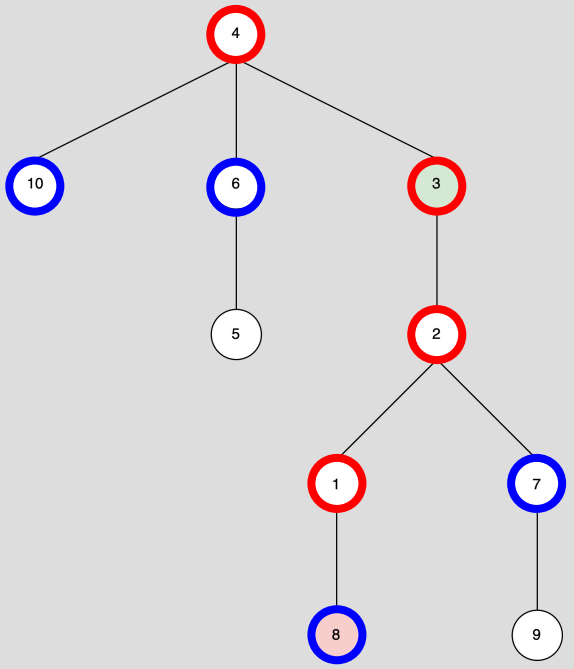
In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3	Explored:
Frontier: 2, 4	Explored: 3
Frontier: 4, 1, 7	Explored: 3, 2
Frontier: 1, 7, 6, 10	Explored: 3, 2, 4
Frontier: 7, 6, 10, 8	Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



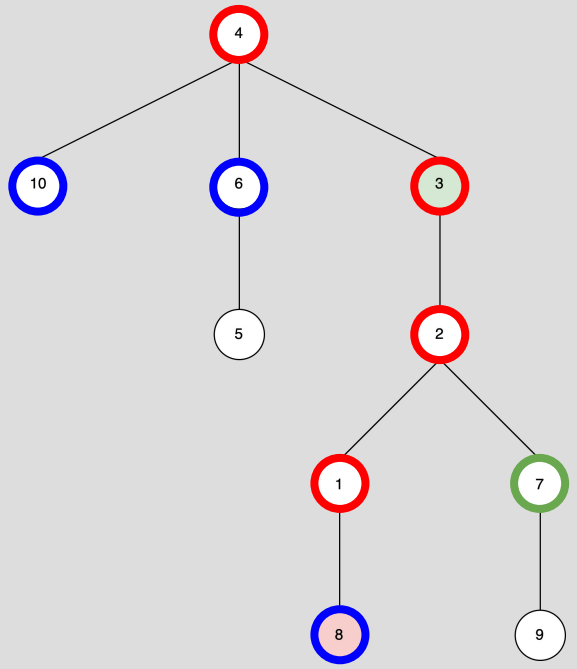
In order, list the nodes found by running breadth first search (BFS) beginning from node 3 and searching for node 8. Break ties numerically.

Frontier (FIFO): 3	Explored:
Frontier: 2, 4	Explored: 3
Frontier: 4, 1, 7	Explored: 3, 2
Frontier: 1, 7, 6, 10	Explored: 3, 2, 4
Frontier: 7, 6, 10, 8	Explored: 3, 2, 4, 1

Solution by early detection: 3, 2, 4, 1, 7, 6, 10, 8

BFS EXAMPLE

Frontier
Current
Explored



Frontier: 6, 10, 8, 9 Explored: 3, 2, 4, 1, 7

Frontier: 10, 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6

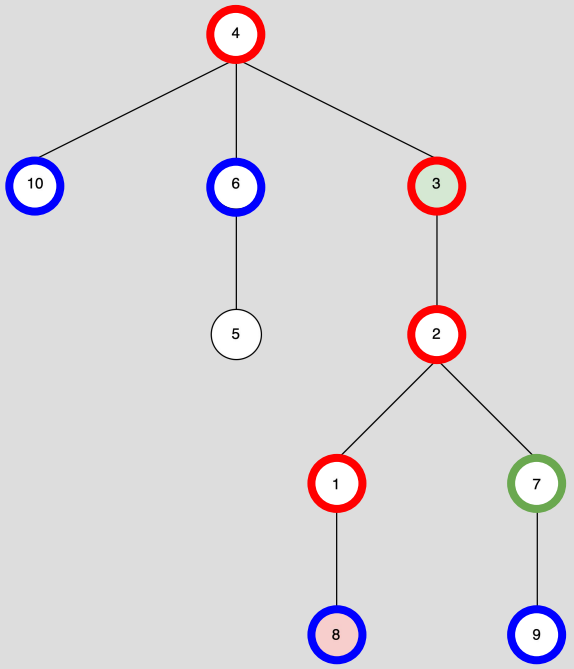
Frontier: 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10

Frontier: 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10, 8

Solution: 3, 2, 4, 1, 7, 6, 10, 8

BFS EXAMPLE

Frontier
Current
Explored



Frontier: 6, 10, 8, 9 Explored: 3, 2, 4, 1, 7

Frontier: 10, 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6

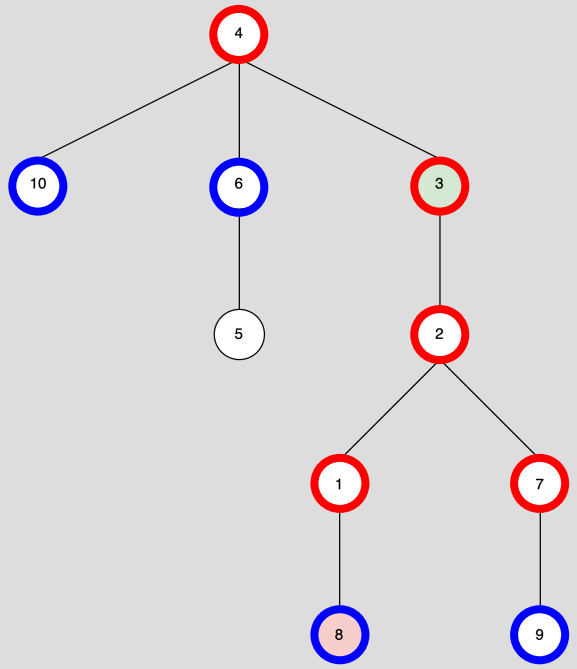
Frontier: 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10

Frontier: 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10, 8

Solution: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



Frontier: 6, 10, 8, 9 Explored: 3, 2, 4, 1, 7

Frontier: 10, 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6

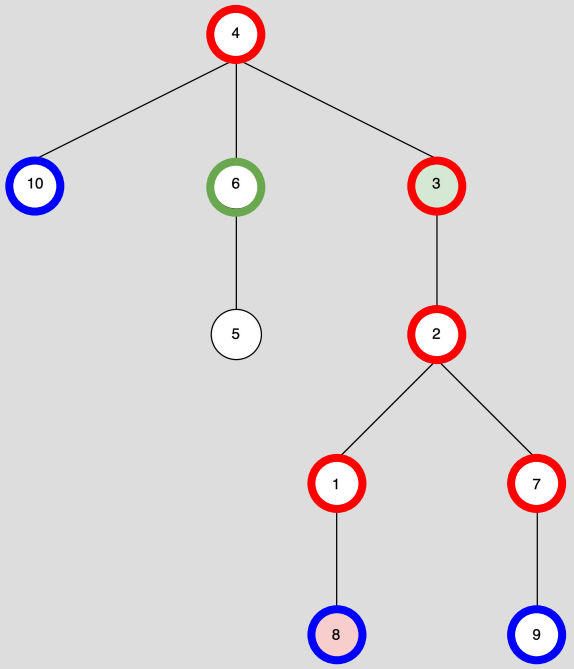
Frontier: 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10

Frontier: 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10, 8

Solution: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



Frontier: 6, 10, 8, 9 Explored: 3, 2, 4, 1, 7

Frontier: 10, 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6

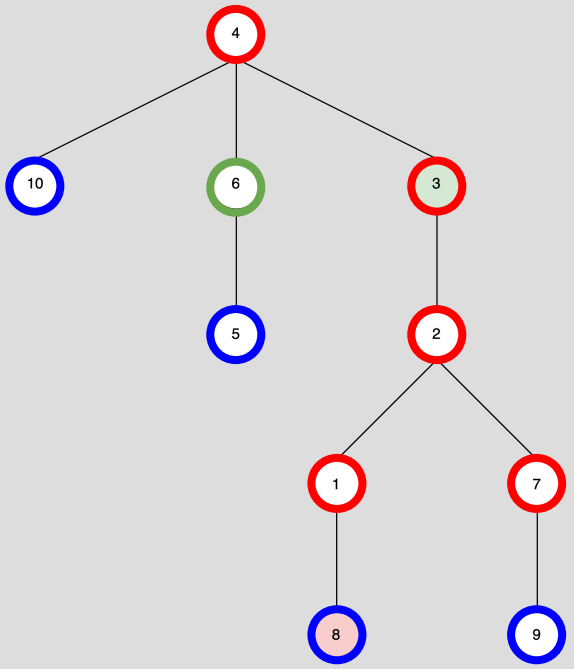
Frontier: 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10

Frontier: 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10, 8

Solution: 3, 2, 4, 1, 7, 6, 10, 8

BFS EXAMPLE

Frontier
Current
Explored



Frontier: 6, 10, 8, 9 Explored: 3, 2, 4, 1, 7

Frontier: 10, 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6

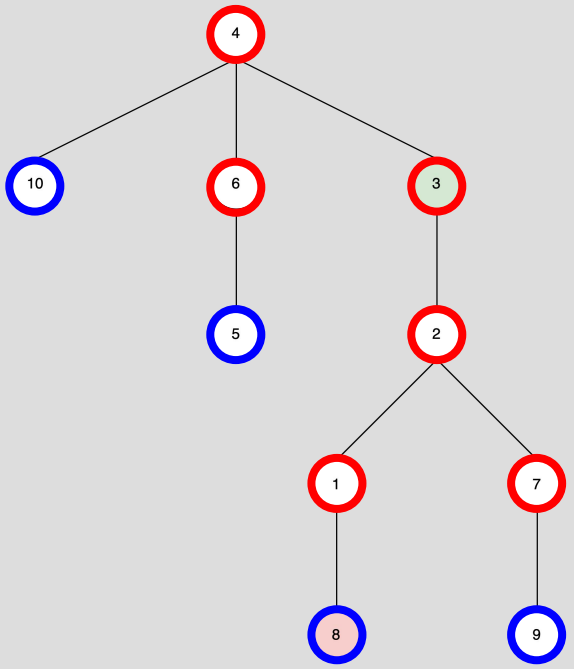
Frontier: 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10

Frontier: 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10, 8

Solution: 3, 2, 4, 1, 7, 6, 10, 8

BFS EXAMPLE

Frontier
Current
Explored



Frontier: 6, 10, 8, 9 Explored: 3, 2, 4, 1, 7

Frontier: 10, 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6

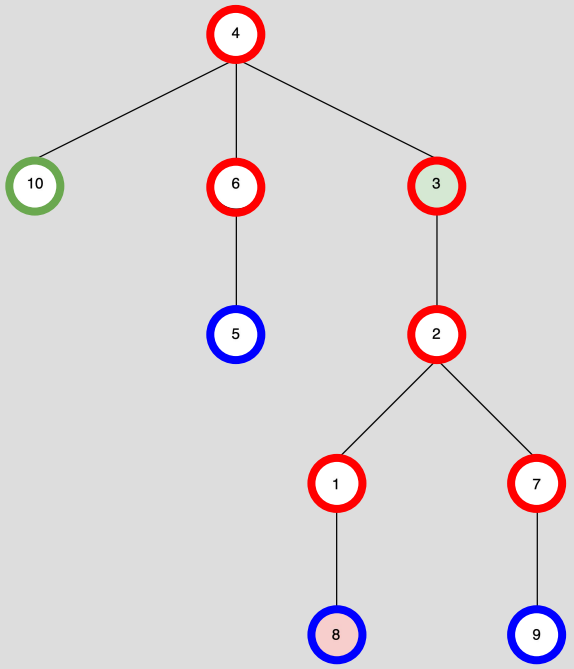
Frontier: 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10

Frontier: 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10, 8

Solution: 3, 2, 4, 1, 7, 6, 10, 8

BFS EXAMPLE

Frontier
Current
Explored



Frontier: 6, 10, 8, 9 Explored: 3, 2, 4, 1, 7

Frontier: 10, 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6

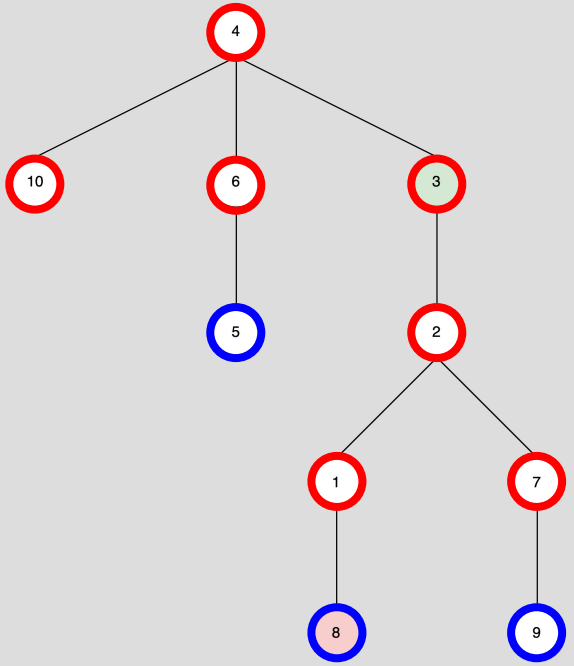
Frontier: 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10

Frontier: 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10, 8

Solution: 3, 2, 4, 1, 7, 6, 10, 8

Frontier
Current
Explored

BFS EXAMPLE



Frontier: 6, 10, 8, 9 Explored: 3, 2, 4, 1, 7

Frontier: 10, 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6

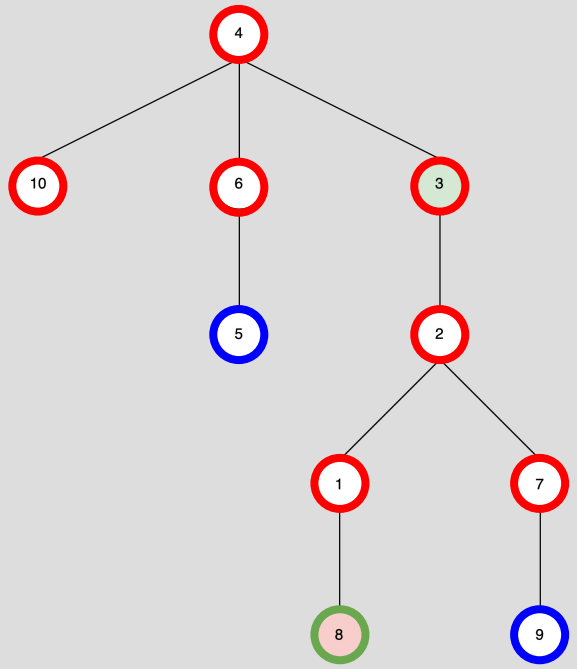
Frontier: 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10

Frontier: 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10, 8

Solution: 3, 2, 4, 1, 7, 6, 10, 8

BFS EXAMPLE

Frontier
Current
Explored



Frontier: 6, 10, 8, 9 Explored: 3, 2, 4, 1, 7

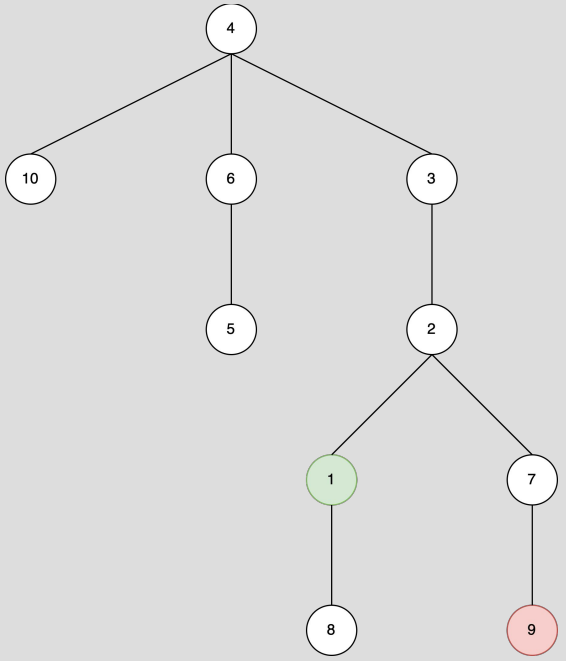
Frontier: 10, 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6

Frontier: 8, 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10

Frontier: 9, 5 Explored: 3, 2, 4, 1, 7, 6, 10, 8

Solution: 3, 2, 4, 1, 7, 6, 10, 8

DFS EXAMPLE

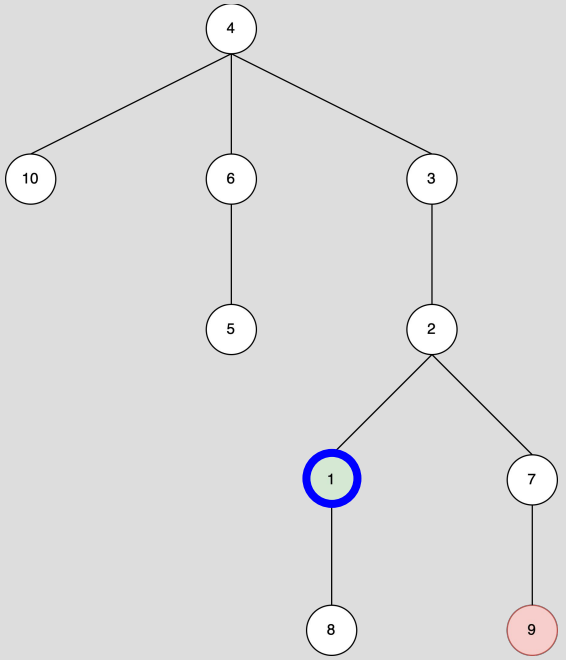


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

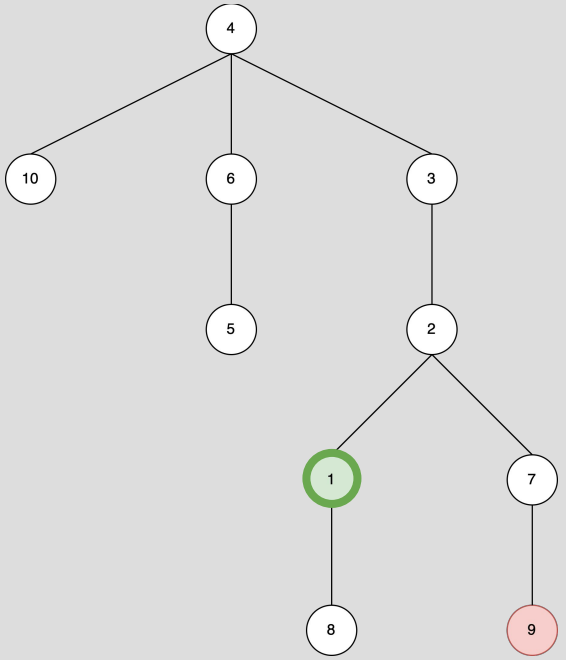


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

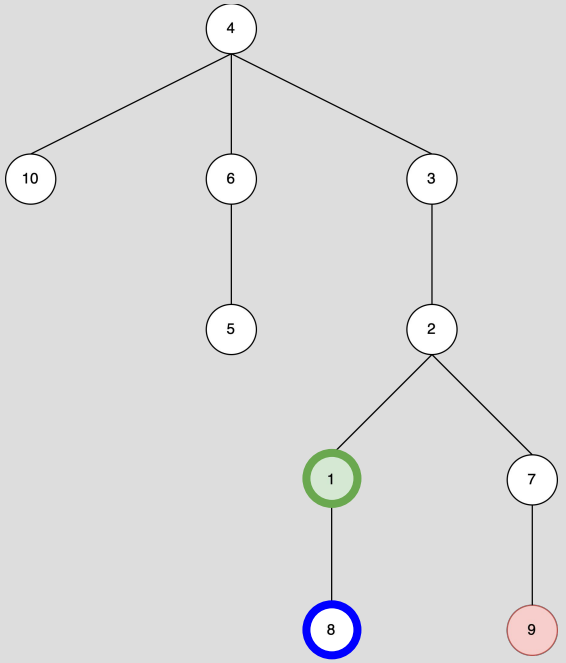


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

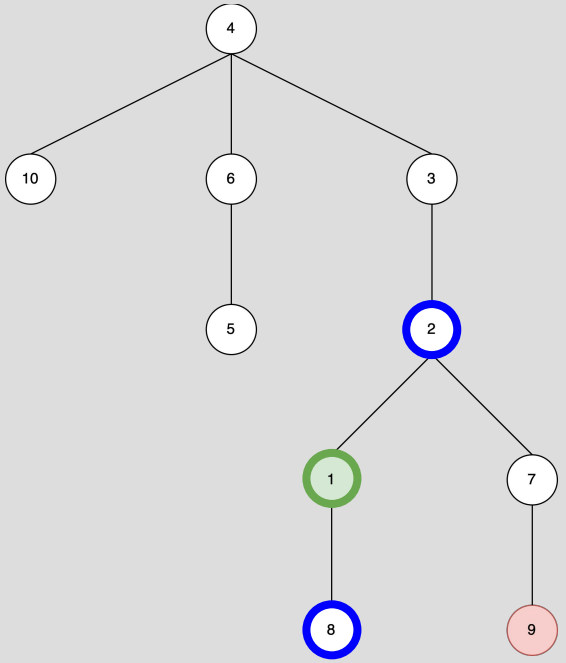


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

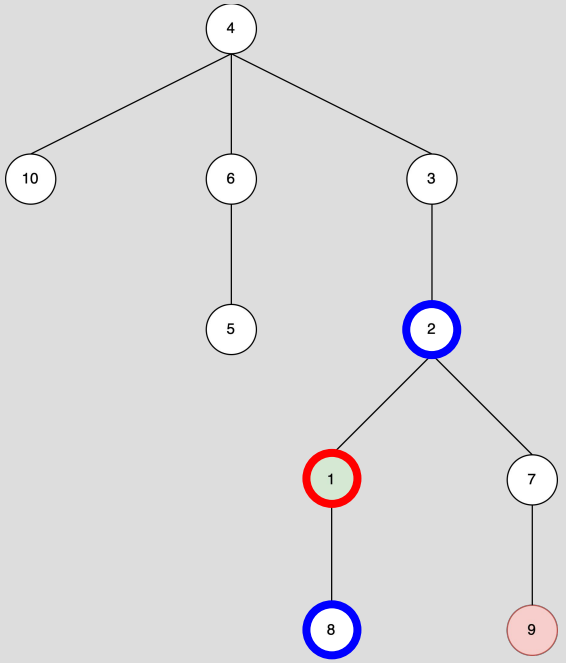


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

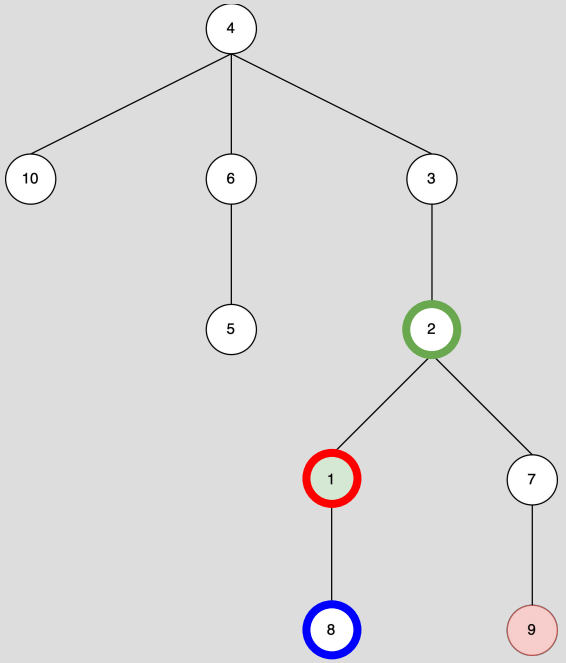


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

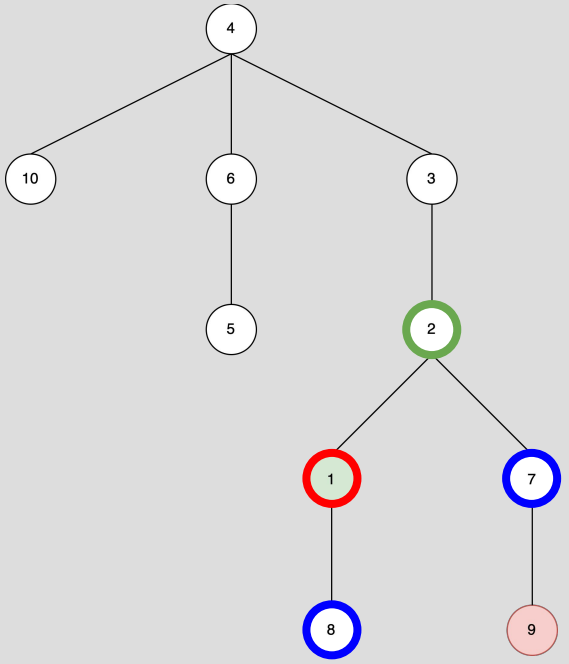


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

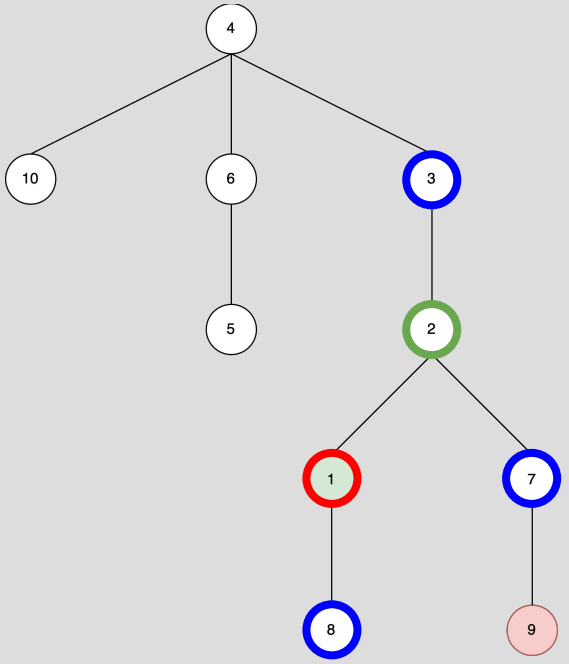


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

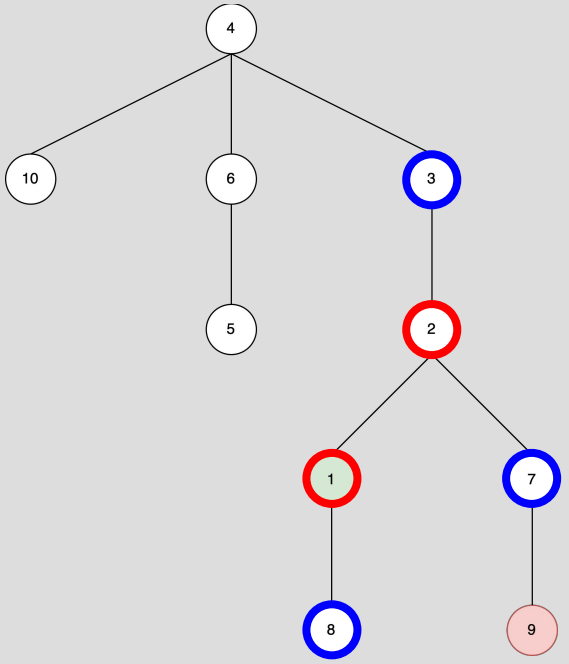


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

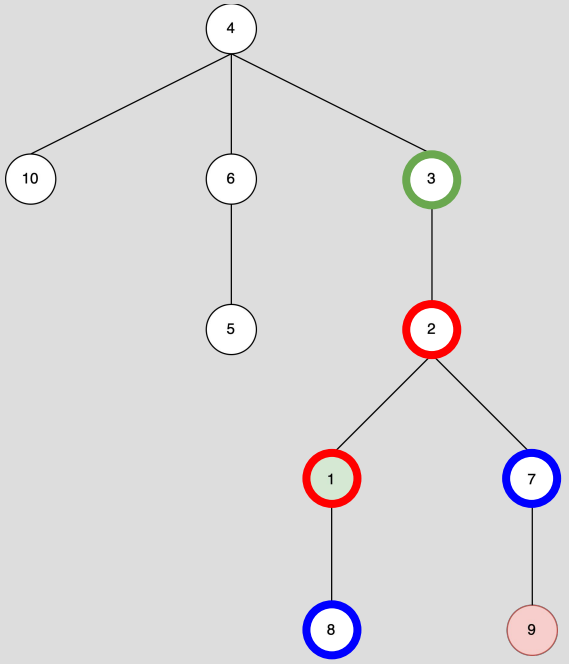


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

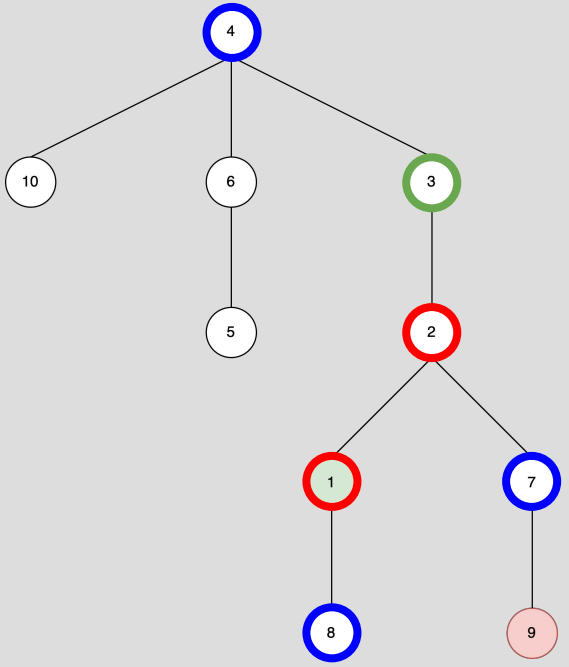


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

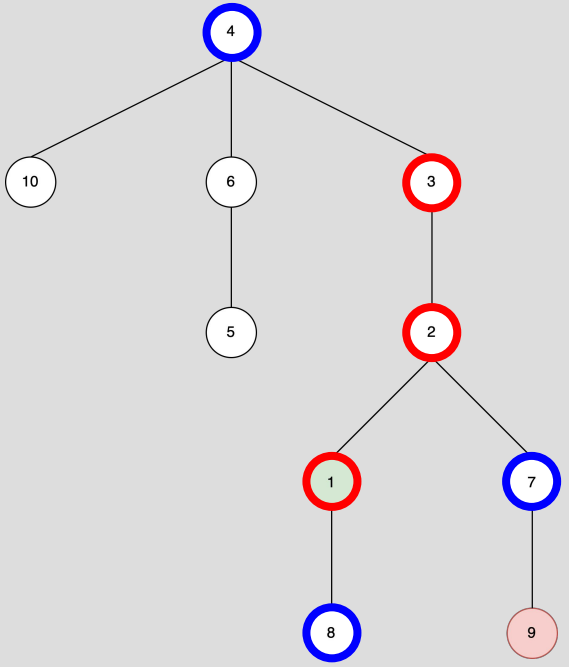


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

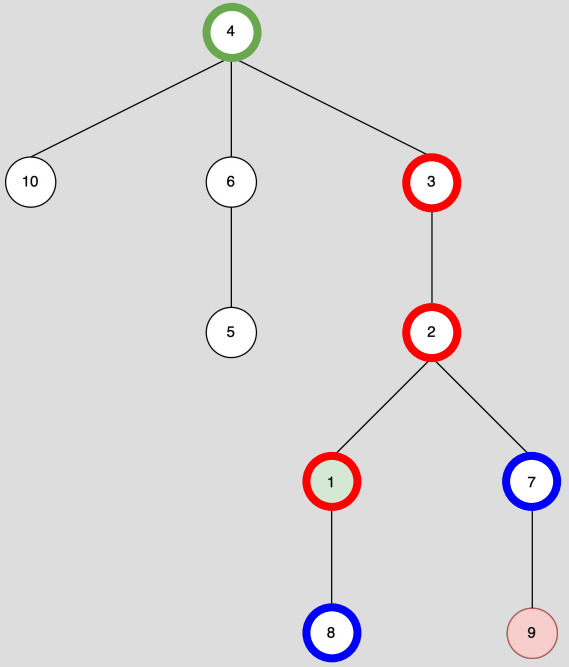


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

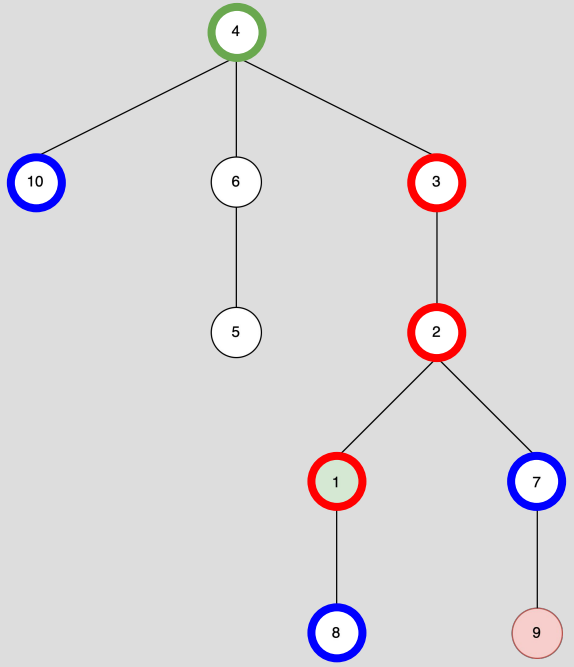


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

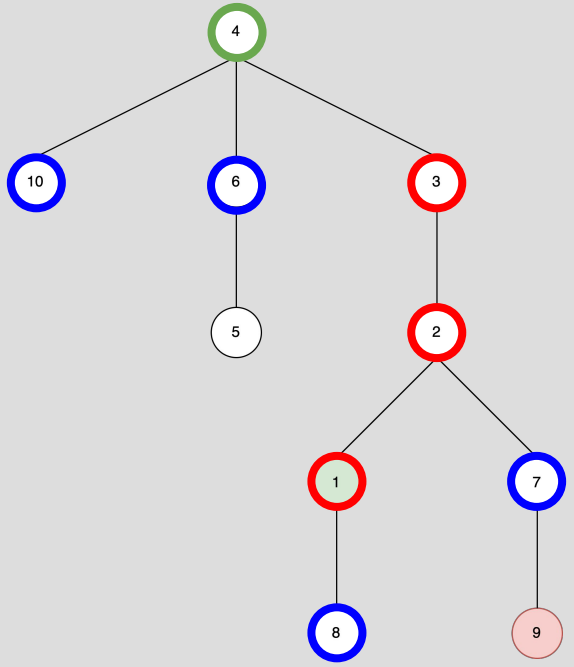


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

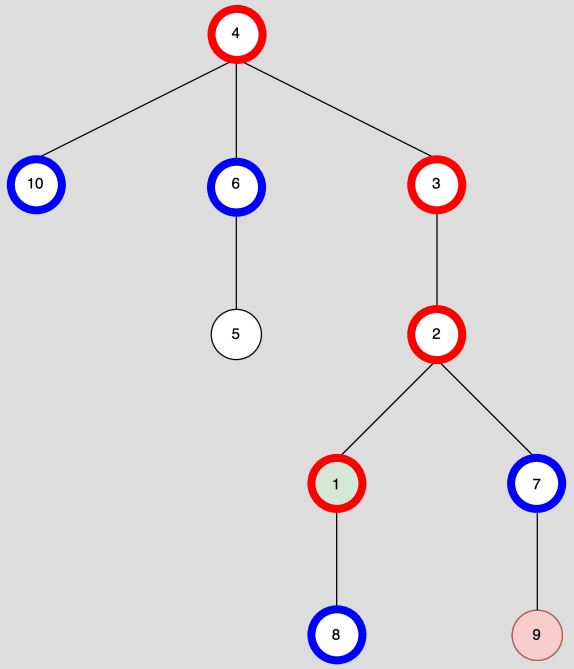


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

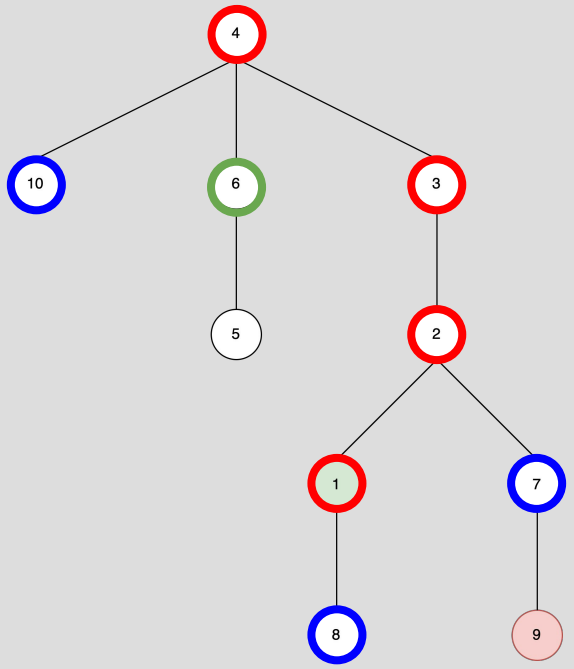


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

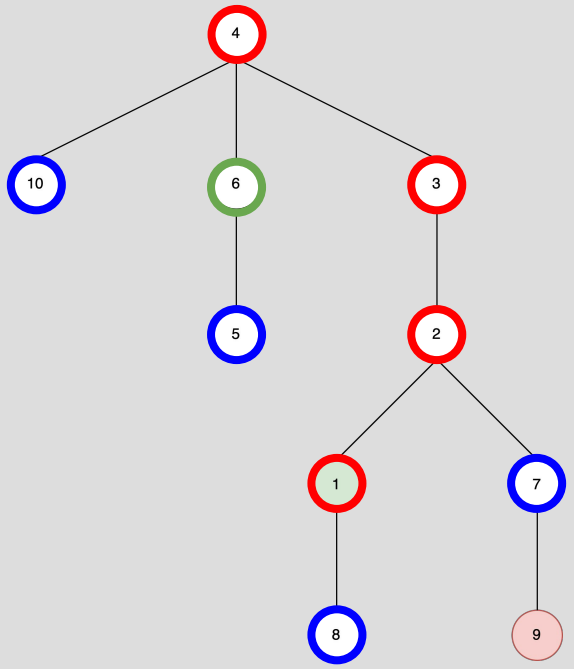


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

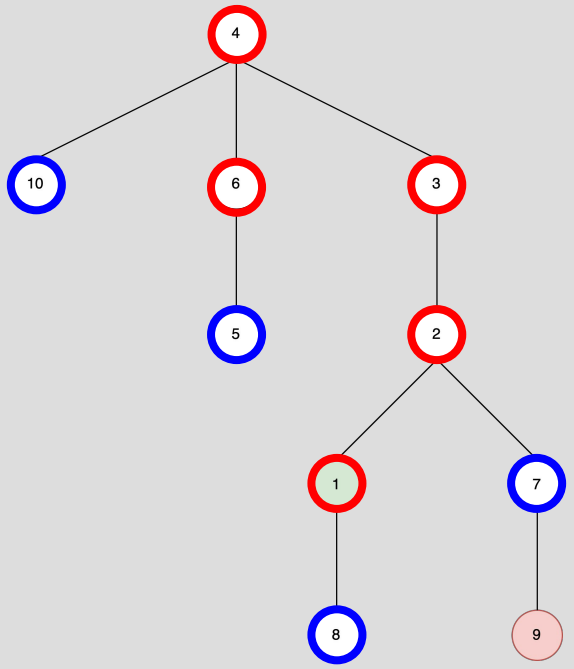


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

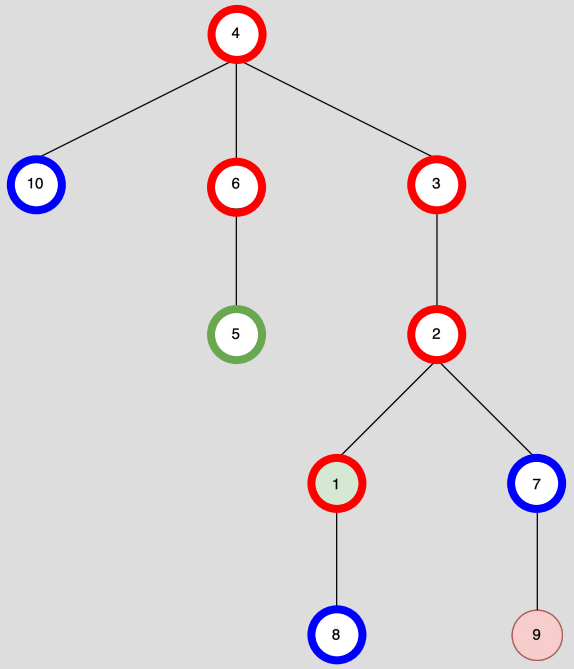


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

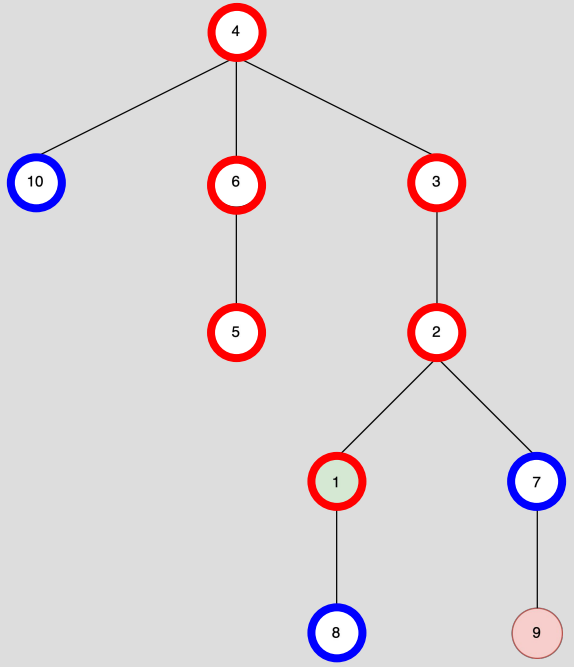


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

Frontier
Current
Explored

DFS EXAMPLE

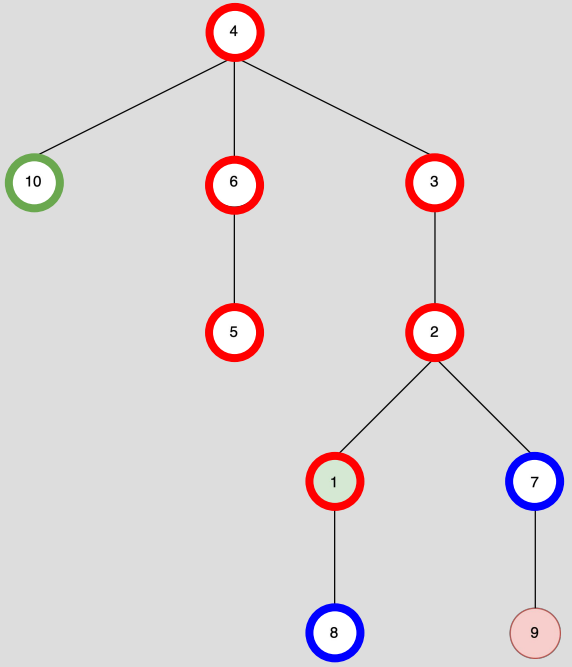


In order, list the nodes found by running depth first search (DFS) beginning from node 1 and searching for node 9. Break ties numerically.

Frontier (LIFO): 1	Explored:
Frontier: 8, 2	Explored: 1
Frontier: 8, 7, 3	Explored: 1, 2
Frontier: 8, 7, 4	Explored: 1, 2, 3
Frontier: 8, 7, 10, 6	Explored: 1, 2, 3, 4
Frontier: 8, 7, 10, 5	Explored: 1, 2, 3, 4, 6
Frontier: 8, 7, 10	Explored: 1, 2, 3, 4, 6, 5

DFS EXAMPLE

Frontier
Current
Explored



Frontier: 8, 7 Explored: 1, 2, 3, 4, 6, 5, 10

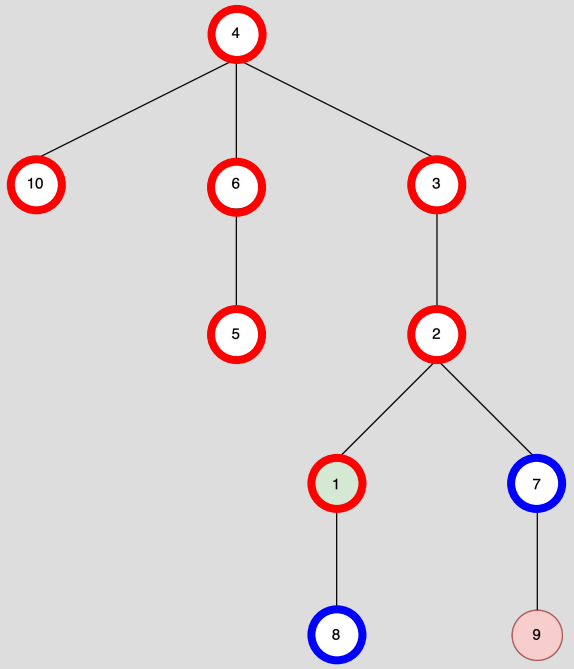
Frontier: 8, 9 Explored: 1, 2, 3, 4, 6, 5, 10, 7

Frontier: 8 Explored: 1, 2, 3, 4, 6, 5, 10, 7, 9

Solution: 1, 2, 3, 4, 6, 5, 10, 7, 9

DFS EXAMPLE

Frontier
Current
Explored



Frontier: 8, 7 Explored: 1, 2, 3, 4, 6, 5, 10

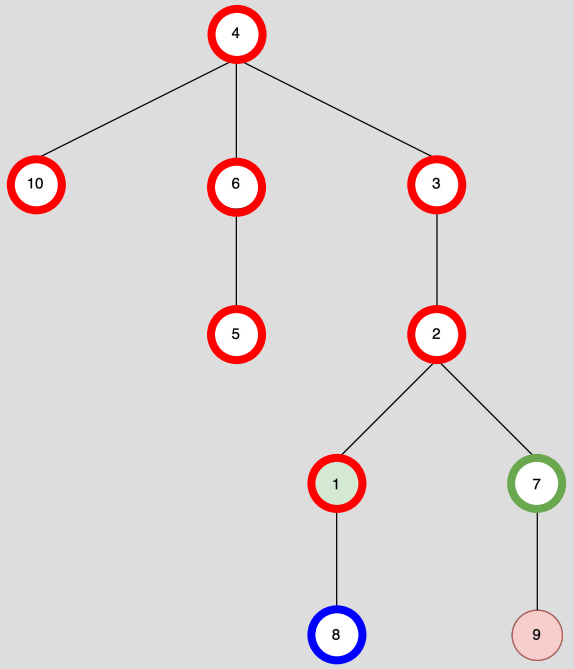
Frontier: 8, 9 Explored: 1, 2, 3, 4, 6, 5, 10, 7

Frontier: 8 Explored: 1, 2, 3, 4, 6, 5, 10, 7, 9

Solution: 1, 2, 3, 4, 6, 5, 10, 7, 9

Frontier
Current
Explored

DFS EXAMPLE



Frontier: 8, 7 Explored: 1, 2, 3, 4, 6, 5, 10

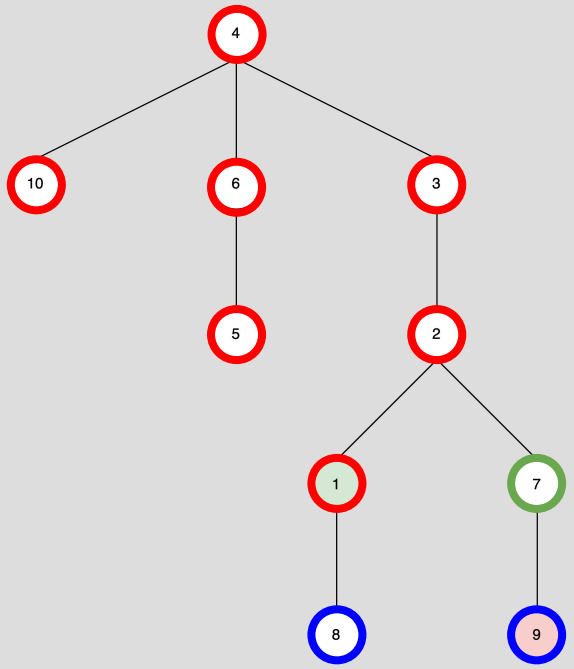
Frontier: 8, 9 Explored: 1, 2, 3, 4, 6, 5, 10, 7

Frontier: 8 Explored: 1, 2, 3, 4, 6, 5, 10, 7, 9

Solution: 1, 2, 3, 4, 6, 5, 10, 7, 9

Frontier
Current
Explored

DFS EXAMPLE



Frontier: 8, 7 Explored: 1, 2, 3, 4, 6, 5, 10

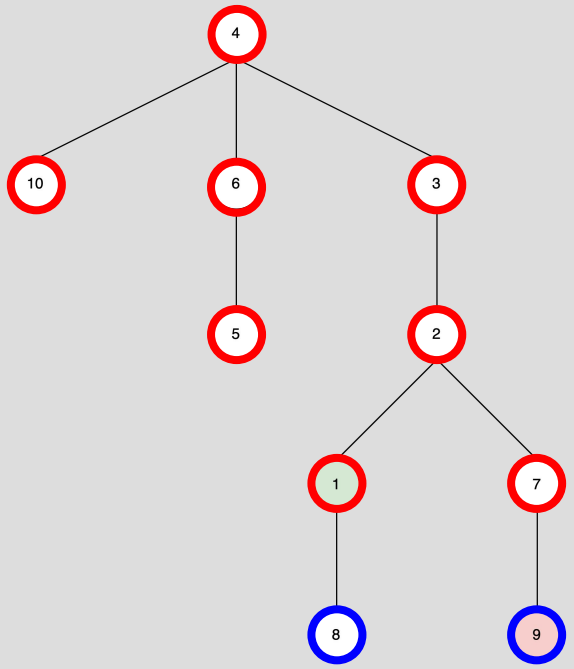
Frontier: 8, 9 Explored: 1, 2, 3, 4, 6, 5, 10, 7

Frontier: 8 Explored: 1, 2, 3, 4, 6, 5, 10, 7, 9

Solution: 1, 2, 3, 4, 6, 5, 10, 7, 9

DFS EXAMPLE

Frontier
Current
Explored



Frontier: 8, 7 Explored: 1, 2, 3, 4, 6, 5, 10

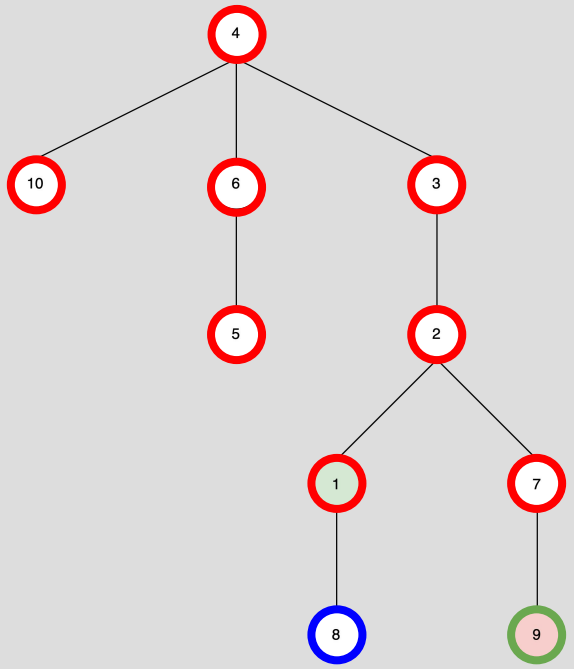
Frontier: 8, 9 Explored: 1, 2, 3, 4, 6, 5, 10, 7

Frontier: 8 Explored: 1, 2, 3, 4, 6, 5, 10, 7, 9

Solution: 1, 2, 3, 4, 6, 5, 10, 7, 9

DFS EXAMPLE

Frontier
Current
Explored



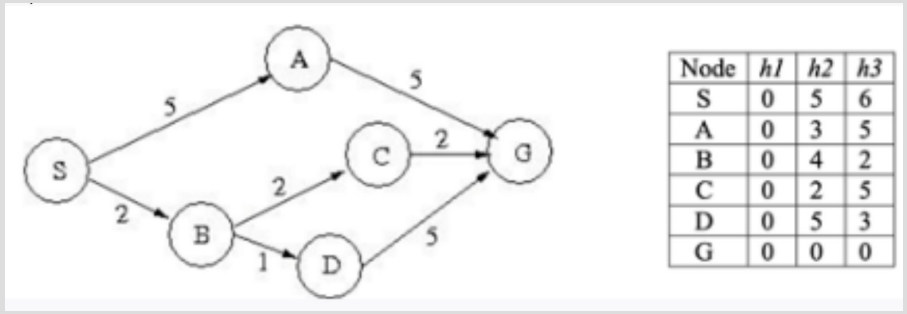
Frontier: 8, 7 Explored: 1, 2, 3, 4, 6, 5, 10

Frontier: 8, 9 Explored: 1, 2, 3, 4, 6, 5, 10, 7

Frontier: 8 Explored: 1, 2, 3, 4, 6, 5, 10, 7, 9

Solution: 1, 2, 3, 4, 6, 5, 10, 7, 9

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

f(G) = h2(G) = 0

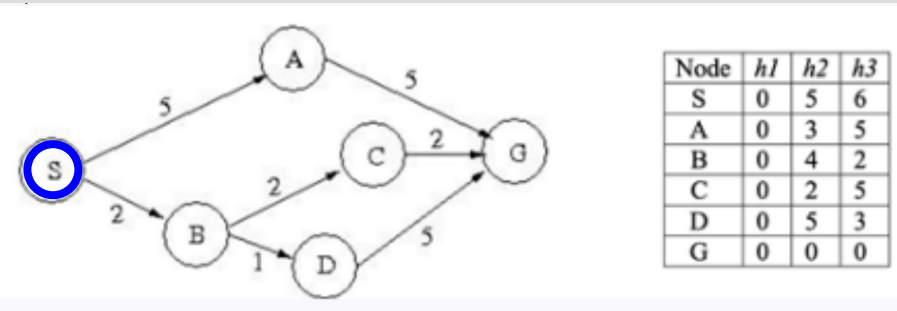
Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

f(G) = h2(G) = 0

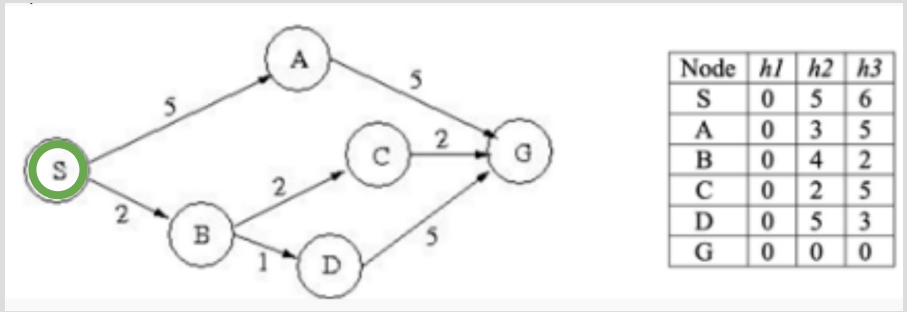
Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

f(G) = h2(G) = 0

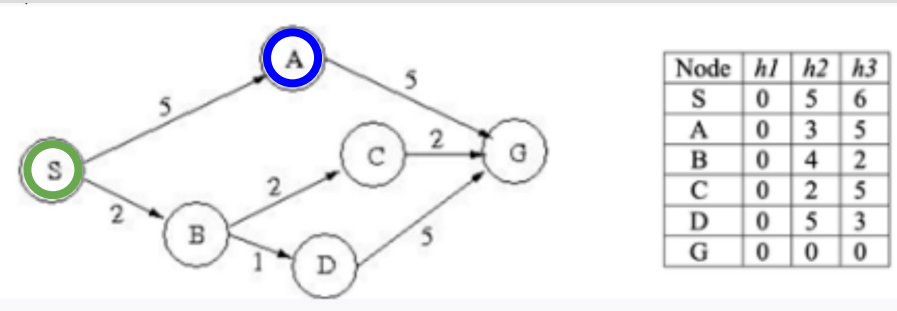
Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

f(G) = h2(G) = 0

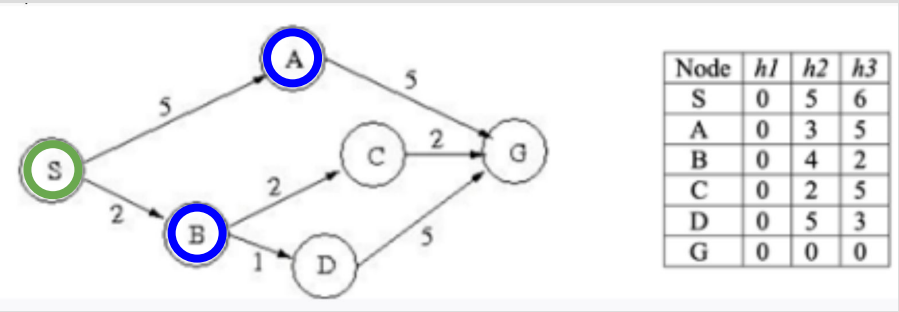
Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

f(G) = h2(G) = 0

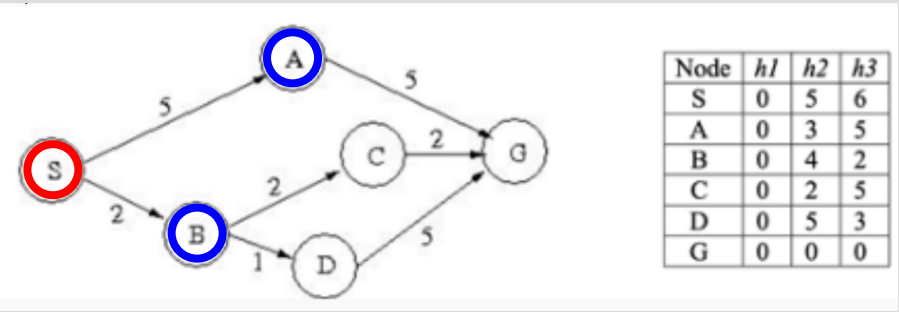
Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

f(G) = h2(G) = 0

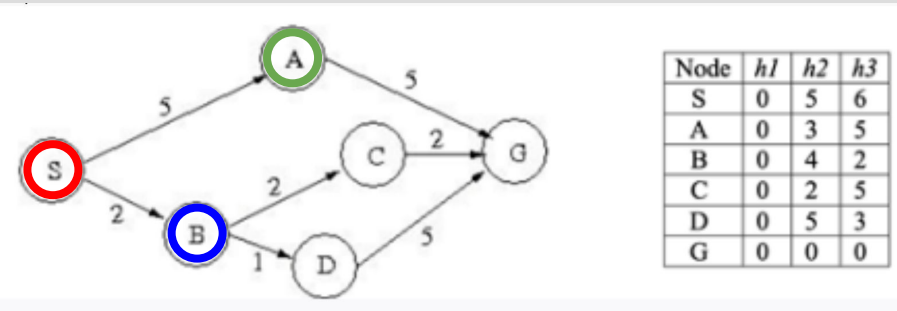
Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

f(G) = h2(G) = 0

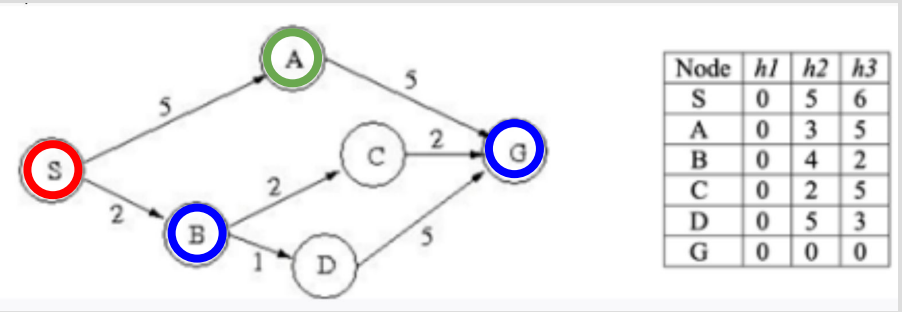
Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

f(G) = h2(G) = 0

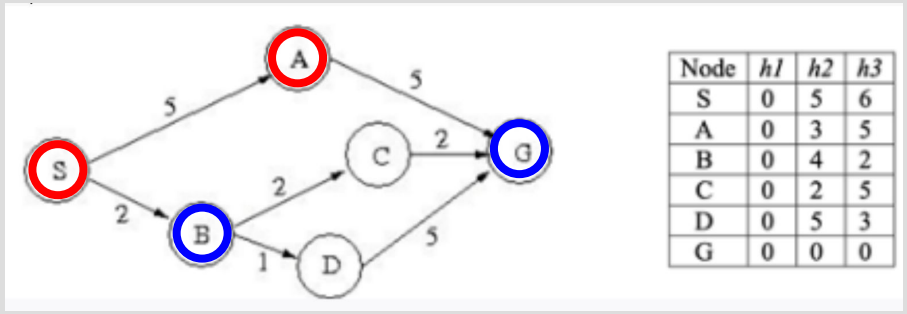
Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

f(G) = h2(G) = 0

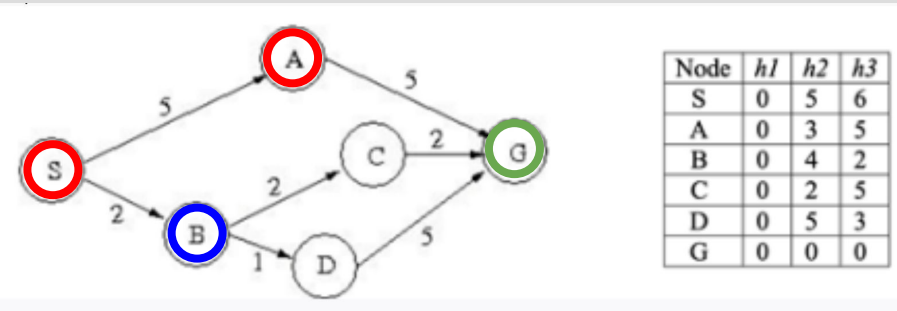
Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Consider the following search space where we want to find a path from the start state S to the goal state G. What solution path is found by greedy best-first search using h2?

Frontier (priority queue): f(S) = 5

Node S is popped from the frontier.

f(A) = h2(A) = 3

f(B) = h2(B) = 4

Frontier: f(A) = 3, f(B) = 4

Node A is popped from the frontier.

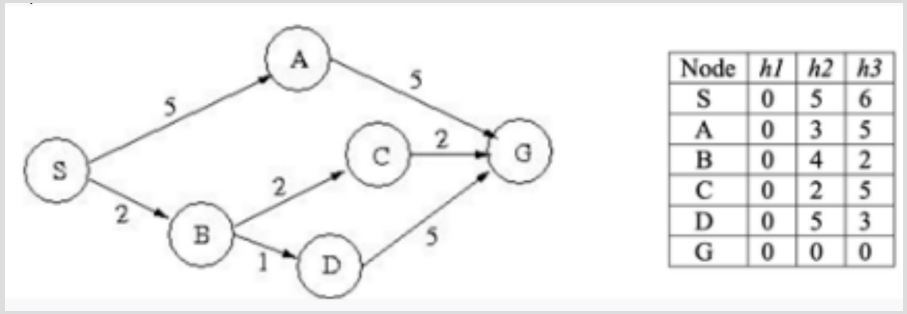
f(G) = h2(G) = 0

Frontier: f(G) = 0, f(B) = 4

Node G is popped from the frontier.

Solution: S, A, G

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

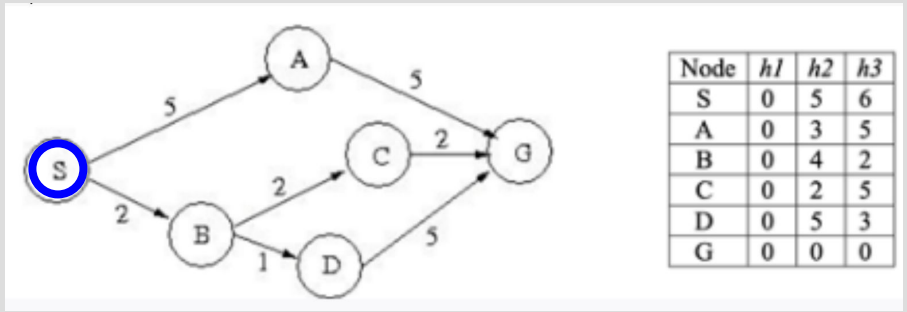
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

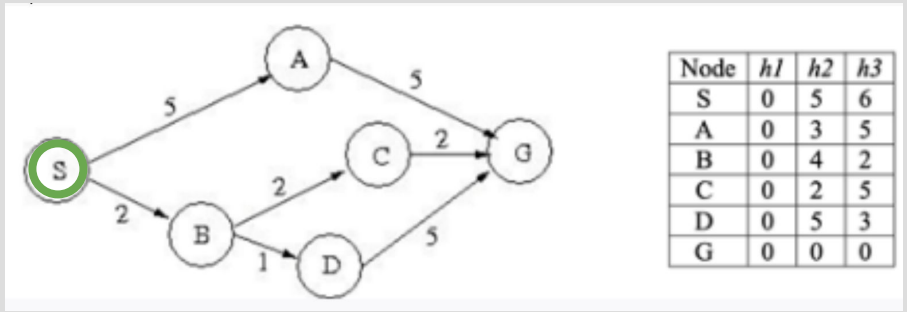
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

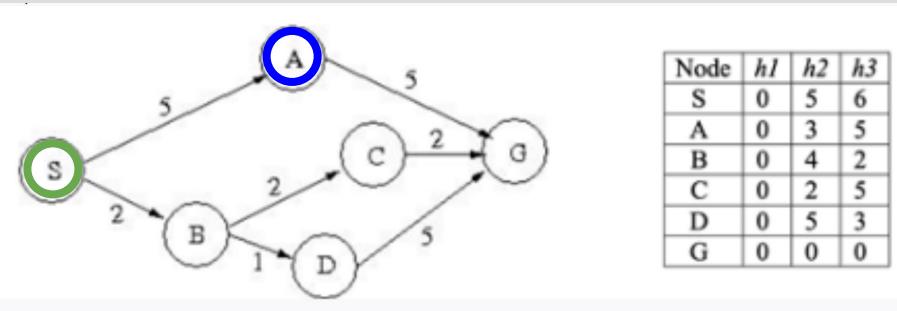
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

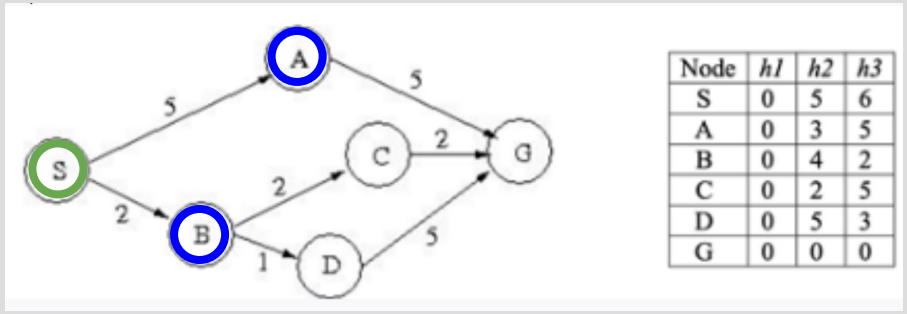
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

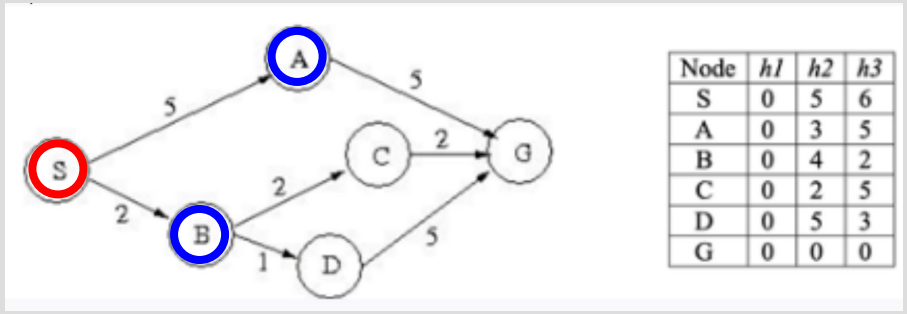
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

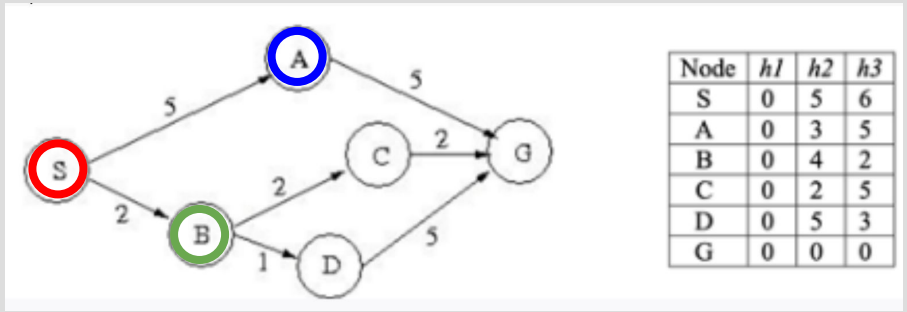
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

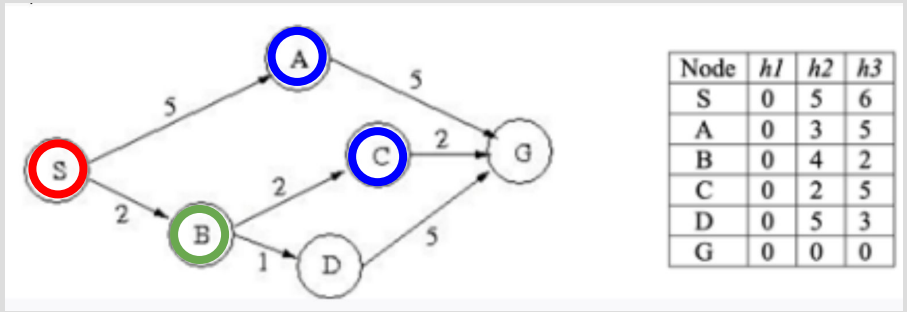
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

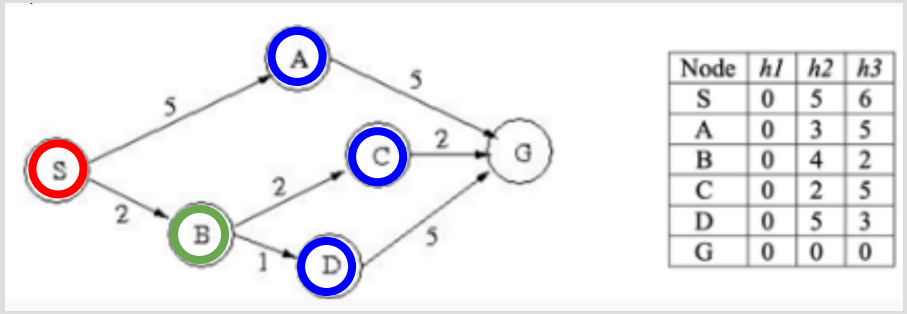
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

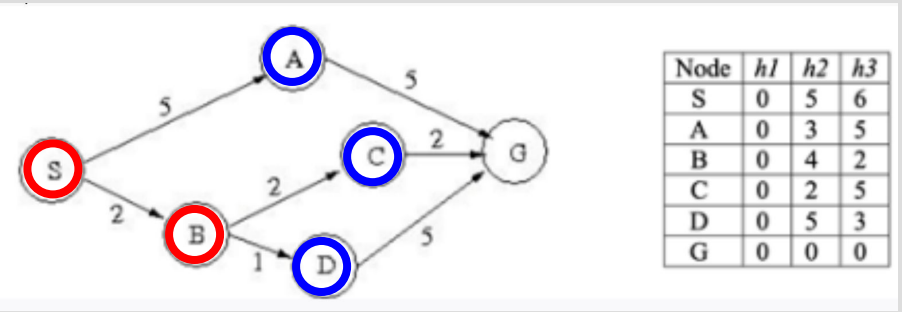
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h1?

Frontier (priority queue): f(S) = 0

Node S is popped from the frontier.

f(A) = g(A) + h1(A) = c(S,A) + h1(A) = 5 + 0 = 5

f(B) = g(B) + h1(B) = c(S, B) + h1(B) = 2 + 0 = 2

Frontier: f(B) = 2, f(A) = 5

Node B is popped from the frontier.

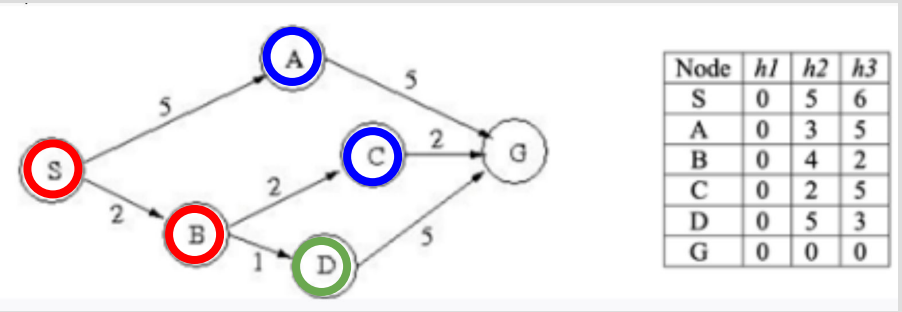
f(C) = g(C) + h1(C) = c(S, B) + c(B, C) + h1(C) = 2 + 2 + 0 = 4

f(D) = g(D) + h1(D) = c(S, B) + c(B, D) + h1(D) = 2 + 1 + 0 = 3

Frontier: f(D) = 3, f(C) = 4, f(A) = 5

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, D) + c(D, G) + h1(G) = 2 + 1 + 5 + 0 = 8

Frontier: f(C) = 4, f(A) = 5, f(G) = 8

Node C is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, C) + c(C, G) + h1(G) = 2 + 2 + 2 + 0 = 6

Frontier: f(A) = 5, f(G) = 6

Node A is popped from the frontier.

f(G) = g(G) + h1(G) = c(S,A) + c(A, G) + h1(G) = 5 + 5 + 0 = 10

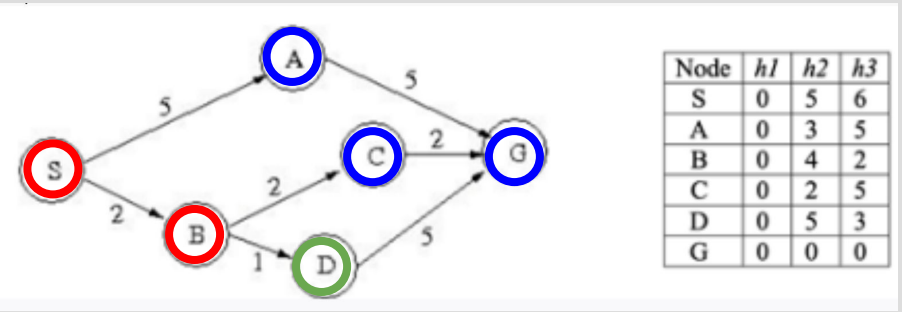
Frontier: f(G) = 6

Node G is popped from the frontier.

Solution: S, B, C, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, D) + c(D, G) + h1(G) = 2 + 1 + 5 + 0 = 8

Frontier: f(C) = 4, f(A) = 5, f(G) = 8

Node C is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, C) + c(C, G) + h1(G) = 2 + 2 + 2 + 0 = 6

Frontier: f(A) = 5, f(G) = 6

Node A is popped from the frontier.

f(G) = g(G) + h1(G) = c(S,A) + c(A, G) + h1(G) = 5 + 5 + 0 = 10

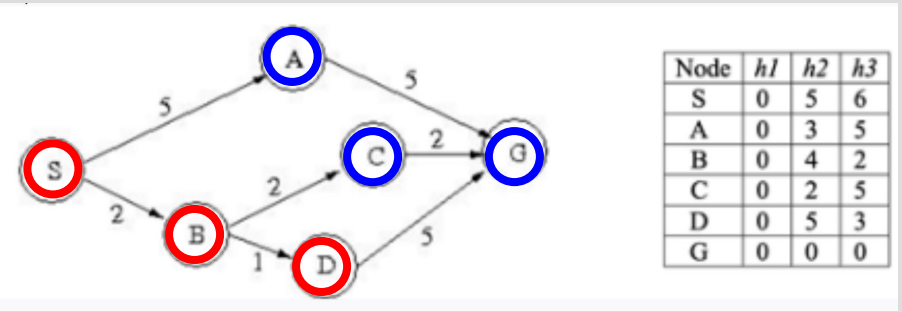
Frontier: f(G) = 6

Node G is popped from the frontier.

Solution: S, B, C, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, D) + c(D, G) + h1(G) = 2 + 1 + 5 + 0 = 8

Frontier: f(C) = 4, f(A) = 5, f(G) = 8

Node C is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, C) + c(C, G) + h1(G) = 2 + 2 + 2 + 0 = 6

Frontier: f(A) = 5, f(G) = 6

Node A is popped from the frontier.

f(G) = g(G) + h1(G) = c(S,A) + c(A, G) + h1(G) = 5 + 5 + 0 = 10

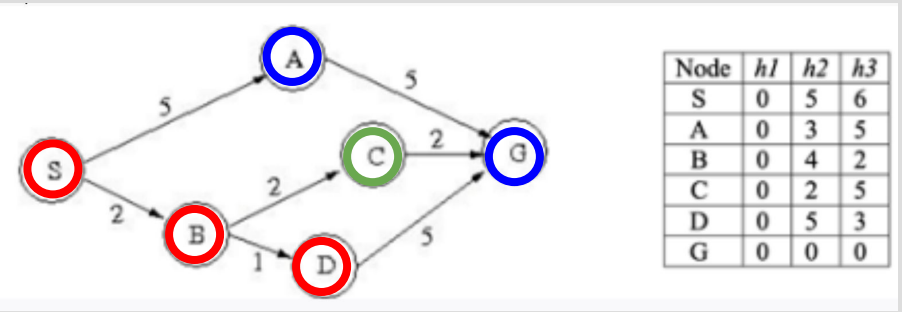
Frontier: f(G) = 6

Node G is popped from the frontier.

Solution: S, B, C, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, D) + c(D, G) + h1(G) = 2 + 1 + 5 + 0 = 8

Frontier: f(C) = 4, f(A) = 5, f(G) = 8

Node C is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, C) + c(C, G) + h1(G) = 2 + 2 + 2 + 0 = 6

Frontier: f(A) = 5, f(G) = 6

Node A is popped from the frontier.

f(G) = g(G) + h1(G) = c(S,A) + c(A, G) + h1(G) = 5 + 5 + 0 = 10

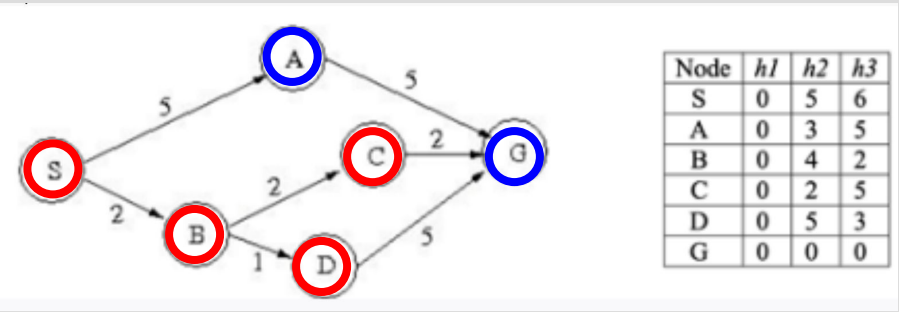
Frontier: f(G) = 6

Node G is popped from the frontier.

Solution: S, B, C, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, D) + c(D, G) + h1(G) = 2 + 1 + 5 + 0 = 8

Frontier: f(C) = 4, f(A) = 5, f(G) = 8

Node C is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, C) + c(C, G) + h1(G) = 2 + 2 + 2 + 0 = 6

Frontier: f(A) = 5, f(G) = 6

Node A is popped from the frontier.

f(G) = g(G) + h1(G) = c(S,A) + c(A, G) + h1(G) = 5 + 5 + 0 = 10

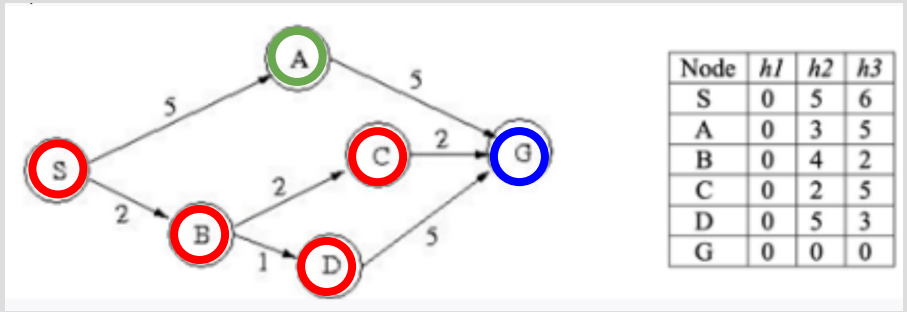
Frontier: f(G) = 6

Node G is popped from the frontier.

Solution: S, B, C, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, D) + c(D, G) + h1(G) = 2 + 1 + 5 + 0 = 8

Frontier: f(C) = 4, f(A) = 5, f(G) = 8

Node C is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, C) + c(C, G) + h1(G) = 2 + 2 + 2 + 0 = 6

Frontier: f(A) = 5, f(G) = 6

Node A is popped from the frontier.

f(G) = g(G) + h1(G) = c(S,A) + c(A, G) + h1(G) = 5 + 5 + 0 = 10

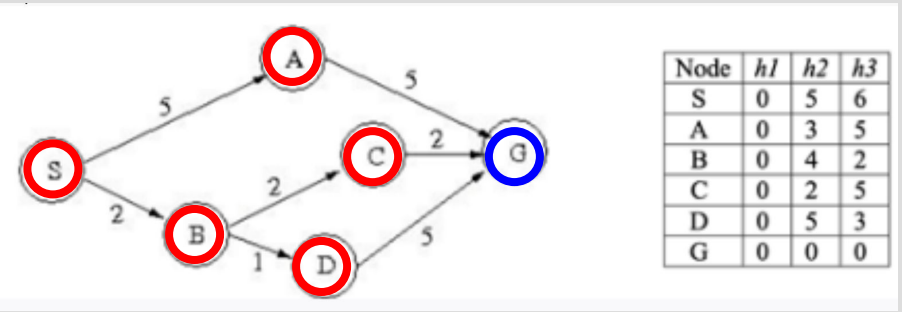
Frontier: f(G) = 6

Node G is popped from the frontier.

Solution: S, B, C, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, D) + c(D, G) + h1(G) = 2 + 1 + 5 + 0 = 8

Frontier: f(C) = 4, f(A) = 5, f(G) = 8

Node C is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, C) + c(C, G) + h1(G) = 2 + 2 + 2 + 0 = 6

Frontier: f(A) = 5, f(G) = 6

Node A is popped from the frontier.

f(G) = g(G) + h1(G) = c(S,A) + c(A, G) + h1(G) = 5 + 5 + 0 = 10

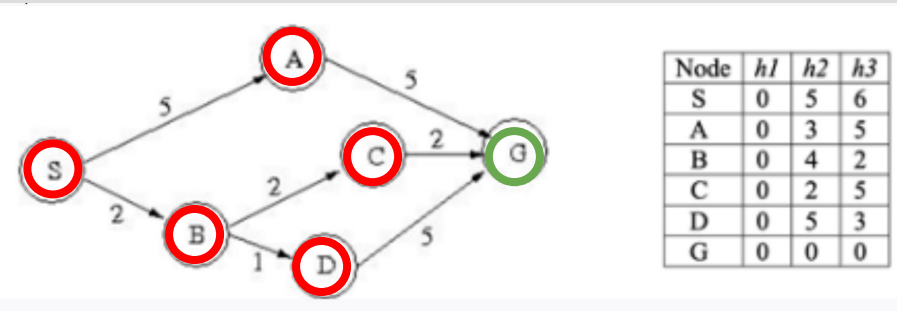
Frontier: f(G) = 6

Node G is popped from the frontier.

Solution: S, B, C, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, D) + c(D, G) + h1(G) = 2 + 1 + 5 + 0 = 8

Frontier: f(C) = 4, f(A) = 5, f(G) = 8

Node C is popped from the frontier.

f(G) = g(G) + h1(G) = c(S, B) + c(B, C) + c(C, G) + h1(G) = 2 + 2 + 2 + 0 = 6

Frontier: f(A) = 5, f(G) = 6

Node A is popped from the frontier.

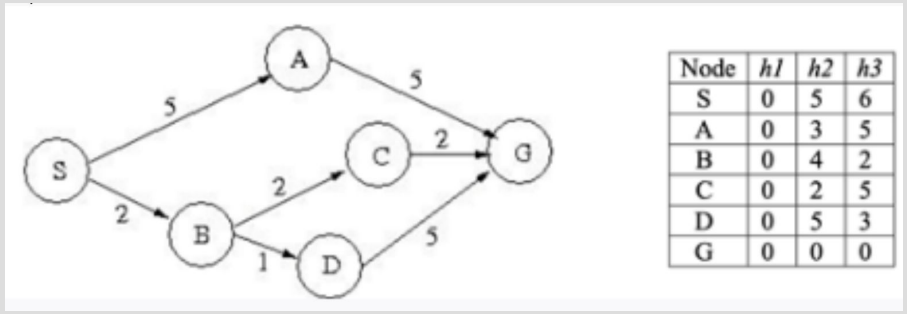
f(G) = g(G) + h1(G) = c(S,A) + c(A, G) + h1(G) = 5 + 5 + 0 = 10

Frontier: f(G) = 6

Node G is popped from the frontier.

Solution: S, B, C, G

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S, A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

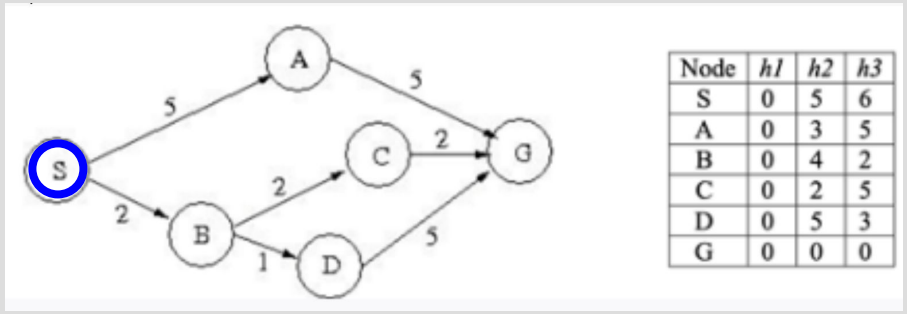
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S,A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

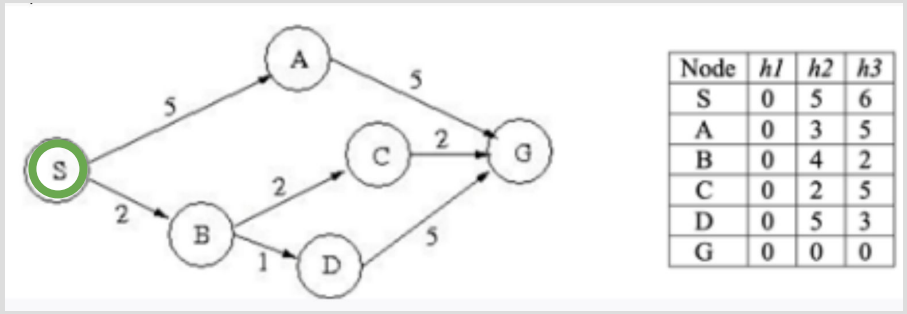
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S,A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

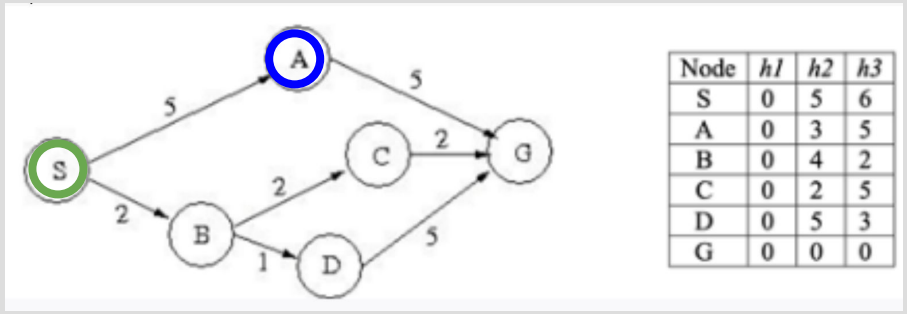
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S, A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

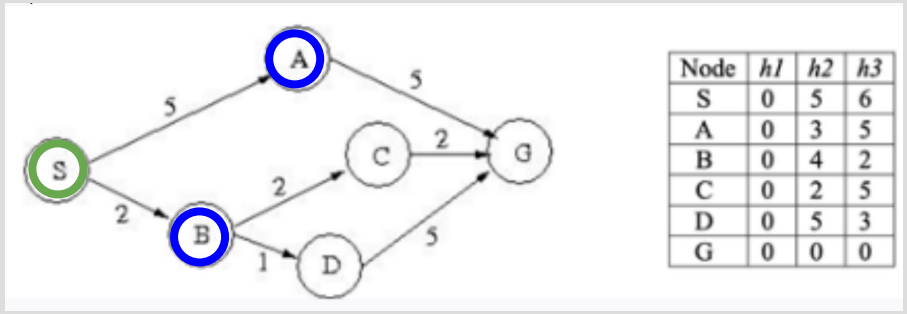
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S, A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

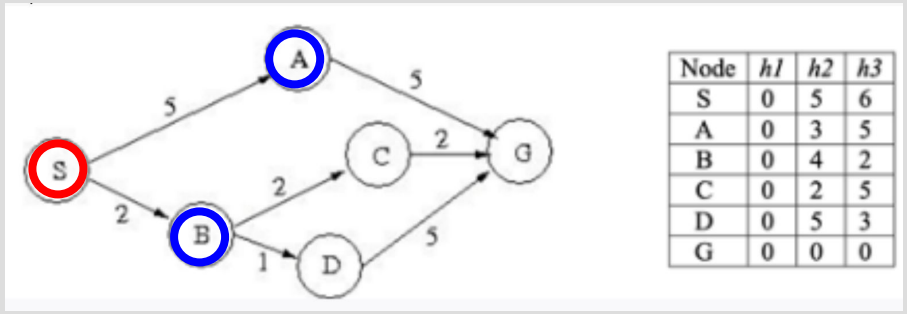
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S,A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

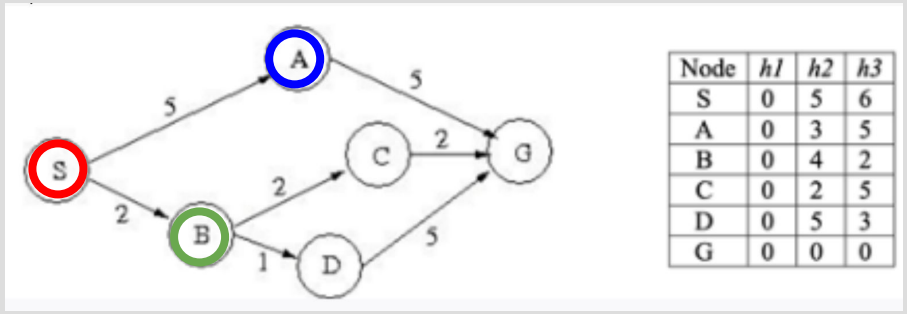
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S, A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

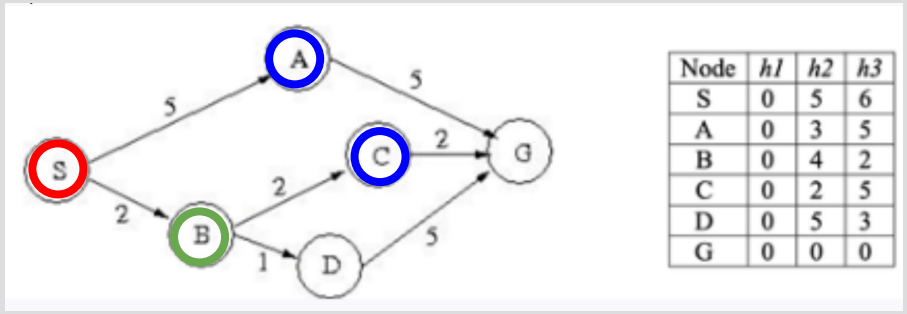
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S,A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

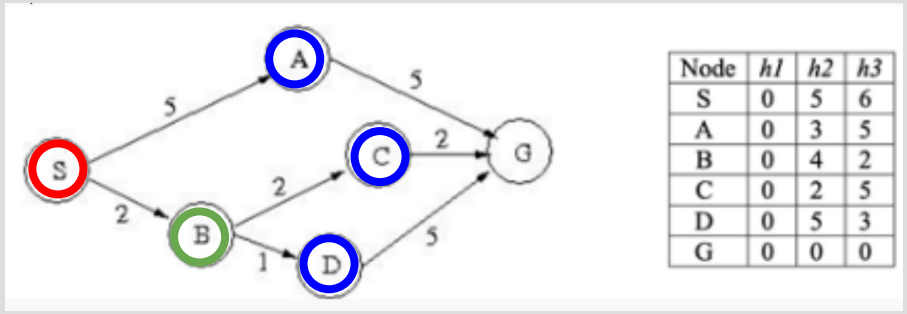
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S,A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

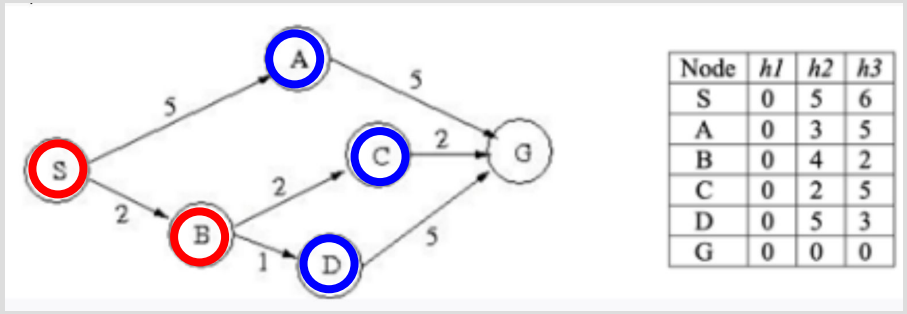
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

What solution path is found by A* search using h3?

Frontier (priority queue): f(S) = 6

Node S is popped from the frontier.

f(A) = g(A) + h3(A) = c(S,A) + h3(A) = 5 + 5 = 10

f(B) = g(B) + h3(B) = c(S, B) + h3(B) = 2 + 2 = 4

Frontier: f(B) = 4, f(A) = 10

Node B is popped from the frontier.

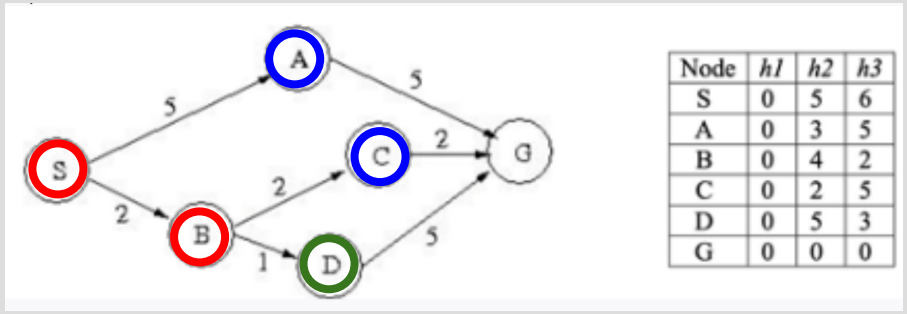
f(C) = g(C) + h3(C) = c(S, B) + c(B, C) + h3(C) = 2 + 2 + 5 = 9

f(D) = g(D) + h3(D) = c(S, B) + c(B, D) + h3(D) = 2 + 1 + 3 = 6

Frontier: f(D) = 6, f(C) = 9, f(A) = 10

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h3(G) = c(S, B) + c(B, D) + c(D, G) + h3(G) = 2 + 1 + 5 + 0 = 8

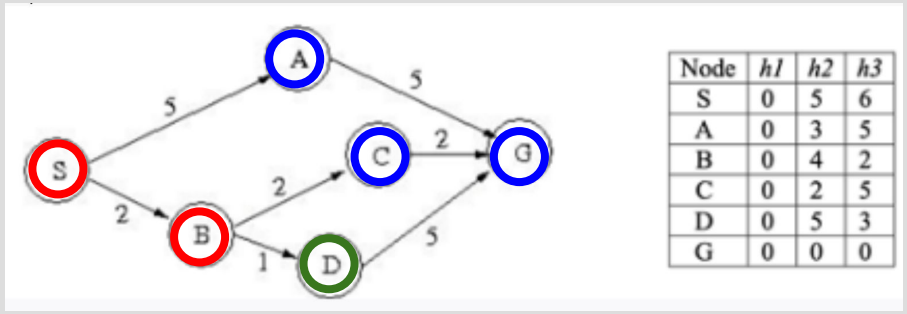
Frontier: f(G) = 8, f(C) = 9, f(A) = 10

Node G is popped from the frontier.

Solution: S, B, D, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h3(G) = c(S, B) + c(B, D) + c(D, G) + h3(G) = 2 + 1 + 5 + 0 = 8

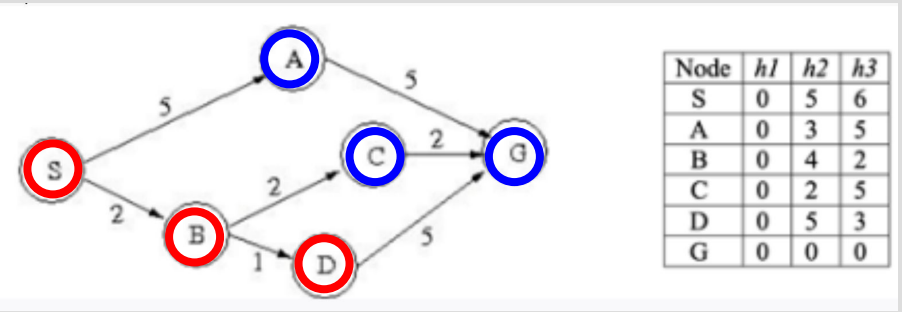
Frontier: f(G) = 8, f(C) = 9, f(A) = 10

Node G is popped from the frontier.

Solution: S, B, D, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h3(G) = c(S, B) + c(B, D) + c(D, G) + h3(G) = 2 + 1 + 5 + 0 = 8

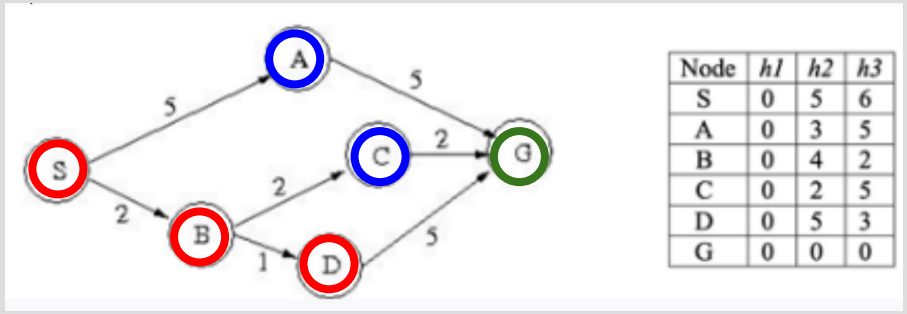
Frontier: f(G) = 8, f(C) = 9, f(A) = 10

Node G is popped from the frontier.

Solution: S, B, D, G

Frontier
Current
Explored

GREEDY BEST-FIRST AND A* EXAMPLES



Node	*h1*	*h2*	*h3*
S	0	5	6
A	0	3	5
B	0	4	2
C	0	2	5
D	0	5	3
G	0	0	0

Node D is popped from the frontier.

f(G) = g(G) + h3(G) = c(S, B) + c(B, D) + c(D, G) + h3(G) = 2 + 1 + 5 + 0 = 8

Frontier: f(G) = 8, f(C) = 9, f(A) = 10

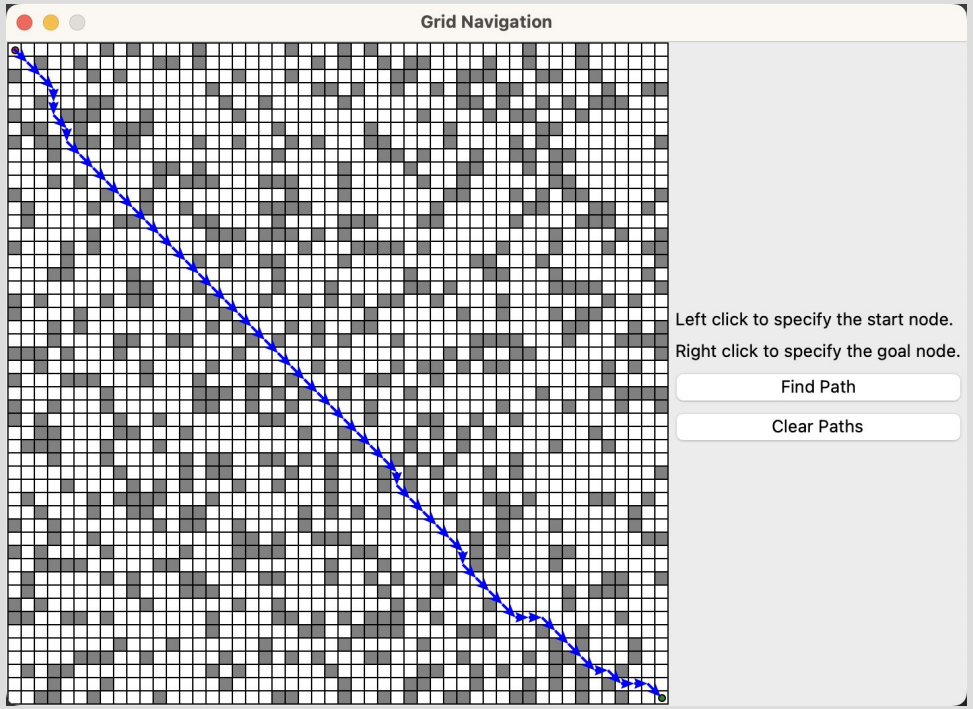
Node G is popped from the frontier.

Solution: S, B, D, G

A* GRAPH SEARCH EXAMPLE (#113)

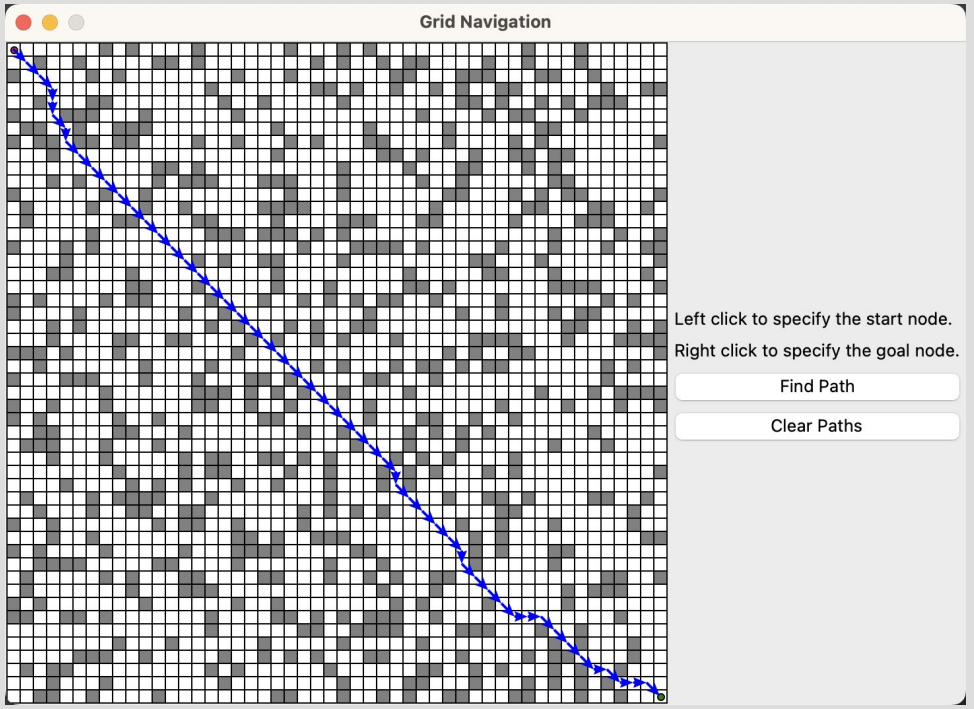
Consistent heuristic (Euclidean distance) with closed set approach

(nodes are not revisited)



Consistent heuristic (Euclidean distance) with open set approach

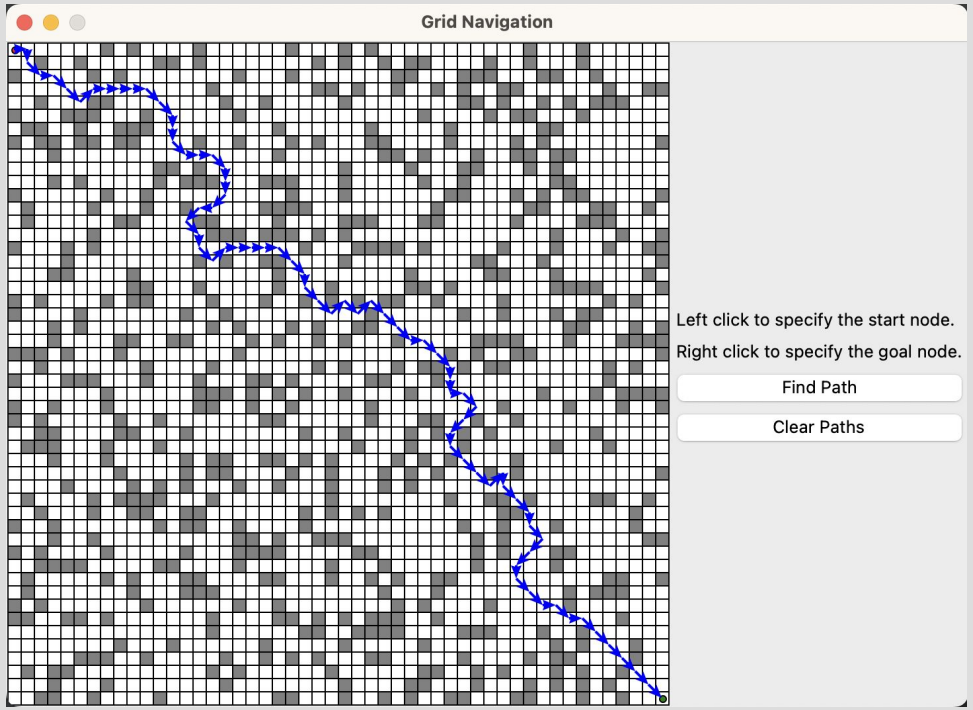
(nodes are revisited)



A* GRAPH SEARCH EXAMPLE (#113)

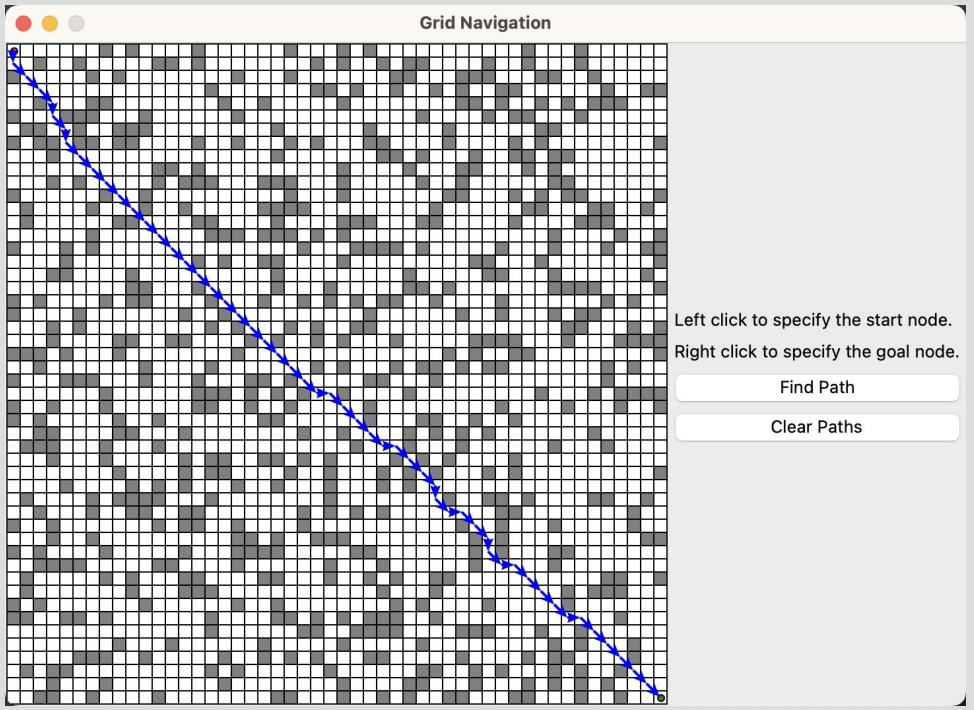
Admissible, inconsistent heuristic with closed set approach

(nodes are not revisited)

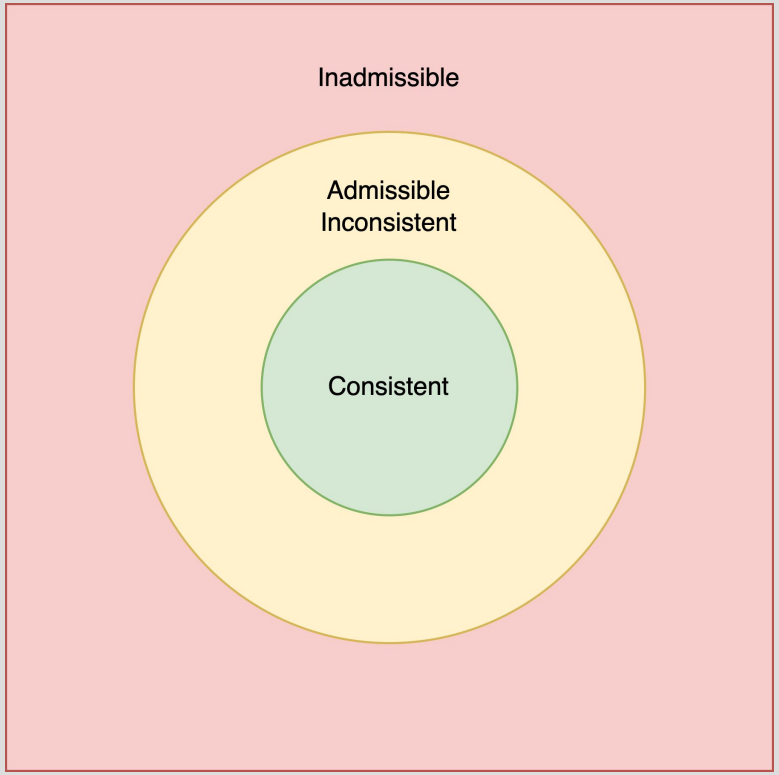


Admissible, inconsistent heuristic with open set approach

(nodes are revisited)



HEURISTICS (#113)



Consistency ⇒ **Admissibility**
Inadmissibility ⇒ **Inconsistency**
Admissibility ⇏ **Consistency**

The graph illustrates different types of heuristics:

- The inner **green circle** represents **consistent** heuristics.
- The outer **yellow circle** encompasses **admissible** heuristics.
- The **yellow region** denotes **admissible** but **inconsistent** heuristics.
- The **red region** indicates **inadmissible** heuristics.

SUMMARY (#113)

Tree Search:

- Assumes a **tree** structure that contains no cycles.
- Does not track visited nodes, which may lead to unnecessary revisits and infinite loops if applied to graphs with cycles.
- May be less efficient due to potential repeated exploration of nodes.
- Requires **admissible** heuristics in A* **tree** search.

Graph Search:

- Assumes a **graph** structure that may include cycles.
- Keeps track of visited nodes to avoid redundant exploration and infinite loops.
- Enhances efficiency by preventing unnecessary reprocessing of nodes.
- Requires **consistent** heuristics in A* **graph** search when implementing the **closed** set approach.
- Requires **admissible** heuristics in A* **graph** search when implementing the **open** set approach.

Closed Set Approach:

- Uses an explored/visited set data structure to track processed nodes.
- Does not allow nodes to be revisited.
- Requires **consistent** heuristics in A* **graph** search.

Open Set Approach:

- Uses a cost_so_far hashmap data structure to track the cost g(n) to each node n.
- Allows nodes to be revisited when a lower-cost path is discovered.
- Requires **admissible** heuristics in A* **graph** search.

SUMMARY (#113)

A* Tree Search:

- **Tree** search + **consistent** heuristic: Guarantees *cost-optimality*.
- **Tree** search + **admissible**, **inconsistent** heuristic: Guarantees *cost-optimality*.

A* Graph Search:

- **Graph** search + **consistent** heuristic + **closed** set: Guarantees *cost-optimality*.
- **Graph** search + **consistent** heuristic + **open** set: Guarantees *cost-optimality*.
- **Graph** search + **admissible**, **inconsistent** heuristic + **closed** set: May generate *suboptimal* solutions.
- **Graph** search + **admissible**, **inconsistent** heuristic + **open** set: Guarantees *cost-optimality*.

The textbook states in section 3.5.2 A* search:

"With an admissible heuristic, A* is cost-optimal, which we can show with a proof by contradiction."

A* search is complete.[11] Whether A* is cost-optimal depends on certain properties of the heuristic. A key property is **admissibility**: an **admissible heuristic** is one that *never overestimates* the cost to reach a goal. (An admissible heuristic is therefore *optimistic*.) With an admissible heuristic, A* is cost-optimal, which we can show with a proof by contradiction. Suppose the optimal path has cost C^*, but the algorithm returns a path with cost $C > C^*$. Then there must be some node n which is on the optimal path and is unexpanded (because if all the nodes on the optimal path had been expanded, then we would have returned that optimal solution). So then, using the notation $g^*(n)$ to mean the cost of the optimal path from the start to n, and $h^*(n)$ to mean the cost of the optimal path from n to the nearest goal, we have:

$$
\begin{aligned}
f(n) &> C^* \quad \text{(otherwise } n \text{ would have been expanded)} \\
f(n) &= g(n) + h(n) \quad \text{(by definition)} \\
f(n) &= g^*(n) + h(n) \quad \text{(because } n \text{ is on an optimal path)} \\
f(n) &\leq g^*(n) + h^*(n) \quad \text{(because of admissibility, } h(n) \leq h^*(n)\text{)} \\
f(n) &\leq C^* \quad \text{(by definition, } C^* = g^*(n) + h^*(n)\text{)}
\end{aligned}
$$

The first and last lines form a contradiction, so the supposition that the algorithm could return a suboptimal path must be wrong—it must be that A* returns only cost-optimal paths.

ADVERSARIAL SEARCH

Week 4

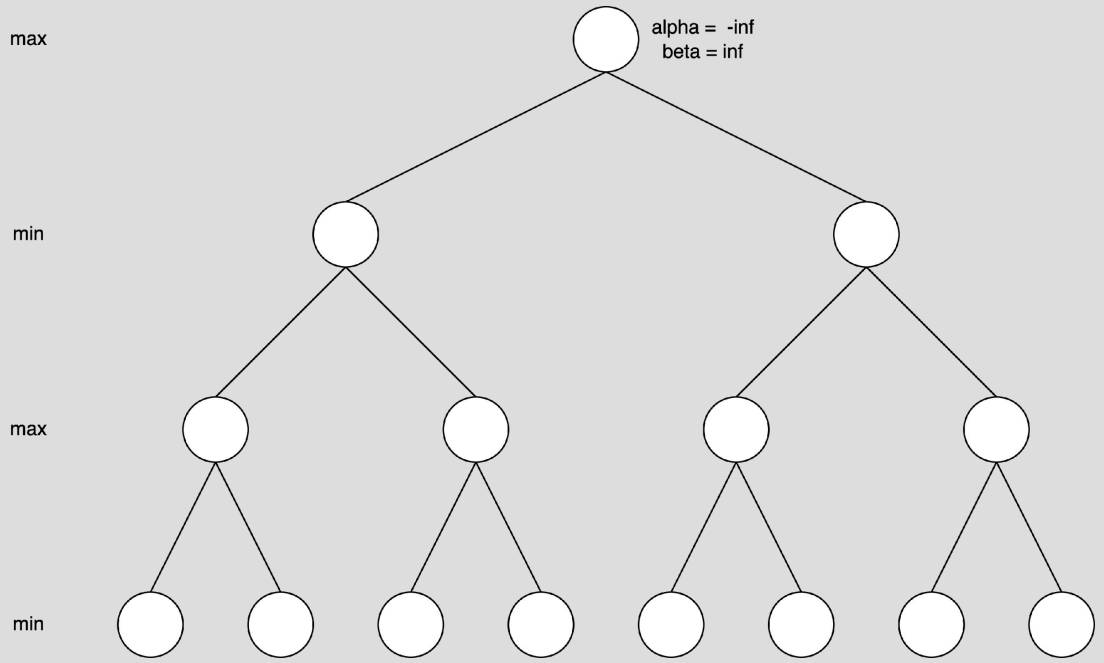
AIMA Chapter 5 "Adversarial Search" (5.1-5.3 and 5.5)

AIMA Chapter 16 "Making Simple Decisions" (16.1-16.3)

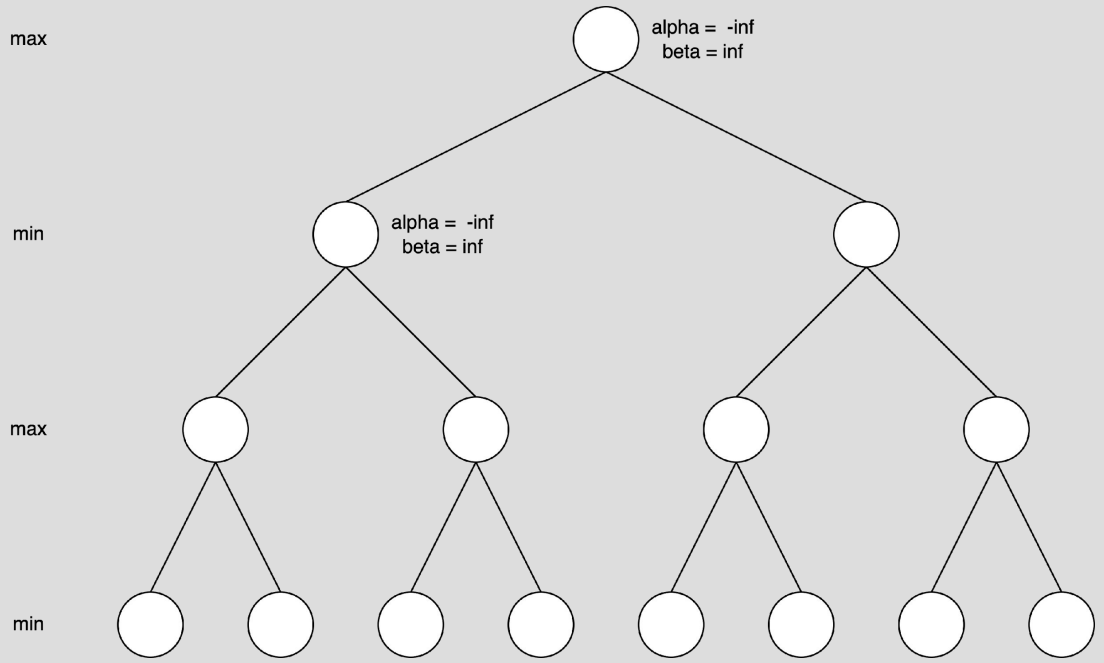
MINIMAX WITH ALPHA BETA PRUNING

- Alpha
  - The minimum score guaranteed for Max (Max gets at least alpha)
  - alpha = -inf (the initial score of Max, also the worst possible score for Max)
  - Max's goal: maximize alpha
- Beta
  - The maximum score guaranteed for Min (Min gets at most beta)
  - beta = inf (the initial score of Min, also the worst possible score for Min)
  - Min's goal: minimize beta
- Pruning condition: alpha ≥ beta

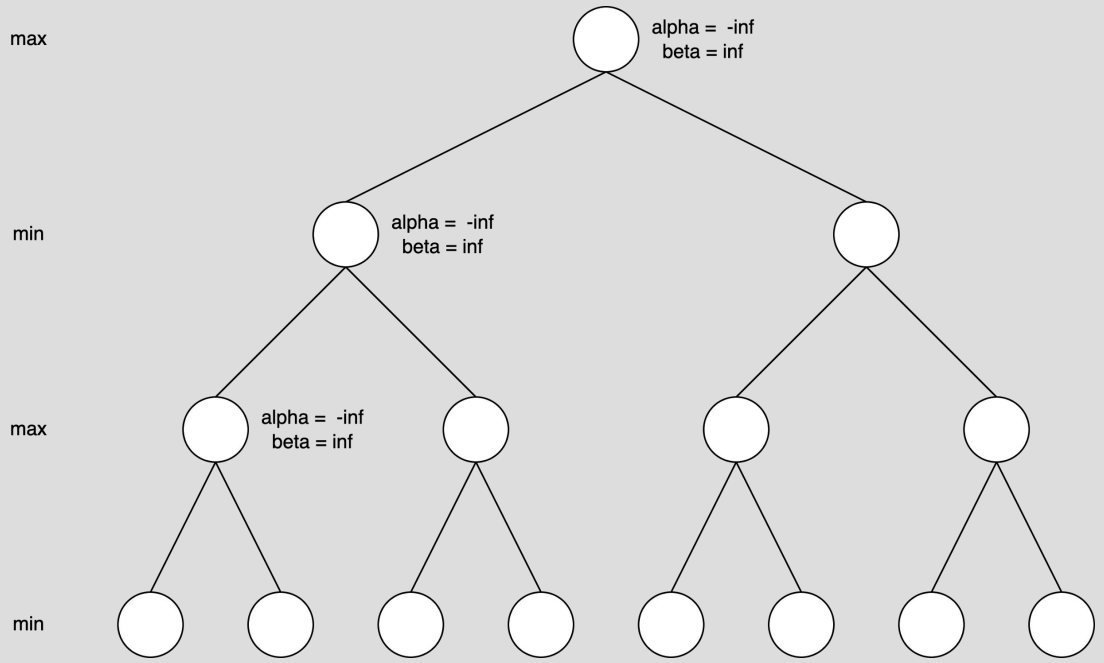
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



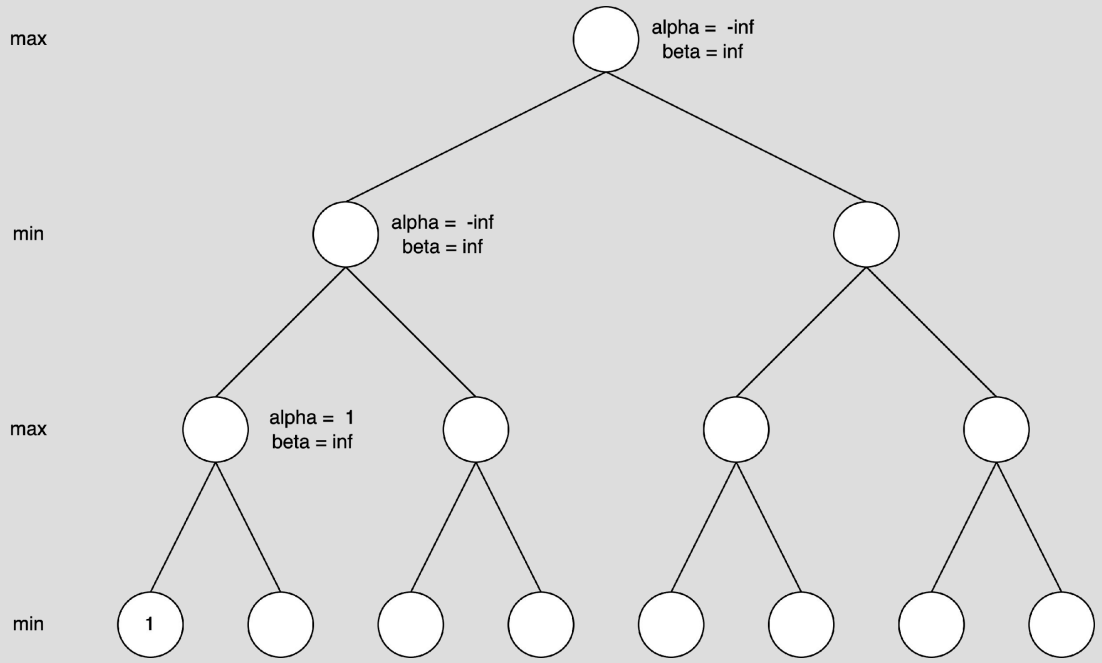
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



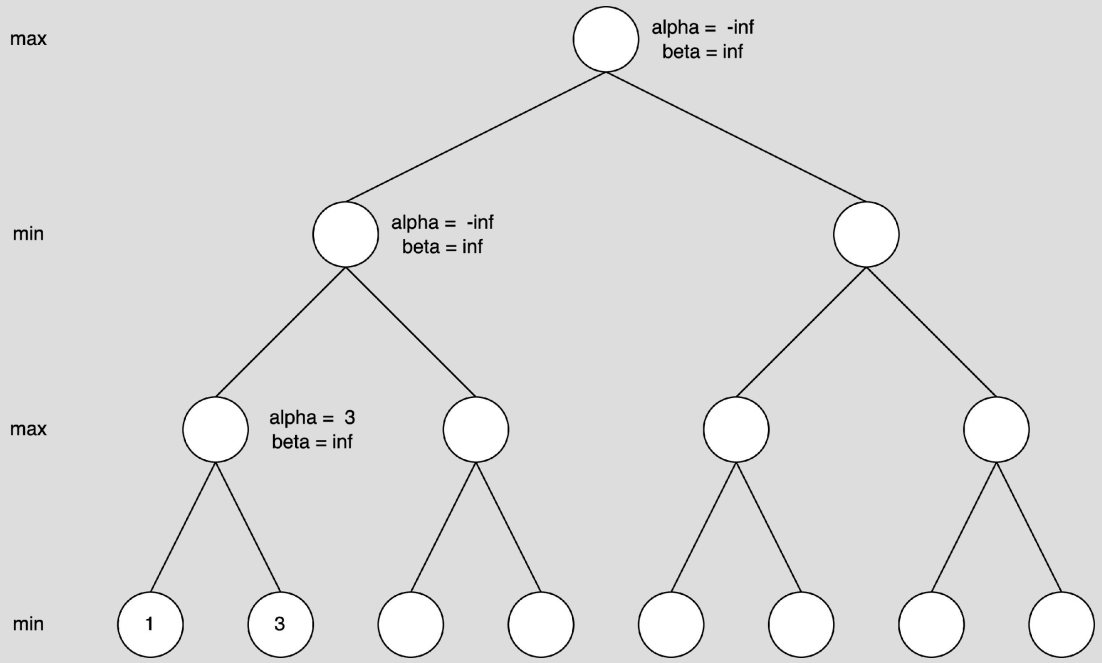
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



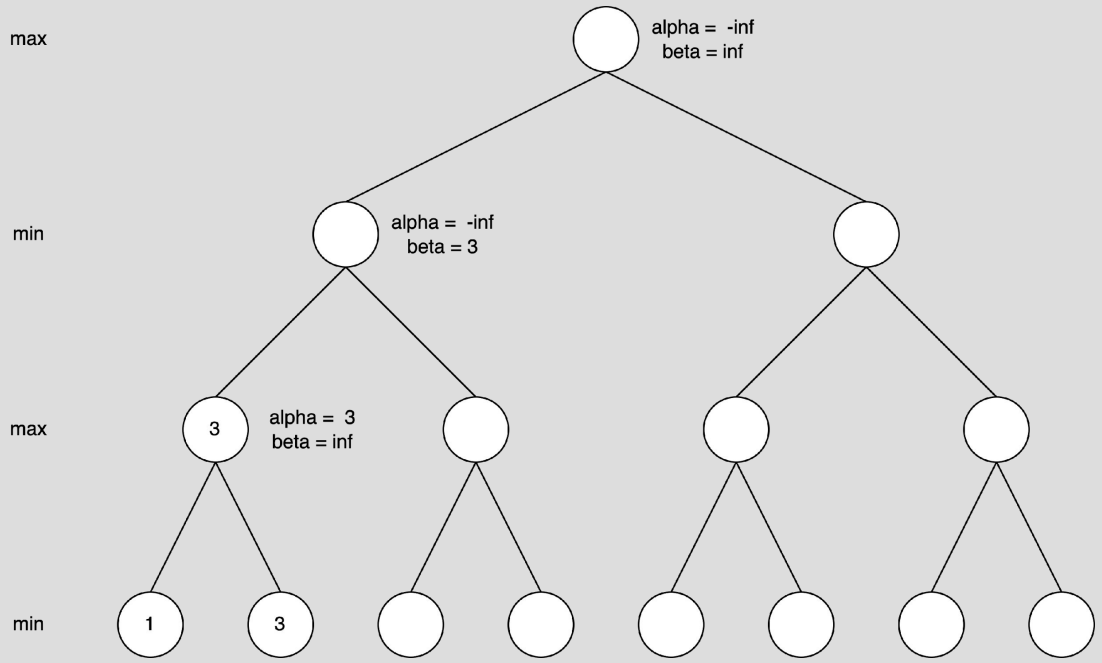
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



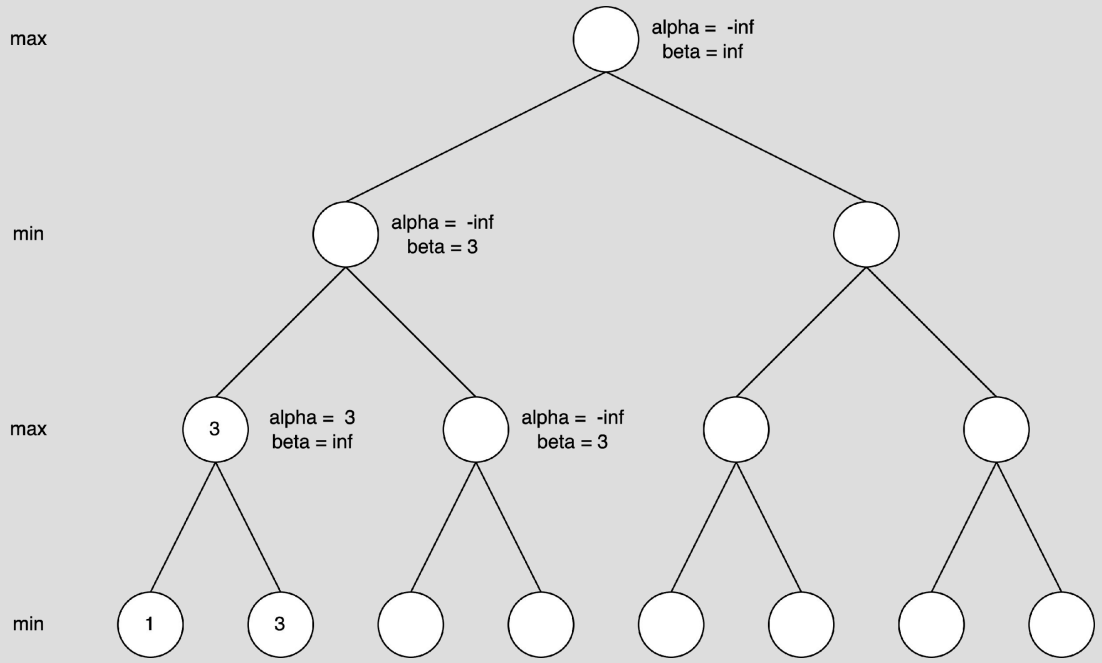
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



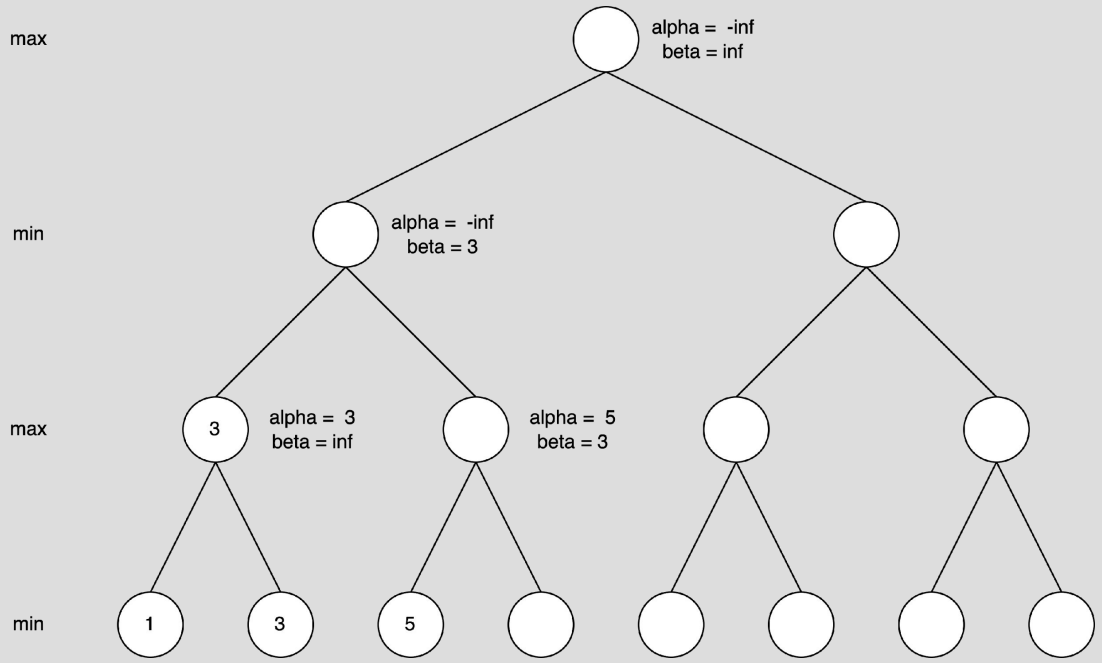
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



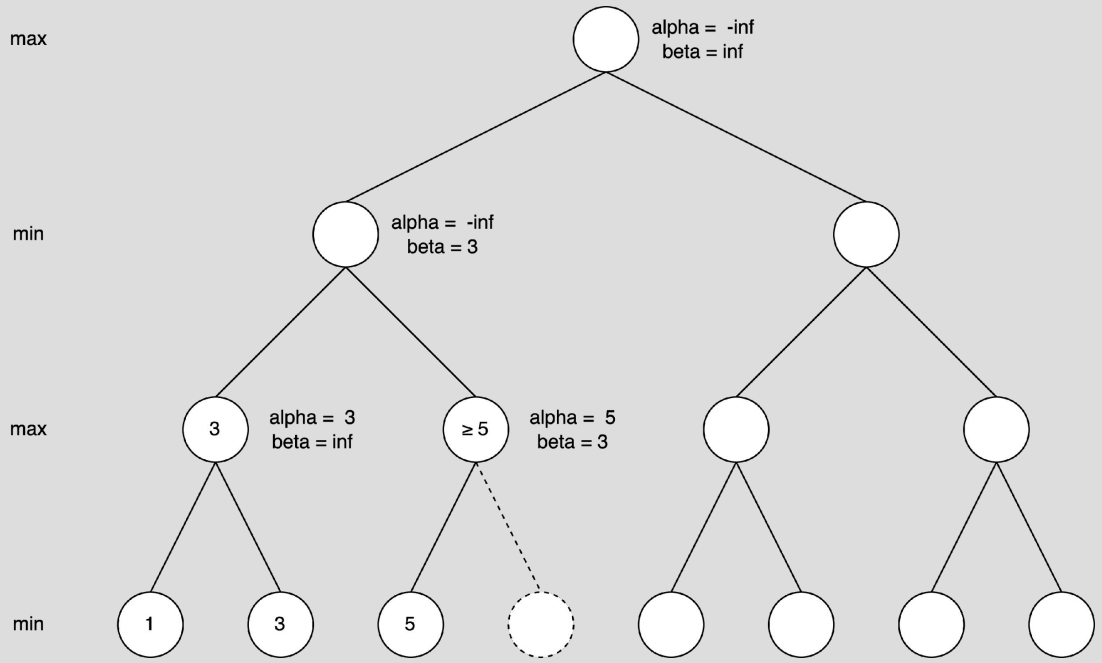
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



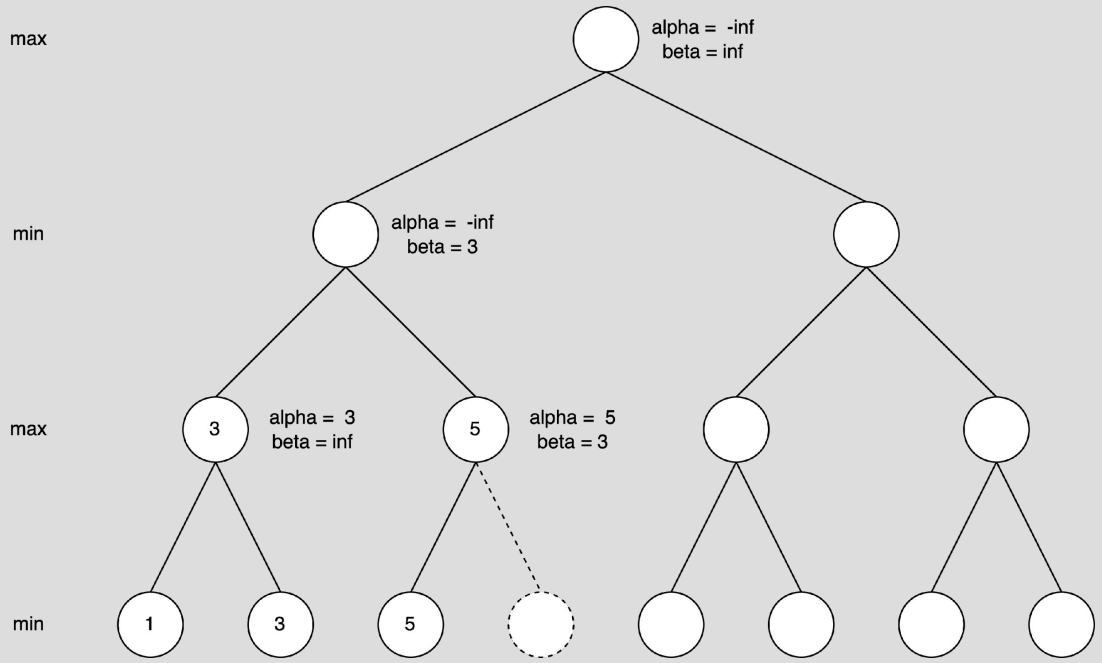
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



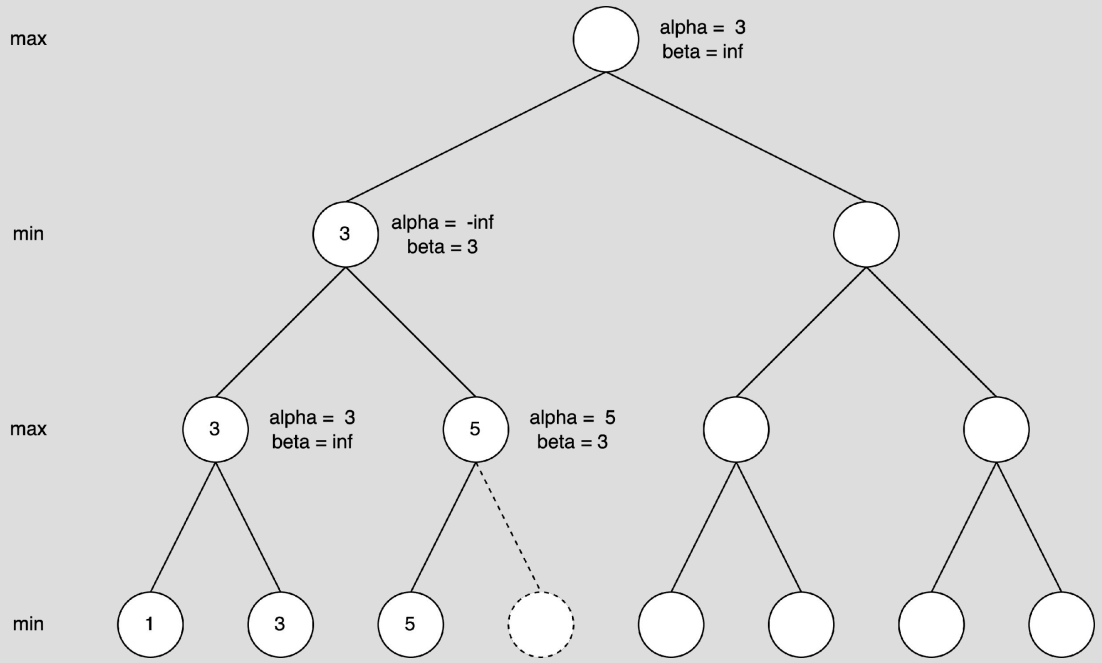
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



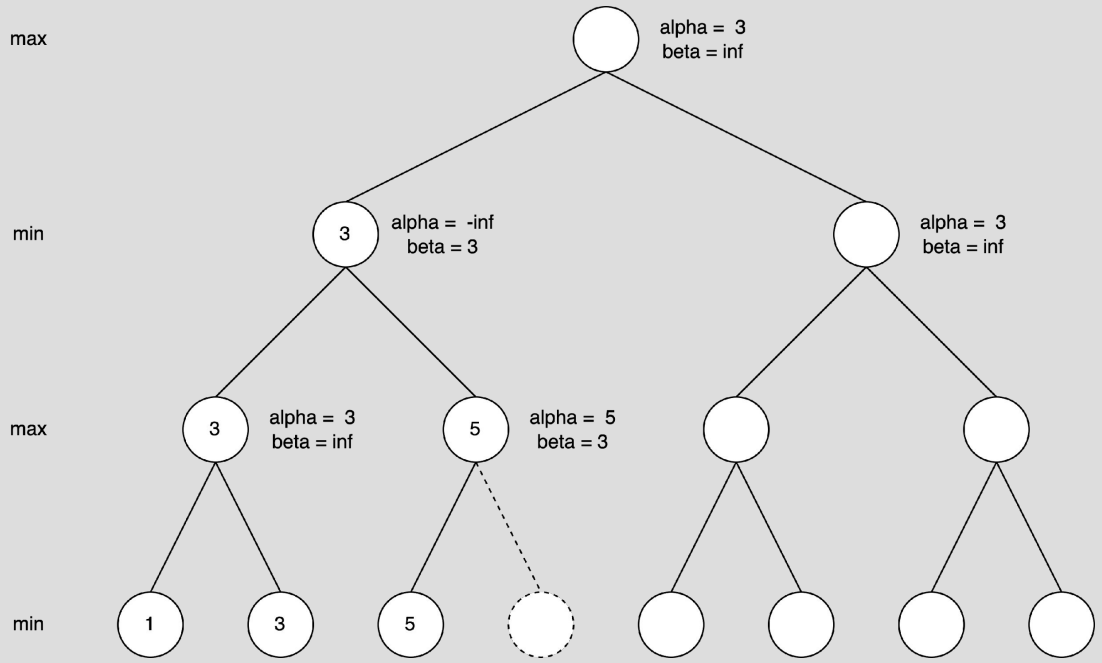
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



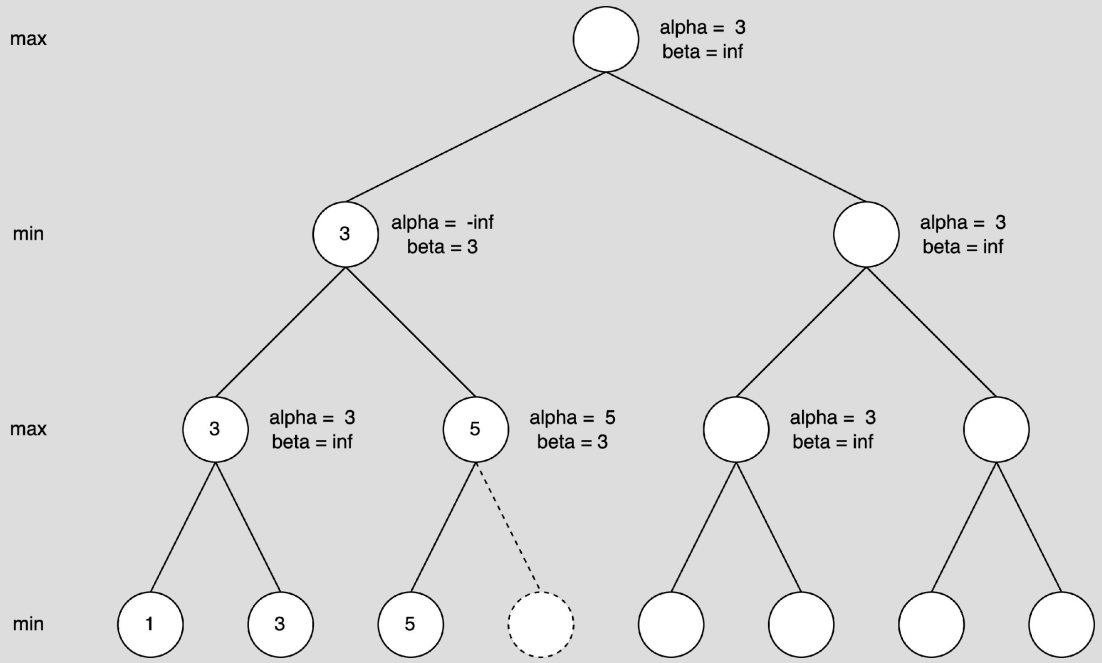
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



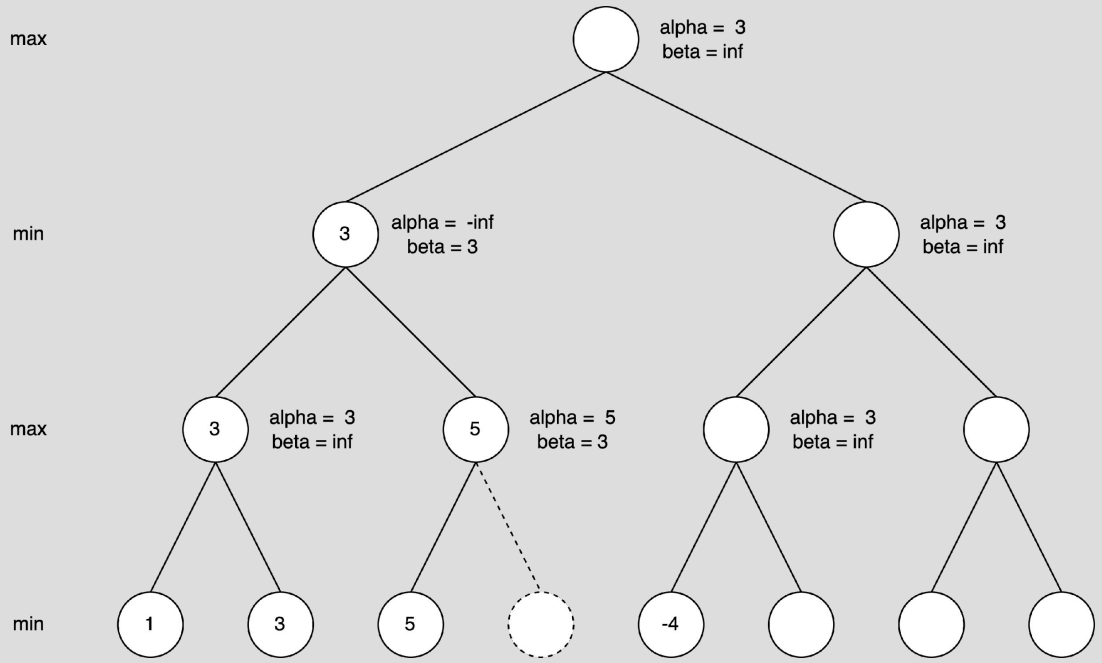
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



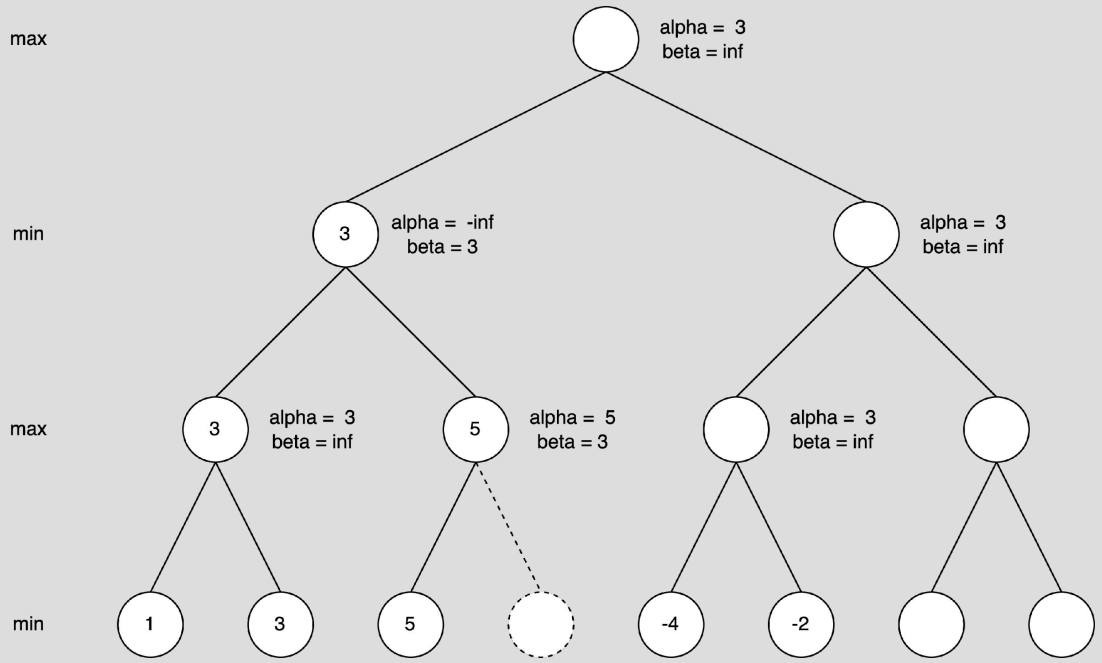
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



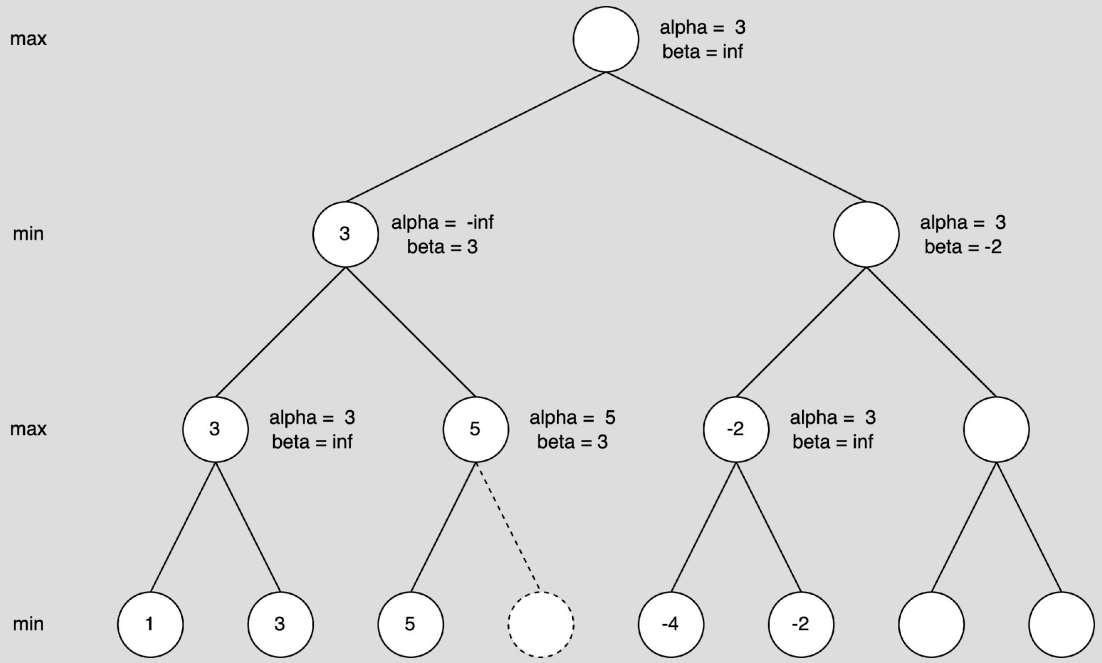
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



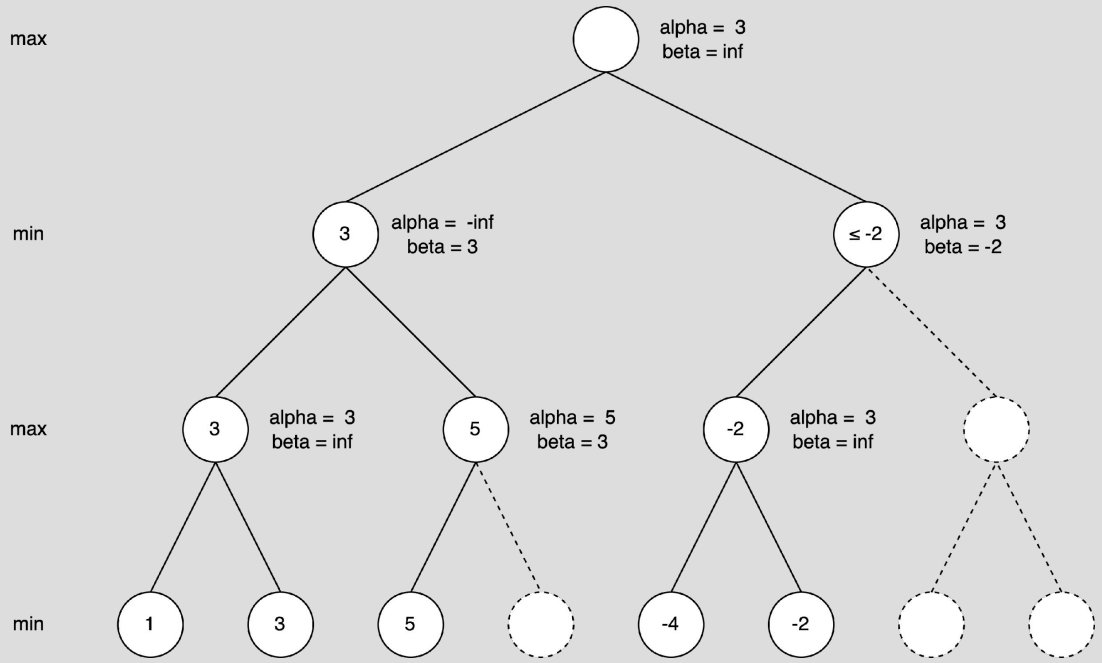
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



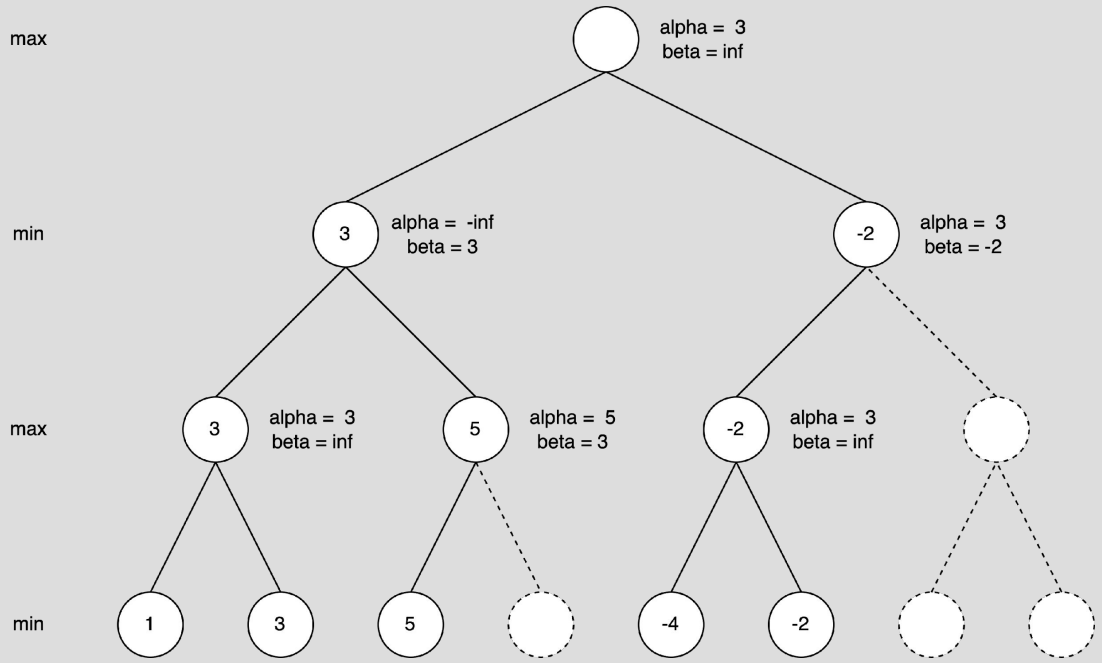
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



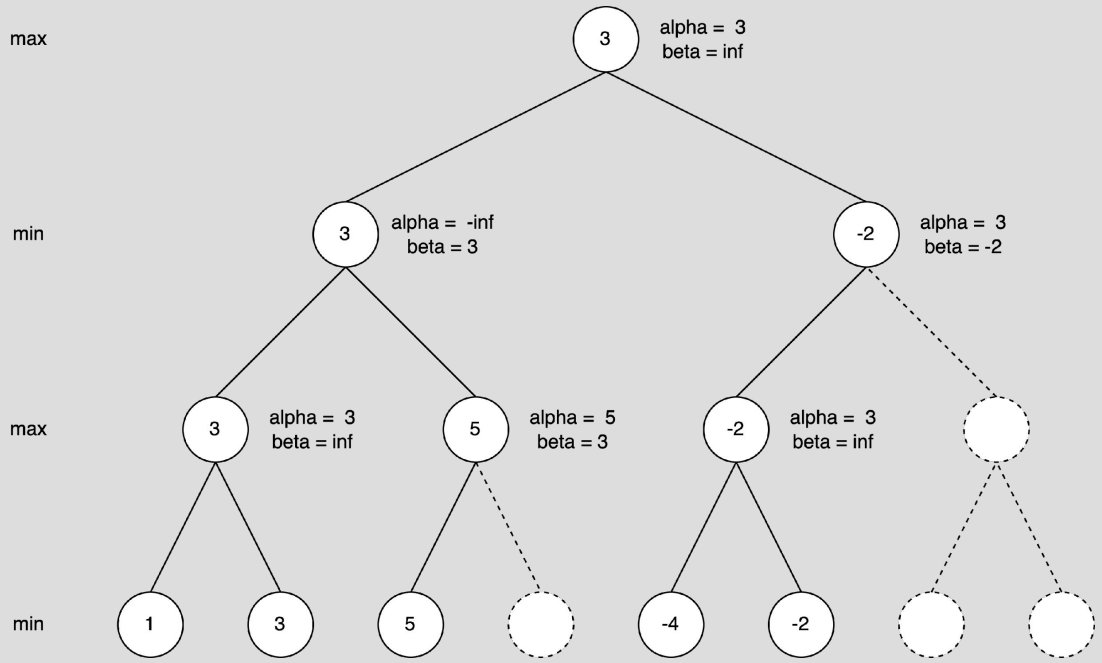
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



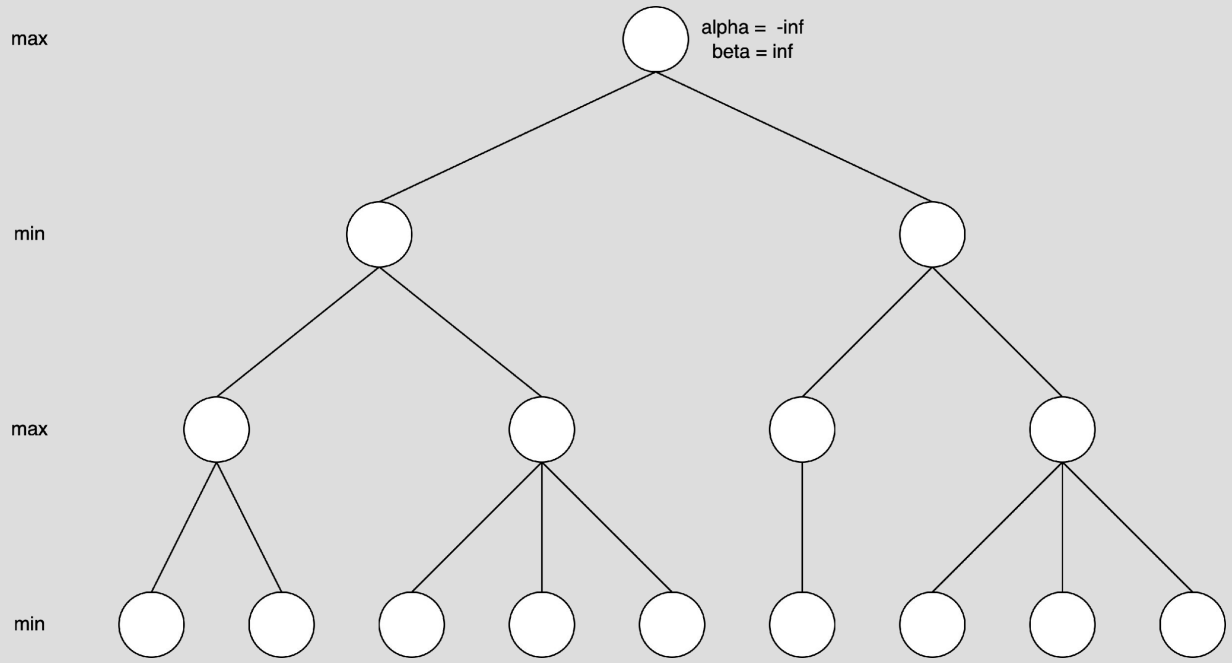
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



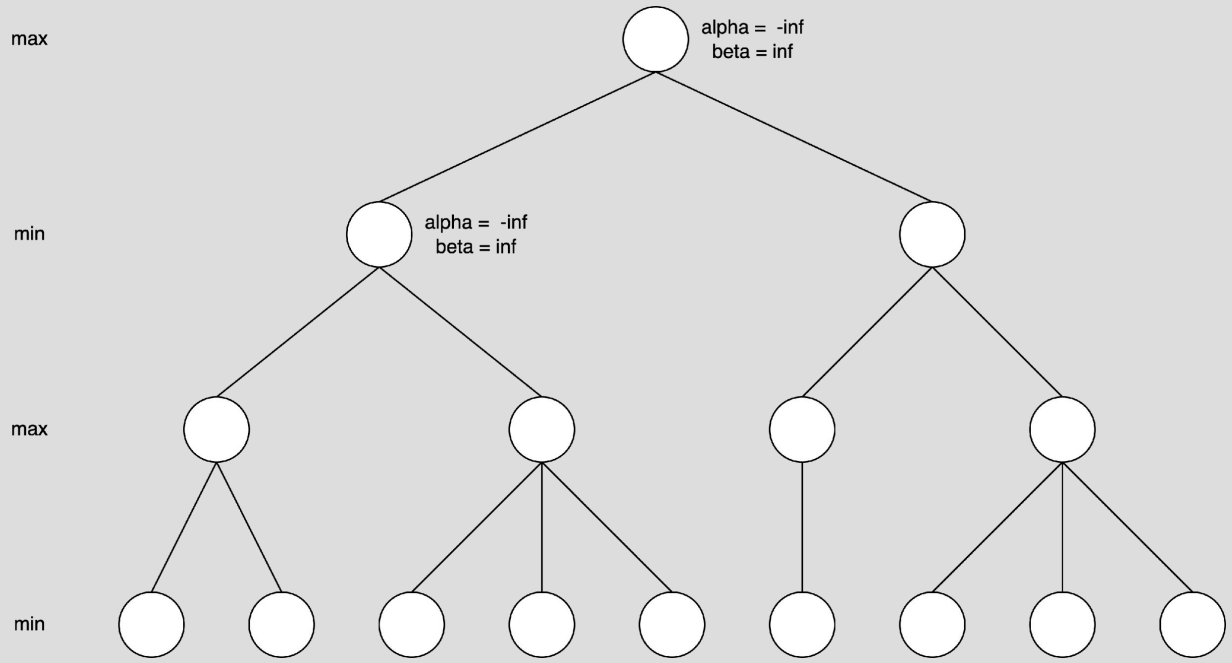
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 1



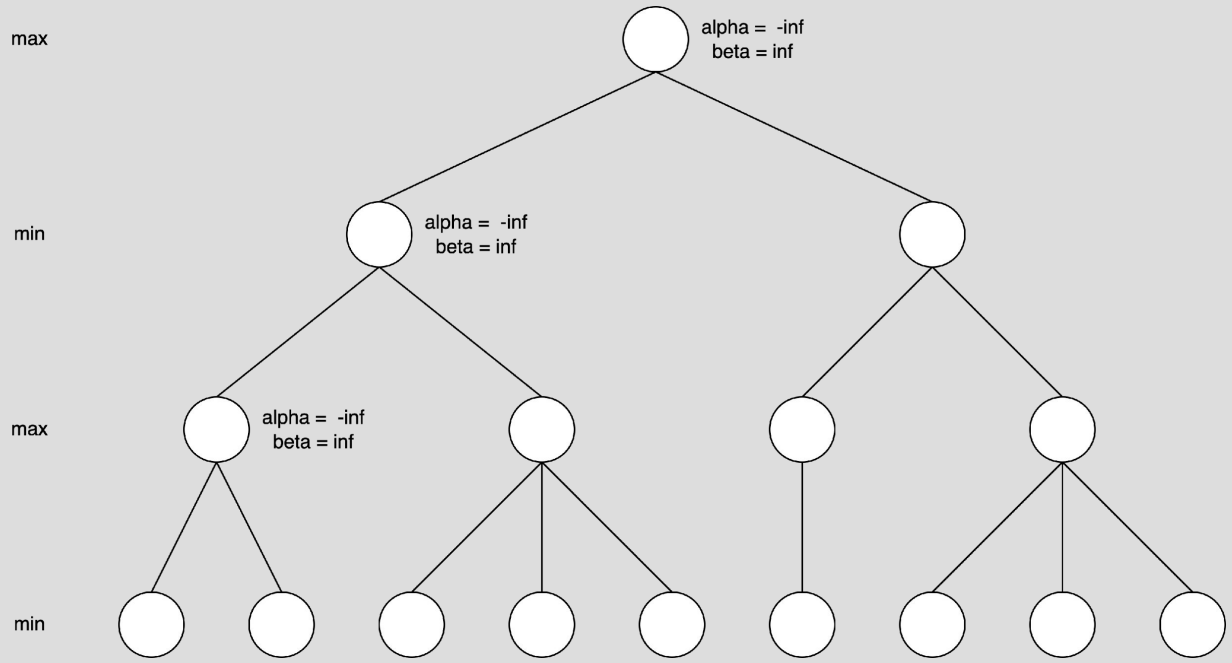
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



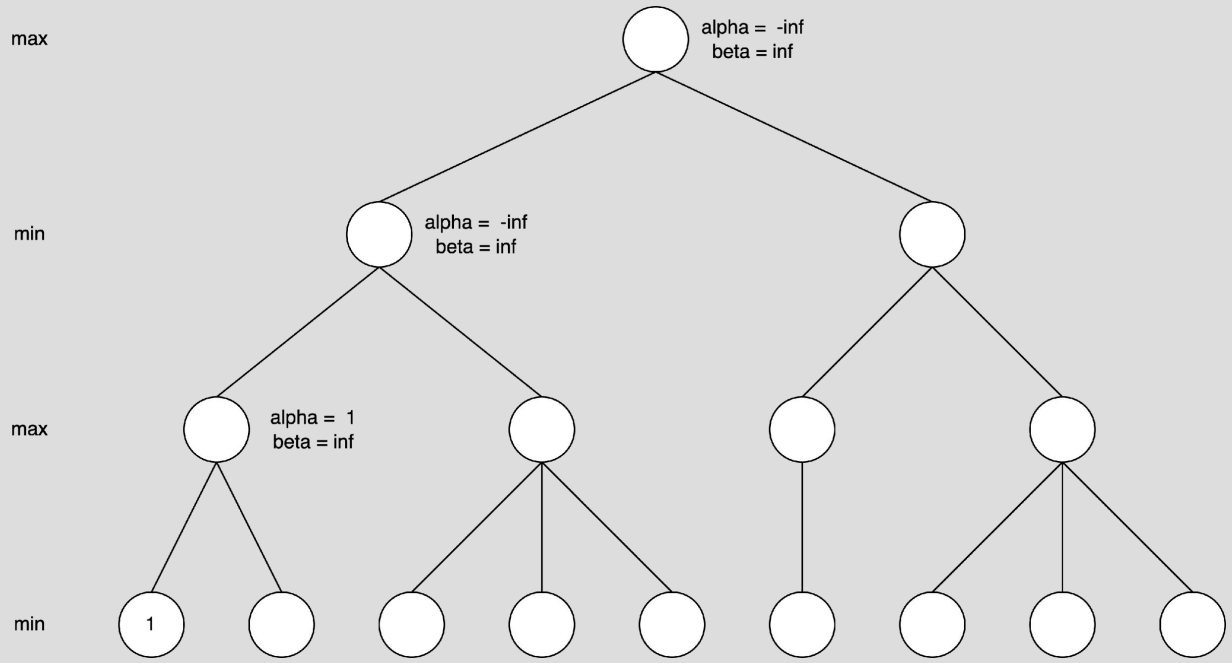
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



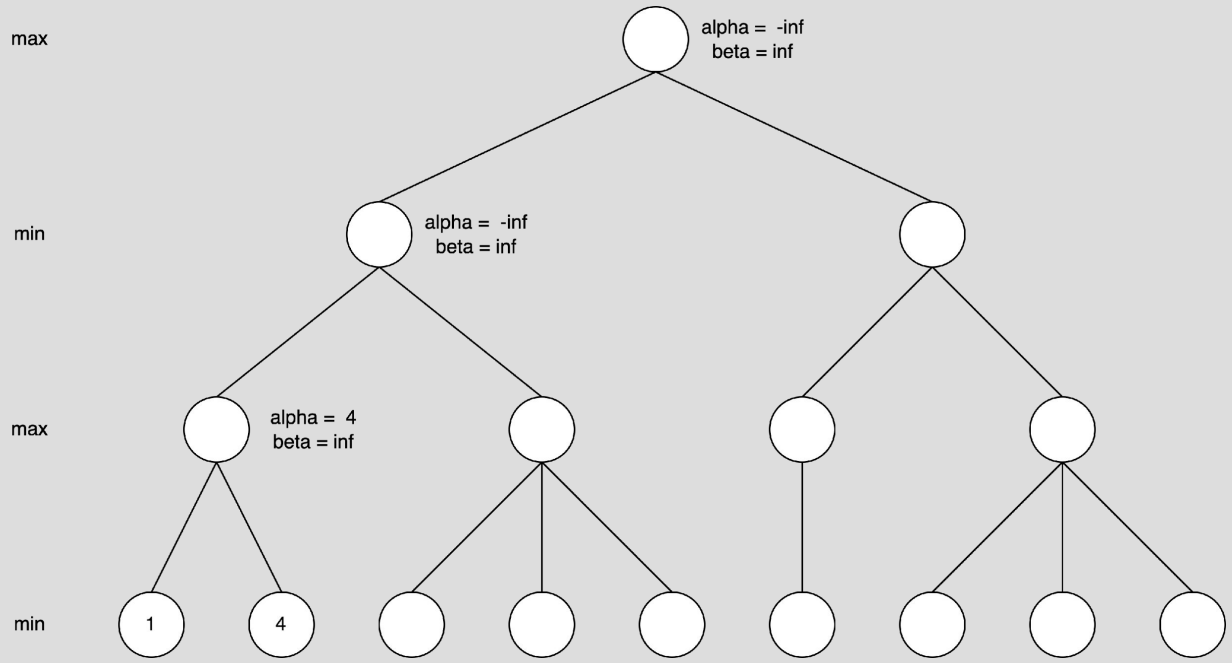
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



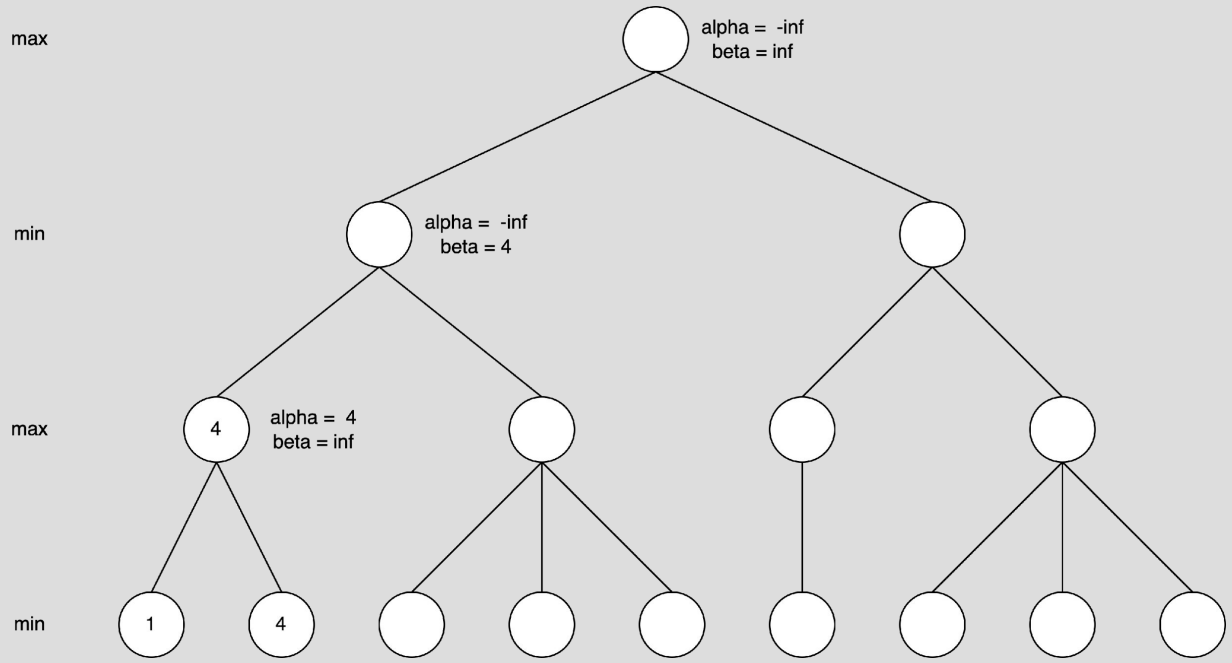
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



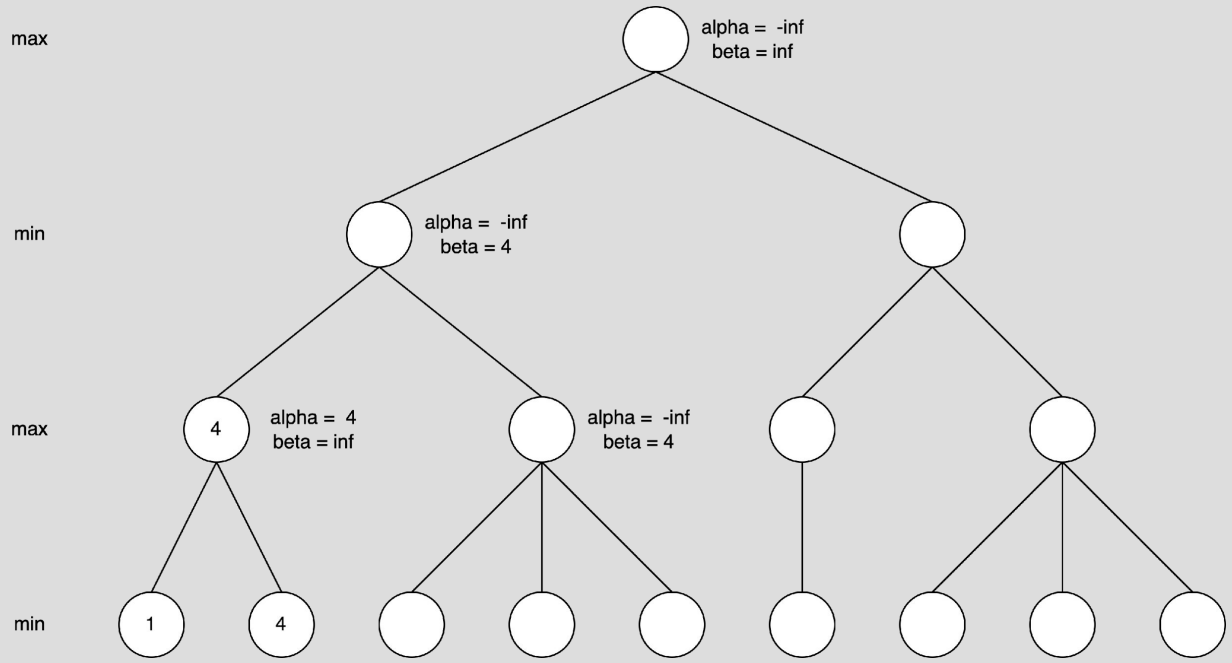
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



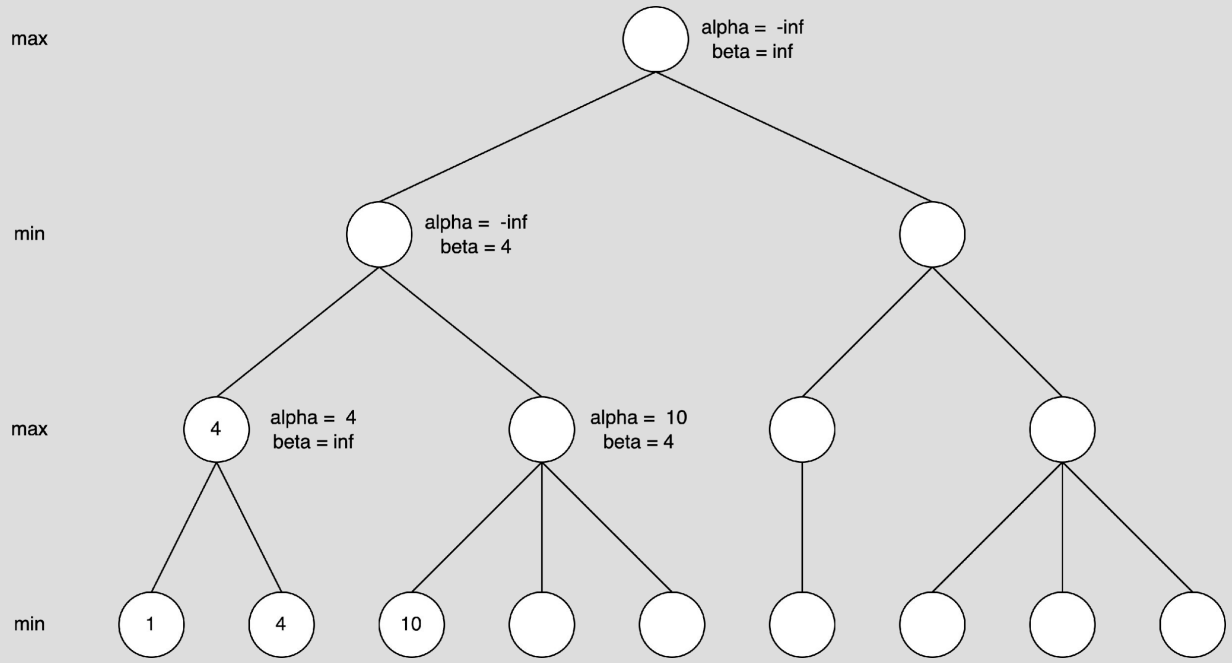
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



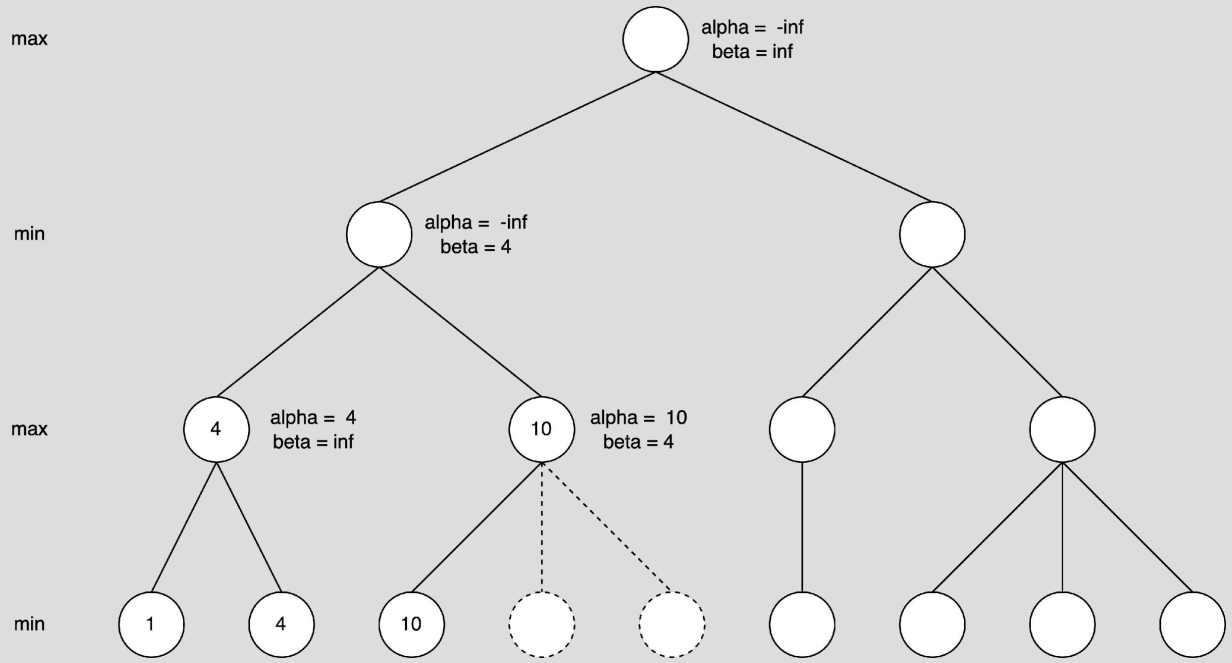
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



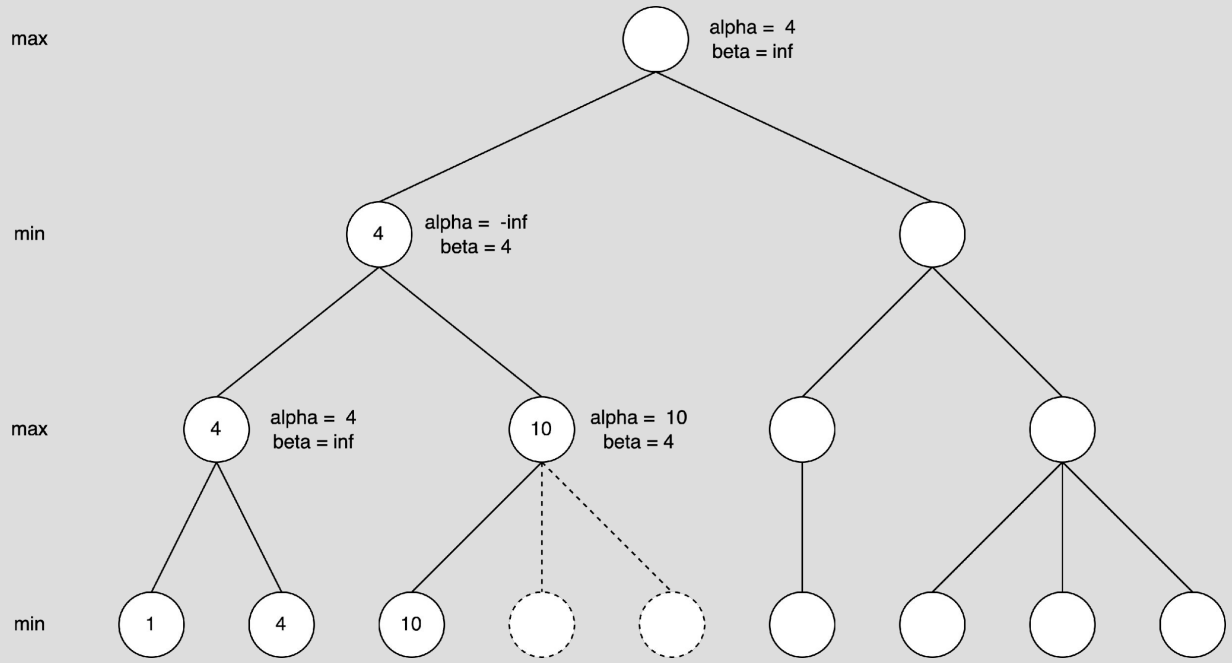
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



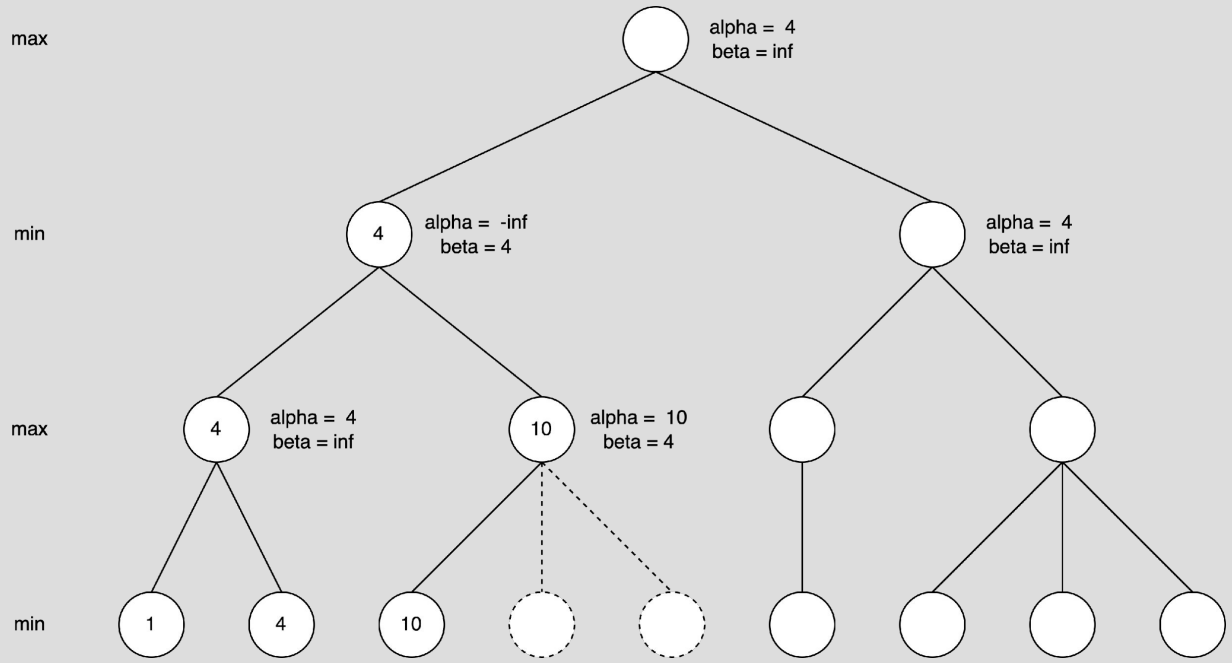
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



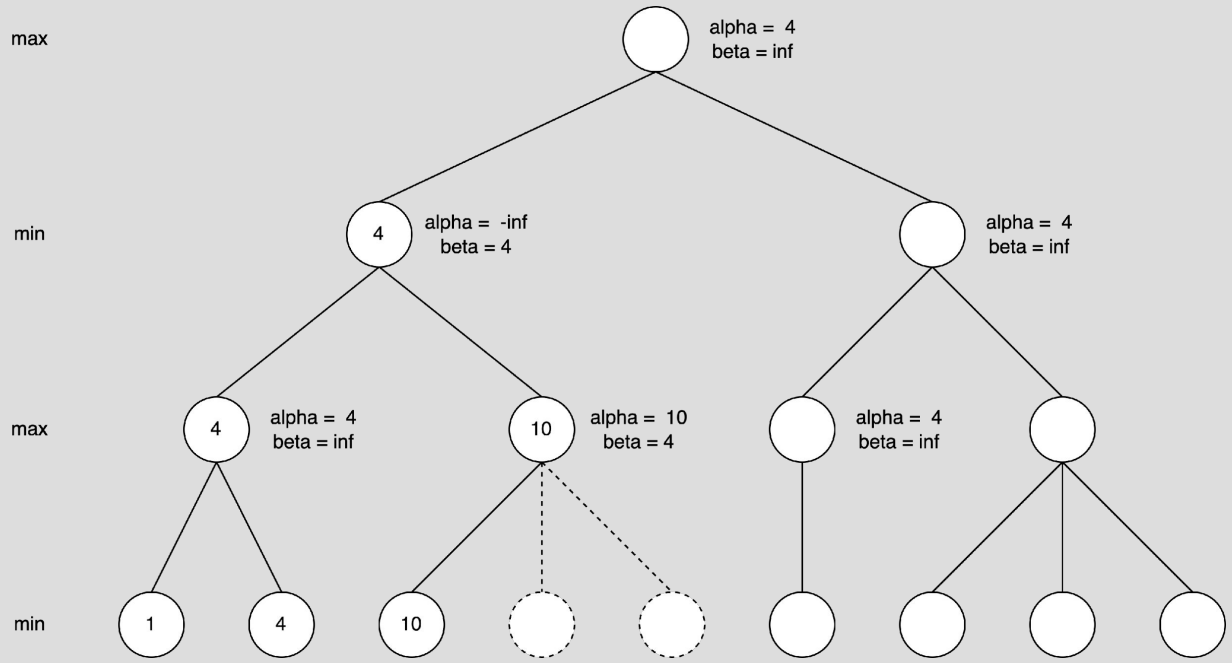
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



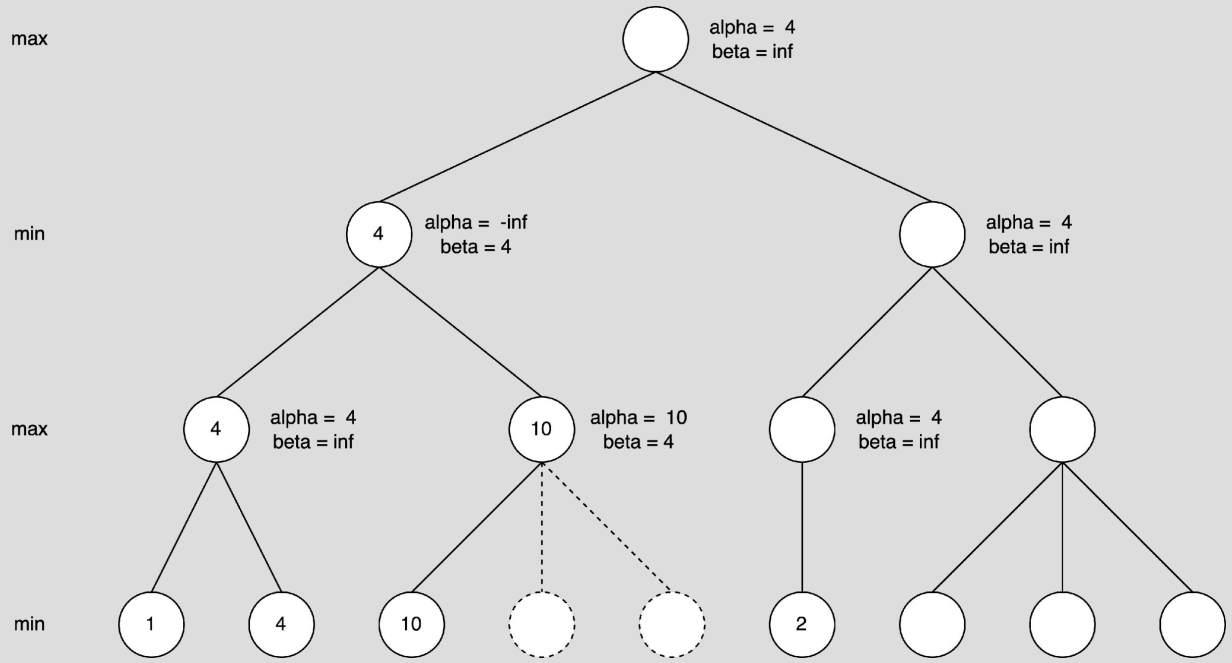
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



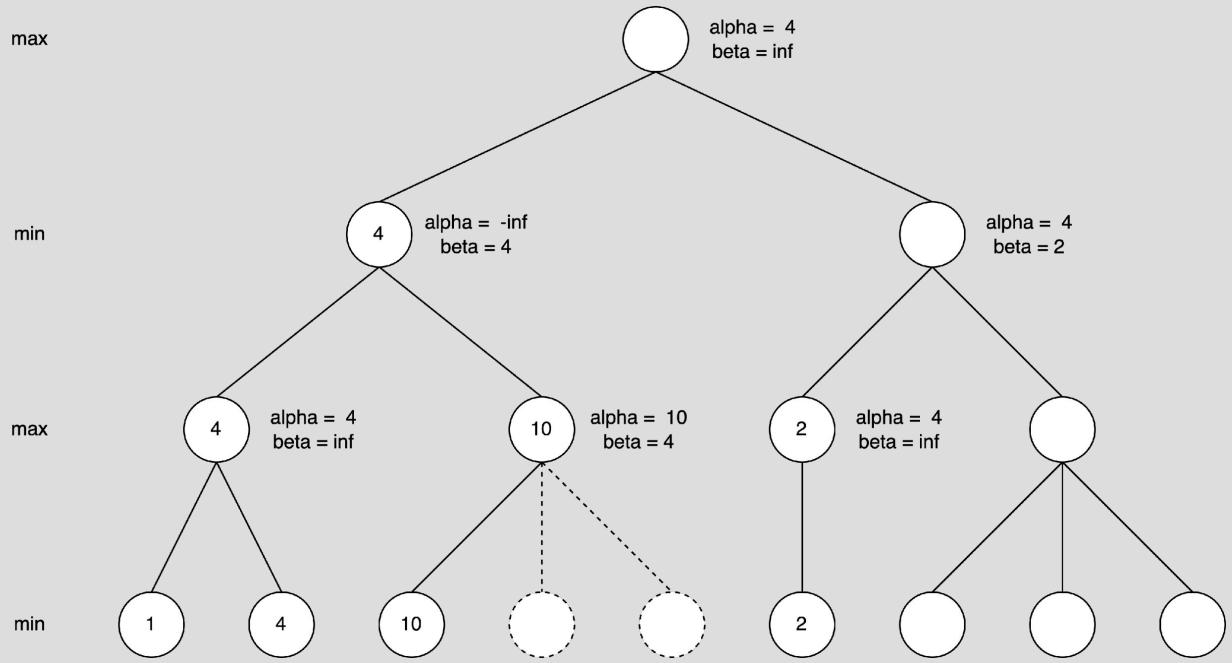
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



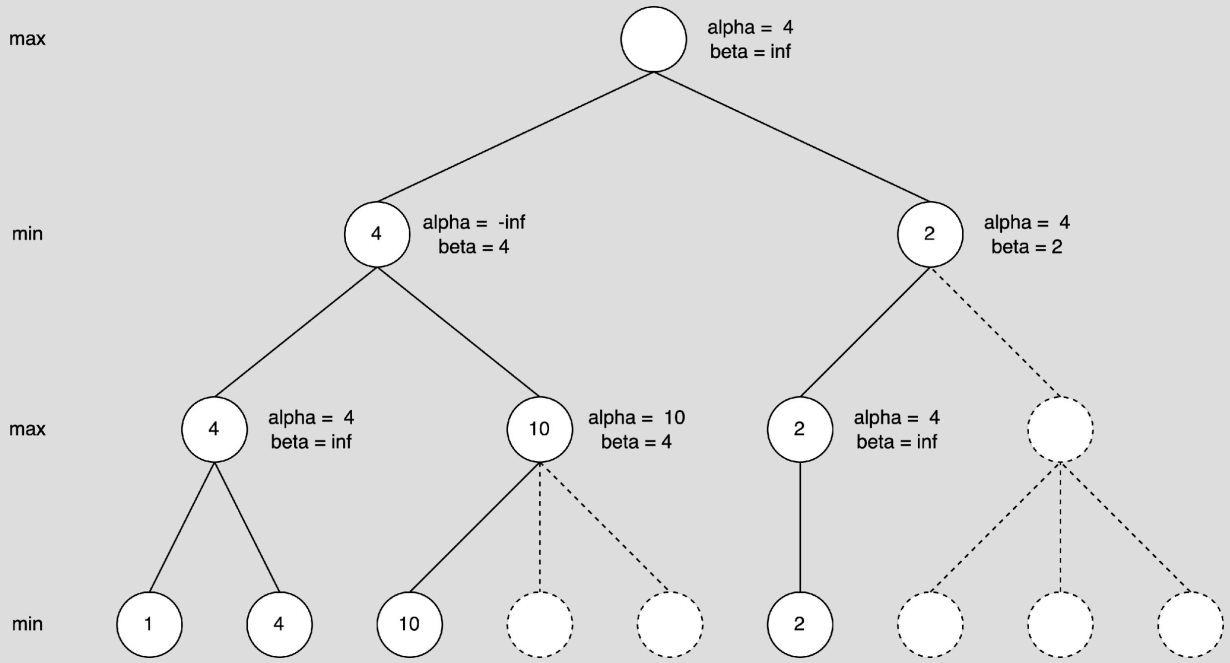
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



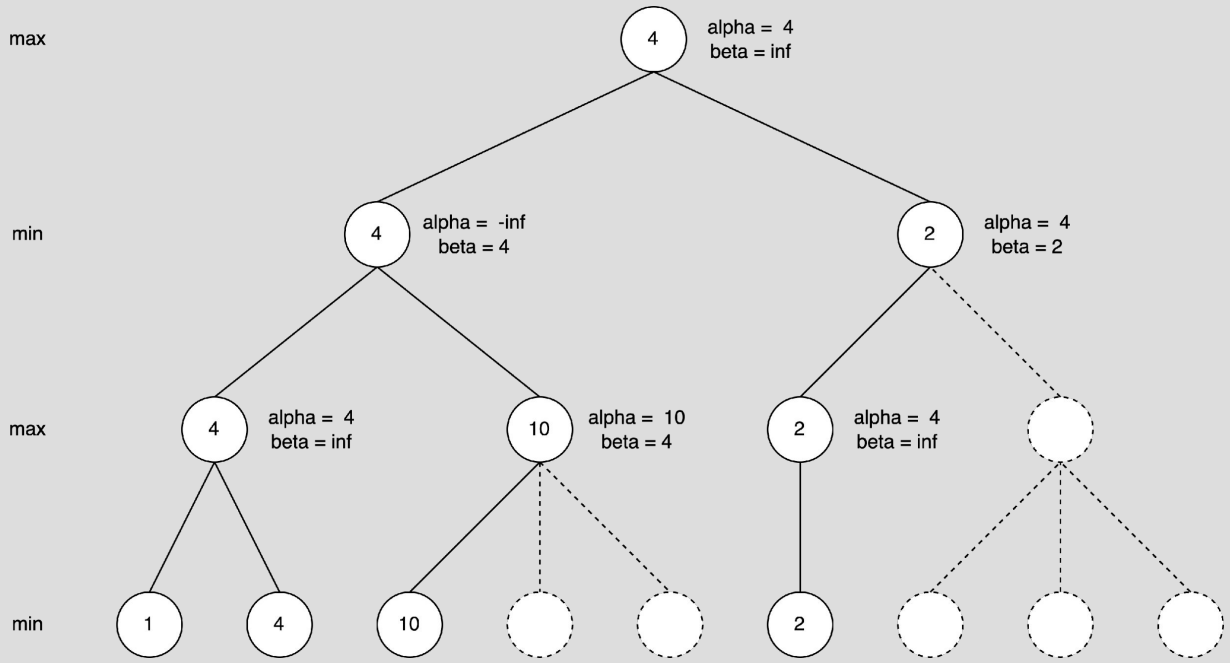
MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



MINIMAX WITH ALPHA BETA PRUNING EXAMPLE 2



CONSTRAINT SATISFACTION PROBLEMS

Week 5

AIMA, Chapter 6 "Constraint Satisfaction Problems" (6.1-6.5)

QUESTION TYPES

Definitions

- Variables, Constraints
- MRV
- LCV

Applications

- Graph Coloring
- Cryptarithmetic Puzzles
- Sudoku
- N-Queens

AC-3 EXAMPLE

Consider a constraint satisfaction problem (CSP) with three variables: X, Y, and Z, each with a domain of integers 1, 2, and 3. These variables are subject to the following binary constraints: $X = Y$, $Y < Z$.

Variables: $X = \{1, 2, 3\}$, $Y = \{1, 2, 3\}$, $Z = \{1, 2, 3\}$

Arcs:

$X = Y$

$Y = X$

$Y < Z$

$Z > Y$

AC-3 EXAMPLE

The algorithm starts by adding all arcs into the queue.

Variables: X = {1, 2, 3}, Y = {1, 2, 3}, Z = {1, 2, 3}

Queue:

X = Y

Y = X

Y < Z

Z > Y

Pop: X = Y

Queue:

Y = X

Y < Z

Z > Y

As every value in X satisfies the constraint X = Y, the domain of X remains unchanged.

AC-3 EXAMPLE

Variables: X = {1, 2, 3}, Y = {1, 2, 3}, Z = {1, 2, 3}

Pop: Y = X

Queue:

Y < Z

Z > Y

As every value in Y satisfies the constraint Y = X, the domain of Y remains unchanged.

AC-3 EXAMPLE

Variables: $X = \{1, 2, 3\}, Y = \{1, 2, 3\}, Z = \{1, 2, 3\}$

Pop: $Y < Z$

Queue:

$Z > Y$

The value 3 in Y fails to satisfy the constraint $Y < Z$, therefore, 3 is removed from the domain of Y to enforce arc consistency between Y and Z.

As Y lost a value, the arc $X = Y$ is added back to the queue, while the arc $Z > Y$ is not added back because it is the reverse of the arc $Y < Z$.

AC-3 EXAMPLE

Variables: $X = \{1, 2, 3\}, Y = \{1, 2\}, Z = \{1, 2, 3\}$

Queue:

$Z > Y$

$X = Y$

Pop: $Z > Y$

Queue:

$X = Y$

The value 1 in Z fails to satisfy the constraint $Z > Y$, therefore, 1 is removed from the domain of Z to enforce arc consistency between Z and Y.

As Z lost a value, arcs with Z on the right-hand side should be added back to the queue. However, the only such arc is $Y < Z$, which is the reverse of the arc $Z > Y$, thus no arc is added back.

AC-3 EXAMPLE

Variables: X = {1, 2, 3}, Y = {1, 2}, Z = {2, 3}

Pop: X = Y

Queue: empty

The value 3 in X fails to satisfy the constraint X = Y, therefore, 3 is removed from the domain of X to enforce arc consistency between X and Y.

As X lost a value, arcs with X on the right-hand side should be added back to the queue. However, the only such arc is Y = X, which is the reverse of the arc X = Y, thus no arc is added back.

AC-3 EXAMPLE

Variables: $X = \{1, 2\}, Y = \{1, 2\}, Z = \{2, 3\}$

As the queue is now empty, the AC-3 algorithm is complete.

The solution to this CSP is $X = \{1, 2\}, Y = \{1, 2\}, Z = \{2, 3\}$.

Binary constraints: $X = Y, Y < Z$

Consistent assignments of this CSP are:

- $X = 1, Y = 1, Z = 2$
- $X = 1, Y = 1, Z = 3$
- $X = 2, Y = 2, Z = 3$

LOGICAL AGENTS

Week 6

AIMA Chapter 7 "Logical Agents"

QUESTION TYPES

- Definitions
  - What is a Knowledge Base (KB)?
  - How does the Resolution algorithm work?
- Application
  - Formal logic
    - Propositional logic
      - Propositions (T, F)
      - Logical connectives ($\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$)
    - First-order logic
      - Quantifiers ($\forall$, $\exists$)
      - Predicates
        - planet(earth), orbits(earth, sun)
  - Inferences
    - Soundness
    - Completeness
  - Logical equivalence
  - Proof

$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \ldots \\
\textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \\
&\mid \neg \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
\text{OPERATOR PRECEDENCE} &: \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

LOGICAL EQUIVALENCE RULES

- Some are common sense.
- Some can be derived by a truth table, e.g.

p	q	p→q
T	T	T
T	F	F
F	T	T
F	F	T

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

CONJUNCTIVE NORMAL FORM (CNF)

- We can prove a statement must be true by contradiction.
- Apply rules to rewrite into CNF.
- Refer to textbook/ quiz for example.

$$
\begin{aligned}
\textit{CNFSentence} &\rightarrow \textit{Clause}_1 \wedge \cdots \wedge \textit{Clause}_n \\
\textit{Clause} &\rightarrow \textit{Literal}_1 \vee \cdots \vee \textit{Literal}_m \\
\textit{Fact} &\rightarrow \textit{Symbol} \\
\textit{Literal} &\rightarrow \textit{Symbol} \mid \neg \textit{Symbol} \\
\textit{Symbol} &\rightarrow P \mid Q \mid R \mid \ldots \\
\textit{HornClauseForm} &\rightarrow \textit{DefiniteClauseForm} \mid \textit{GoalClauseForm} \\
\textit{DefiniteClauseForm} &\rightarrow \textit{Fact} \mid (\textit{Symbol}_1 \wedge \cdots \wedge \textit{Symbol}_l) \Rightarrow \textit{Symbol} \\
\textit{GoalClauseForm} &\rightarrow (\textit{Symbol}_1 \wedge \cdots \wedge \textit{Symbol}_l) \Rightarrow \textit{False}
\end{aligned}
$$

CNF EXAMPLE

Original statement

$$(A \land B) \Leftrightarrow (C \lor D)$$

Biconditional elimination

$$\big((A \land B) \Rightarrow (C \lor D)\big) \land \big((C \lor D) \Rightarrow (A \land B)\big)$$

Implication elimination

$$\big(\neg(A \land B) \lor (C \lor D)\big) \land \big(\neg(C \lor D) \lor (A \land B)\big)$$

De Morgan

$$\big((\neg A \lor \neg B) \lor (C \lor D)\big) \land \big((\neg C \land \neg D) \lor (A \land B)\big)$$

CNF EXAMPLE

Associativity of $\vee$

$$(\neg A \vee \neg B \vee C \vee D) \wedge ((\neg C \wedge \neg D) \vee (A \wedge B))$$

Distributivity of $\vee$ over $\wedge$

$$(\neg A \vee \neg B \vee C \vee D) \wedge (((\neg C \wedge \neg D) \vee A) \wedge ((\neg C \wedge \neg D) \vee B))$$

Distributivity of $\vee$ over $\wedge$

$$(\neg A \vee \neg B \vee C \vee D) \wedge (((\neg C \vee A) \wedge (\neg D \vee A)) \wedge ((\neg C \vee B) \wedge (\neg D \vee B)))$$

Associativity of $\wedge$

$$(\neg A \vee \neg B \vee C \vee D) \wedge (\neg C \vee A) \wedge (\neg D \vee A) \wedge (\neg C \vee B) \wedge (\neg D \vee B)$$

Q&A